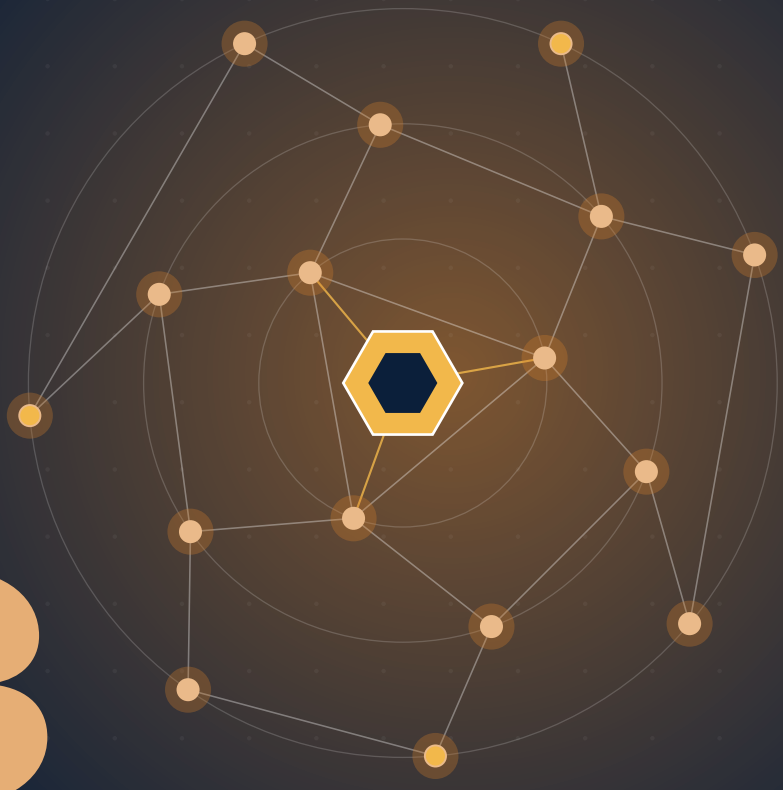
MACHINE LEARNING AND DEEP LEARNING

Foundations, Methods and Practice



CHAPTER

03

Advanced, Integrated, Practical and Research-Oriented Topics

36 sections · 42 figures · 47 tables · 15 algorithms · 63 numbered equations · 57 code listings

LUXSHI KARUNAKARAN

B.Sc. (Hons) Computer Science and Technology · M.Sc. Data Science and AI (Reading)
Department of Social Statistics, University of Sri Jayewardenepura, Sri Lanka

Foundations, Methods and Practice

CHAPTER 3

Advanced, Integrated, Practical and Research-Oriented Topics

36 sections · 42 figures · 47 tables · 15 algorithms · 63 numbered equations · 57 code listings

Luxshi Karunakaran

B.Sc. (Hons) in Computer Science and Technology
M.Sc. in Data Science and Artificial Intelligence (Reading)

Department of Social Statistics
University of Sri Jayewardenepura, Sri Lanka

Machine Learning and Deep Learning: Foundations, Methods and Practice
Chapter 3 – Advanced, Integrated, Practical and Research-Oriented Topics

Author. Luxshi Karunakaran, B.Sc. (Hons) in Computer Science and Technology; M.Sc. in Data Science and Artificial Intelligence (Reading). Department of Social Statistics, University of Sri Jayewardenepura, Sri Lanka.

Copyright. © 2026 Luxshi Karunakaran. All rights reserved.

Suggested citation. Karunakaran, L. (2026). Advanced, integrated, practical and research-oriented topics. In *Machine Learning and Deep Learning: Foundations, Methods and Practice* (Chapter 3). Department of Social Statistics, University of Sri Jayewardenepura, Sri Lanka.

Edition. Version 1.0, October 2026. Distributed as a book-chapter preprint via ResearchGate.

Conventions. Sections, figures, tables, equations, algorithms, code listings and highlight panels are numbered in the form *chapter.sequence*; see *How to Read This Chapter* for details. Code listings are written in Python 3.

Typesetting. Typeset with XeLaTeX in TeX Gyre Pagella, Inter and DejaVu Sans Mono.

Contents

How to Read This Chapter xiv

Chapter Overview 1

Part A: Advanced and Integrated Topics 5

3.1 Statistical Foundations for Machine Learning and Deep Learning 6

3.1.1 Descriptive Statistics and Sampling 6

3.1.2 Point Estimation: Bias, Variance and Consistency 7

3.1.3 Maximum Likelihood and Maximum A Posteriori Estimation 8

3.1.4 Hypothesis Testing and Confidence Intervals 9

3.2 Linear Algebra and Calculus for AI 11

3.2.1 Vectors, Matrices and Norms 11

3.2.2 Eigen-Decomposition and the Singular Value Decomposition 12

3.2.3 Derivatives, Gradients, Jacobians and Hessians 14

3.2.4 The Chain Rule and Backpropagation 14

3.3 Probability and Statistics for AI 16

3.3.1 Random Variables and Probability Distributions 17

3.3.2 Bayes' Theorem 18

3.3.3 Expectation, Variance and Covariance 19

3.3.4 Law of Large Numbers and Central Limit Theorem 20

3.4 Information Theory and Entropy 21

3.4.1 Entropy 21

3.4.2 Joint and Conditional Entropy 22

3.4.3 Cross-Entropy and Kullback–Leibler Divergence 23

3.4.4 Mutual Information 24

3.5 Data Engineering for Machine Learning 26

3.5.1 ETL and ELT Pipelines 26

3.5.2 Batch and Streaming Processing 27

3.5.3 Data Lakes, Warehouses and Lakehouses 27

3.5.4 Data Quality and Validation 28

3.6 Big Data Analytics and Machine Learning 30

3.6.1 The Five Vs of Big Data 30

3.6.2 Hadoop and the MapReduce Programming Model 30

3.6.3 Apache Spark and MLlib 32

3.7 Feature Stores and Data Versioning 34

3.7.1 Training–Serving Skew 34

3.7.2 Feature Store Architecture . . . 34
3.7.3 Data Versioning with DVC and Related Tools . . . 36

3.8 Model Selection and Automated Hyperparameter Optimization 37
3.8.1 Information Criteria: AIC and BIC . . . 37
3.8.2 Bayesian Optimisation . . . 38
3.8.3 Successive Halving and Hyperband . . . 40

3.9 Causal Machine Learning 42
3.9.1 Correlation Versus Causation . . . 43
3.9.2 Potential Outcomes and Treatment Effects . . . 43
3.9.3 Causal Graphs, the do-Operator and Backdoor Adjustment . . . 44
3.9.4 Propensity Scores . . . 45
3.9.5 Uplift Modelling and Double Machine Learning . . . 46

3.10 Federated Learning 48
3.10.1 Architecture and Training Round . . . 48
3.10.2 The FedAvg Algorithm . . . 49
3.10.3 Horizontal, Vertical and Cross-Silo Settings . . . 50
3.10.4 Challenges: Non-IID Data, Communication and Privacy . . . 50

3.11 Online and Incremental Learning 52
3.11.1 The Online Learning Protocol and Online Gradient Descent . . . 52
3.11.2 Regret . . . 53
3.11.3 Concept Drift . . . 53
3.11.4 Incremental Learning in Practice . . . 55

3.12 Continual and Lifelong Learning 56
3.12.1 Catastrophic Forgetting and the Stability–Plasticity Dilemma . . . 56
3.12.2 Regularisation-Based Methods: Elastic Weight Consolidation . . . 57
3.12.3 Replay and Parameter-Isolation Methods . . . 57

3.13 Few-Shot and Zero-Shot Learning 60
3.13.1 The N-Way K-Shot Problem . . . 60
3.13.2 Metric-Based Approaches: Siamese and Prototypical Networks . . . 61
3.13.3 Zero-Shot Learning . . . 62

3.14 Meta-Learning 64
3.14.1 Problem Formulation and Taxonomy . . . 64
3.14.2 Model-Agnostic Meta-Learning (MAML) . . . 64
3.14.3 Strengths, Limitations and Connections . . . 66

3.15 Retrieval-Augmented Generation (RAG) 67
3.15.1 The RAG Pipeline . . . 67
3.15.2 Chunking, Embeddings and Similarity . . . 68

3.15.3 Vector Databases 68
3.15.4 Evaluating and Improving RAG Systems 69

3.16 AI Agents and Tool-Using Models 71
3.16.1 The Agent Loop and the ReAct Pattern 71
3.16.2 Tool Calling, Planning and Memory 72
3.16.3 Multi-Agent Systems 73
3.16.4 Risks and Safeguards 74

3.17 Responsible AI and AI Governance 75
3.17.1 Principles of Responsible AI 75
3.17.2 Regulation and Standards: The EU AI Act and NIST AI RMF 76
3.17.3 Documentation: Model Cards and Datasheets 77

3.18 Privacy-Preserving Machine Learning 78
3.18.1 Differential Privacy 78
3.18.2 Differentially Private Training: DP-SGD 80
3.18.3 Homomorphic Encryption and Secure Multi-Party Computation 81

3.19 Green AI and Energy-Efficient Computing 82
3.19.1 Estimating Energy Use and Carbon Footprint 83
3.19.2 Strategies for Energy-Efficient AI 83

3.20 Real-World AI Case Studies and End-to-End Projects 85
3.20.1 A Common End-to-End Lifecycle 85
3.20.2 Case Studies 86
3.20.3 Cross-Cutting Lessons 88

Part B: Practical and Research-Oriented Chapters 89

3.21 Setting Up Python, Jupyter Notebook, and Google Colab 90
3.21.1 The Python Data Science Stack 90
3.21.2 Local Installation Step by Step 90
3.21.3 Virtual Environments and Conda 91
3.21.4 Working in Jupyter Notebook 93
3.21.5 Google Colab: GPU Runtimes and Drive Mounting 93

3.22 Data Analysis with NumPy and Pandas 94
3.22.1 NumPy Arrays 94
3.22.2 Broadcasting and Vectorisation 95
3.22.3 Pandas DataFrames 96

3.23 Data Visualization with Matplotlib and Other Tools 98
3.23.1 Choosing the Right Chart 99
3.23.2 Matplotlib Fundamentals 99

3.23.3 Scatter Plots and Heatmaps . . . 100
3.23.4 Seaborn and Plotly . . . 101

3.24 Building Machine Learning Models with Scikit-learn 102
3.24.1 The Estimator API . . . 102
3.24.2 Pipelines and ColumnTransformer . . . 103
3.24.3 A Complete Scikit-learn Workflow . . . 105

3.25 Machine Learning Workflows in KNIME 107
3.25.1 Nodes, Ports and Workflows . . . 107
3.25.2 A Classification Workflow . . . 107
3.25.3 Building the Workflow Step by Step . . . 107

3.26 Building Neural Networks with TensorFlow and Keras 110
3.26.1 The Sequential API . . . 110
3.26.2 The Functional API . . . 111
3.26.3 Callbacks and Training Control . . . 113

3.27 Building Neural Networks with PyTorch 114
3.27.1 Tensors and Automatic Differentiation . . . 114
3.27.2 Defining Models with nn.Module . . . 115
3.27.3 Datasets and DataLoaders . . . 116
3.27.4 The Training Loop . . . 117

3.28 Dataset Selection and Data Annotation 119
3.28.1 Criteria for Selecting a Dataset . . . 119
3.28.2 Annotation Process and Tools . . . 119
3.28.3 Inter-Annotator Agreement and Cohen's Kappa . . . 121
3.28.4 Active Learning . . . 122

3.29 Benchmark Datasets and Experimental Design 123
3.29.1 Influential Benchmark Datasets . . . 123
3.29.2 Baselines . . . 124
3.29.3 Controls, Ablations and Experimental Protocol . . . 125

3.30 Reproducibility and Experiment Tracking 126
3.30.1 Controlling Randomness . . . 126
3.30.2 Pinning the Environment and Versioning Data . . . 127
3.30.3 Experiment Tracking with MLflow and Weights & Biases . . . 128
3.30.4 A Reproducibility Checklist . . . 129

3.31 Model Comparison and Statistical Significance Testing 130
3.31.1 Choosing a Test . . . 130
3.31.2 Paired t-Test and the Corrected Resampled t-Test . . . 130
3.31.3 McNemar's Test . . . 132

3.31.4 Wilcoxon Signed-Rank Test 133
3.31.5 Friedman Test, Nemenyi Post-hoc and Critical-Difference Diagrams 134

3.32 Interpreting Experimental Results 136
3.32.1 Confidence Intervals 137
3.32.2 Effect Sizes 138
3.32.3 Error Analysis 138
3.32.4 Learning Curves 138
3.32.5 Reporting Results 139

3.33 Model Deployment Using APIs and Web Applications 140
3.33.1 Serving Patterns 140
3.33.2 A REST API with FastAPI 141
3.33.3 Interactive Web Apps with Streamlit and Gradio 143
3.33.4 Containerising with Docker 144
3.33.5 Deployment Architecture 145

3.34 End-to-End Machine Learning Project 146
3.34.1 Problem Framing 146
3.34.2 Exploratory Data Analysis 147
3.34.3 Preprocessing and Baselines 149
3.34.4 Model Training and Tuning 150
3.34.5 Evaluation 150
3.34.6 Choosing the Decision Threshold 151
3.34.7 Interpretation 152
3.34.8 Deployment and Monitoring 153

3.35 End-to-End Deep Learning Project 153
3.35.1 Data Preparation and Augmentation 154
3.35.2 Model: Transfer Learning with a Pretrained Backbone 155
3.35.3 Training 156
3.35.4 Evaluation 157
3.35.5 Explaining Predictions with Grad-CAM 159
3.35.6 Exporting the Model 159

3.36 Research Trends, Open Problems, and Future Directions 160
3.36.1 Current Trends 160
3.36.2 Open Problems 161
3.36.3 Writing a Machine Learning Research Paper 161
3.36.4 Choosing a Venue 161
3.36.5 Ethics Statements and Responsible Research 163

Chapter Summary 164

Key Terms 165

Review Questions 167
Exercises and Mini-Projects 168
Further Reading 169

List of Figures

3.1 Forty 95% confidence intervals for a population mean $\mu = 0$, each computed from an independent sample of size 20 . . . 10

3.2 Geometric view of the SVD of a 2×2 matrix $A = U\Sigma V^T$: the unit circle is rotated by V^T, stretched along the axes by Σ and rotated by U into an ellipse whose semi-axes are the singular values . . . 13

3.3 Contours of $f(x, y) = x^2 + 8y^2$ with negative-gradient direction arrows (unit length) and the zig-zag path of gradient descent ($\eta = 0.11$) from $(-4.5, 1.5)$ . . . 15

3.4 Left: Gaussian densities for different means and standard deviations . . . 18

3.5 Central limit theorem in action . . . 20

3.6 Binary entropy $H_b(p)$ in bits, together with the Gini impurity $2p(1-p)$ scaled by 2 for comparison . . . 22

3.7 Information diagram for two variables . . . 25

3.8 ETL transforms data before loading them into the target store, whereas ELT loads raw data first and transforms them inside a scalable warehouse or lakehouse, keeping the raw copy available for re-processing . . . 26

3.9 Data flow of a MapReduce job: input splits are processed by parallel mappers, intermediate key–value pairs are shuffled and sorted by key, and reducers aggregate each key's values . . . 32

3.10 Feature store architecture: one set of feature definitions populates an offline store for point-in-time-correct training data and an online store for low-latency serving, eliminating training–serving skew . . . 35

3.11 Bayesian optimisation in one dimension . . . 39

3.12 Successive halving with 27 configurations and $\eta = 3$: after each rung only the best third of configurations survive and receive three times the training budget, so most compute goes to promising candidates . . . 41

3.13 The three elementary causal structures . . . 44

3.14 Federated learning architecture . . . 48

3.15 Temporal patterns of concept drift . . . 54

3.16 Accuracy on task A while a network is trained sequentially on tasks A, B and C . . . 56

3.17 Prototypical network in a learned two-dimensional embedding space (3-way 5-shot) 62

3.18 Geometric intuition for MAML . . . 66

3.19 Retrieval-augmented generation . . . 67

3.20 The agent loop. An LLM policy reasons over the goal, context and memory, selects a tool action, observes the result returned by the environment and iterates until it produces a final answer . . . 72

3.21 The four risk tiers of the EU AI Act . . . 77
3.22 Laplace mechanism for a counting query ($\Delta f = 1$) . . . 79
3.23 Estimated emissions of a 10 MWh training run under different grid carbon intensities (illustrative values) . . . 84
3.24 The end-to-end lifecycle of an applied AI project . . . 86
3.25 Layers of a typical machine learning environment . . . 91
3.26 Broadcasting a $(4, 3)$ matrix with a $(3,)$ vector: the vector is virtually repeated along the first axis, so the subtraction is element-wise without any copying . . . 95
3.27 Left: a histogram reveals the right skew of a log-normally distributed income variable (median dashed) . . . 100
3.28 Left: a scatter plot coloured by class shows two partially overlapping groups . . . 101
3.29 A scikit-learn Pipeline wrapping a ColumnTransformer . . . 104
3.30 A node-based classification workflow as built in KNIME . . . 108
3.31 A multi-input, multi-output model expressible with the Keras Functional API: an image branch and a tabular branch are concatenated, then feed a classification head and a regression head . . . 112
3.32 The iterative annotation workflow . . . 120
3.33 A rigorous experimental protocol . . . 125
3.34 Critical-difference diagram for the worked example ($k = 4$, $N = 10$, $CD = 1.48$ at $\alpha = 0.05$) . . . 135
3.35 Learning curves for a high-bias model (left), whose curves converge to a high error, and a high-variance model (right), whose large gap and still-falling validation error suggest that more data would help . . . 139
3.36 A typical online deployment architecture . . . 146
3.37 Roadmap of the end-to-end churn project . . . 147
3.38 Exploratory views of the churn data . . . 148
3.39 Two-phase transfer learning . . . 155
3.40 Training curves for two-phase transfer learning on the flower dataset (representative run) . . . 157
3.41 Test-set confusion matrix for the fine-tuned ResNet-50 (rows: true class, columns: predicted) . . . 158
3.42 The research cycle. Writing is not a final step but part of an iterative loop . . . 162

List of Tables

3.1 Roadmap of Chapter 3: Part A (advanced and integrated topics) and Part B (practical and research-oriented chapters) . . . 3
3.2 Common descriptive statistics and their typical uses in machine learning . . . 7
3.3 Frequently used statistical tests in machine learning research . . . 9
3.4 Vector and matrix norms used in machine learning . . . 12
3.5 Useful matrix-calculus identities (vectors x, a; symmetric matrix A where noted) . . . 16
3.6 Common probability distributions in machine learning . . . 17
3.7 Information-theoretic quantities and where they appear in machine learning . . . 24
3.8 Batch versus stream processing for machine learning workloads . . . 27
3.9 Comparison of data warehouses, data lakes and lakehouses . . . 28
3.10 The five Vs of big data and their implications for machine learning . . . 31
3.11 Big-data processing frameworks relevant to machine learning . . . 33
3.12 Offline versus online feature stores . . . 36
3.13 Tools for data and model versioning . . . 37
3.14 Hyperparameter optimisation strategies compared . . . 41
3.15 Meta-learners for estimating conditional average treatment effects . . . 46
3.16 Types of federated learning . . . 50
3.17 Representative online and incremental learning algorithms . . . 54
3.18 Families of continual learning methods . . . 58
3.19 Approaches to learning from few or no labelled examples . . . 63
3.20 Taxonomy of meta-learning methods . . . 64
3.21 Approximate nearest-neighbour index types used in vector databases . . . 69
3.22 Evaluation metrics for retrieval-augmented generation . . . 70
3.23 Components of LLM-based agent systems . . . 74
3.24 Core principles of responsible AI and corresponding technical practices . . . 76
3.25 Major AI governance frameworks . . . 77
3.26 Comparison of privacy-preserving machine learning techniques . . . 82
3.27 Strategies for reducing the energy and carbon cost of AI . . . 84
3.28 Summary of the five case studies . . . 86
3.29 Recommended installation sequence for a local machine learning environment . . . 92
3.30 Frequently used pandas operations and their purpose . . . 97
3.31 Choosing a chart type according to the analytical question . . . 99
3.32 Core methods of the scikit-learn estimator interface . . . 103

3.33 Core KNIME nodes for a supervised classification workflow . . . 108
3.34 Frequently used Keras callbacks . . . 113
3.35 Comparison of Keras/TensorFlow and PyTorch for everyday work . . . 118
3.36 Criteria for selecting a dataset for a machine learning project . . . 120
3.37 Widely used benchmark datasets in machine learning research . . . 124
3.38 Example ablation study for an image classifier (mean ± standard deviation of top-1 accuracy over 5 seeds on a validation set) . . . 125
3.39 Reproducibility checklist for machine learning experiments . . . 129
3.40 Statistical tests for comparing machine learning models . . . 131
3.41 Templates for reporting experimental results . . . 140
3.42 Tools for deploying machine learning models . . . 141
3.43 Churn prediction results (5-fold CV on the development set; single evaluation on the held-out test set of 1,400 customers; threshold 0.5 with balanced class weights) . . . 151
3.44 Per-class test results of the fine-tuned ResNet-50 (n = 550; derived from the confusion matrix) . . . 158
3.45 Open problems in machine learning research . . . 162
3.46 Structure of a machine learning research paper (adapted IMRaD) . . . 163
3.47 Key terms introduced in Chapter 3 . . . 165

List of Algorithms

3.1 Data Validation Gate in an ML Pipeline . . . 29
3.2 MapReduce Word Count . . . 31
3.3 Bayesian Optimisation with a Gaussian-Process Surrogate . . . 39
3.4 Hyperband (with Successive Halving as the inner loop) . . . 40
3.5 Double Machine Learning with Cross-Fitting (Partially Linear Model) . . . 46
3.6 Federated Averaging (FedAvg) . . . 49
3.7 Online Gradient Descent on a Data Stream . . . 52
3.8 Experience Replay with a Reservoir Buffer . . . 58
3.9 Model-Agnostic Meta-Learning (MAML) for Few-Shot Supervised Learning . . . 65
3.10 ReAct-Style Tool-Using Agent Loop . . . 71
3.11 Differentially Private Stochastic Gradient Descent (DP-SGD) . . . 80
3.12 Grid Search with k-Fold Cross-Validation . . . 104
3.13 Mini-batch Training Loop in PyTorch . . . 117
3.14 Pool-based Active Learning with Uncertainty Sampling . . . 122
3.15 Comparing k Algorithms over N Datasets (Friedman + Nemenyi) . . . 134

List of Code Listings

3.1 Confidence intervals and a paired t-test with NumPy and SciPy. 10
3.2 Eigen-decomposition, SVD, low-rank approximation and gradient checking. 16
3.3 Computing entropy, cross-entropy, KL divergence and mutual information. 24
3.4 Simple data-quality checks and the population stability index in pandas. 29
3.5 A churn-prediction pipeline with PySpark MLlib. 32
3.6 Defining and retrieving features with Feast. 35
3.7 Hyperparameter optimisation with Optuna using TPE sampling and Hyperband pruning. 42
3.8 Estimating heterogeneous treatment effects with double machine learning (EconML). 47
3.9 A minimal simulation of FedAvg for logistic regression on non-IID clients. 51
3.10 Incremental learning with partial_fit, prequential evaluation and ADWIN drift detection. 55
3.11 Computing a diagonal Fisher estimate and the EWC penalty in PyTorch. 59
3.12 Loss for one prototypical-network episode in PyTorch. 62
3.13 A minimal retrieval-augmented generation pipeline with Sentence-Transformers and FAISS. 70
3.14 A minimal tool-calling agent loop (provider-agnostic sketch). 73
3.15 Differentially private training with Opacus. 81
3.16 Measuring the energy and emissions of a training run with CodeCarbon. 85
3.17 Creating, exporting and registering a conda environment (shell commands). 92
3.18 Typical first cell in Google Colab: check the GPU, mount Drive and install extras. 93
3.19 Array creation, broadcasting and a vectorised pairwise-distance computation. 96
3.20 A typical pandas workflow: merge, derive, group, aggregate and pivot. 97
3.21 Histogram and grouped box plot with Matplotlib's object-oriented API. 99
3.22 Exploratory plots with Seaborn (static) and Plotly Express (interactive). 101
3.23 A leak-free scikit-learn pipeline with ColumnTransformer and GridSearchCV. 105
3.24 Inspecting, saving and reloading a fitted pipeline. 106
3.25 A Sequential multilayer perceptron for Fashion-MNIST in Keras. 110
3.26 A two-input, two-output network with the Functional API. 111
3.27 Training with a tf.data pipeline, callbacks and saving the best model. 113
3.28 Tensors, devices and automatic differentiation in PyTorch. 115
3.29 A convolutional classifier defined as an nn.Module. 115
3.30 A custom Dataset and DataLoader for labelled image files. 116

3.31 A complete training and validation loop with mixed precision and checkpointing. 117
3.32 Cohen's kappa and a confusion matrix between two annotators. 121
3.33 A helper that seeds all common random number generators and requests deterministic GPU kernels. 126
3.34 Pinning dependencies, versioning data with DVC and tagging the code state (shell). 127
3.35 Tracking a scikit-learn experiment with MLflow. 128
3.36 Logging a PyTorch training run and its configuration with Weights & Biases. 129
3.37 Paired t-test, corrected resampled t-test and Wilcoxon test with SciPy. 132
3.38 McNemar's test from the predictions of two classifiers. 133
3.39 Friedman test, average ranks and Nemenyi critical difference. 136
3.40 Percentile bootstrap confidence intervals for any metric, including a paired difference. 137
3.41 Computing a learning curve with scikit-learn. 139
3.42 Serving a scikit-learn pipeline as a validated REST API with FastAPI (file `app.py`). 141
3.43 Calling the prediction API from Python (the curl equivalent is shown as a comment). 143
3.44 A Streamlit front end for the churn model (run with `streamlit run churn_app.py`). 143
3.45 A Gradio demo wrapping an image classifier. 144
3.46 Dockerfile for the FastAPI churn service, with build and run commands. 145
3.47 First exploratory checks on the churn data. 148
3.48 Train–test split, preprocessing and candidate models for the churn project. 149
3.49 Cross-validating all candidate pipelines. 150
3.50 Randomised hyperparameter search for gradient boosting. 150
3.51 Selecting a cost-optimal threshold from out-of-fold predictions. 152
3.52 Global and local explanations: coefficients, permutation importance and SHAP. 152
3.53 Data splits, augmentation pipelines and loaders for the flower classifier. 154
3.54 Building the transfer-learning model and the two optimiser configurations. 155
3.55 Two-phase training with early stopping and history tracking. 156
3.56 Test-set evaluation: classification report and confusion matrix. 158
3.57 Exporting to TorchScript and ONNX and verifying the exported model. 160

How to Read This Chapter

Every element in this chapter carries a label and a number so that it can be identified, cited and cross-referenced precisely. Numbers restart in each chapter and take the form *chapter.sequence*. The conventions are summarised in the table below.

Element	Label format	Purpose
Section / Subsection	3.4 / 3.4.2	Numbered heading; subsections are nested inside sections.
Figure	Figure 3.1	Diagram or chart; caption appears below the figure.
Table	Table 3.1	Structured comparison or summary; caption appears above the table.
Equation	(3.1)	Display equation; number at the right margin; symbols explained beneath.
Algorithm	Algorithm 3.1	Pseudocode with inputs, outputs and numbered steps.
Listing	Listing 3.1	Runnable Python code in a shaded panel.
Definition	Definition 3.1	Formal definition of a key term (blue panel).
Worked Example	Worked Example 3.1	Step-by-step numerical illustration (green panel).
Case Study	Case Study 3.1	Real-world application (orange panel).
Key Point / Note / Practice Tip	—	Unnumbered highlight panels: takeaways, caveats and practical advice.

CHAPTER 3

Advanced, Integrated, Practical and Research-Oriented Topics



36 sections · 42 figures · 47 tables · 15 algorithms · 63 numbered equations · 57 code listings

Chapter Overview

The first two chapters of this book built the core of machine learning and deep learning: the learning problem, classical supervised and unsupervised algorithms, model evaluation, neural networks, convolutional and recurrent architectures, transformers and generative models. This final chapter widens the lens. It gathers the mathematical foundations that make those methods work, the engineering infrastructure that makes them usable at scale, the newer learning paradigms that extend them beyond the single static dataset, and the practical and research skills needed to turn them into reliable systems and publishable results.

The chapter is organised in two parts. **Part A: Advanced and Integrated Topics** begins with a compact but rigorous treatment of statistics, linear algebra, calculus, probability and information theory, written so that each idea is tied directly to a machine learning use. It then turns to the data side of the discipline (data engineering, big data analytics, feature stores and data versioning) and to automated model selection. The middle of Part A surveys learning paradigms that relax the assumptions of standard supervised learning: causal machine learning, federated learning, online and continual learning, few-shot and zero-shot learning, and meta-learning. Part A closes with topics that dominate contemporary practice: retrieval-augmented generation, tool-using AI agents, responsible AI and governance, privacy-preserving machine learning, energy-efficient computing and a set of end-to-end case studies.

Part B: Practical and Research-Oriented Chapters is hands-on. It walks through the

working environment (Python, Jupyter and Google Colab), the core scientific libraries (NumPy, Pandas, Matplotlib, scikit-learn), visual workflow tools (KNIME), the two dominant deep learning frameworks (TensorFlow/Keras and PyTorch), and then the research workflow itself: choosing and annotating datasets, designing experiments on benchmarks, ensuring reproducibility, comparing models with proper statistical tests, interpreting results, deploying models, and completing full machine learning and deep learning projects. The chapter ends with a survey of research trends and open problems.

Readers can use the chapter in different ways. Those preparing for research may read Part A in order, because later sections (for example, Bayesian optimisation, differential privacy and federated learning) rely on the foundations laid in the first four sections. Practitioners may move directly to the relevant Part B section and return to Part A for the underlying theory. Each section is largely self-contained and cross-refers to Chapters 1 and 2 where a method was introduced earlier.

Learning Objectives

After studying this chapter, you should be able to:

- ✓ Apply descriptive and inferential statistics, maximum likelihood and MAP estimation, hypothesis tests and confidence intervals to machine learning problems.
- ✓ Use vectors, matrices, eigen-decomposition, the singular value decomposition and multivariable calculus (gradients, Jacobians, Hessians, chain rule) to analyse learning algorithms.
- ✓ Reason with random variables, common probability distributions, Bayes' theorem and the central limit theorem, and compute entropy, cross-entropy, KL divergence and mutual information.
- ✓ Design data pipelines using ETL/ELT, batch and streaming processing, data lakes, warehouses and lakehouses, and explain how feature stores and data versioning prevent training–serving skew.
- ✓ Describe distributed processing with MapReduce and Apache Spark and train models with Spark MLlib.
- ✓ Select models with information criteria and automate hyperparameter search with Bayesian optimisation, successive halving and Hyperband.
- ✓ Distinguish correlation from causation and estimate causal effects with potential outcomes, causal graphs, backdoor adjustment, propensity scores and double machine learning.
- ✓ Explain federated, online, continual, few-shot, zero-shot and meta-learning, including FedAvg, elastic weight consolidation, prototypical networks and MAML.
- ✓ Build retrieval-augmented generation pipelines and tool-using AI agents, and evaluate their risks.
- ✓ Apply principles of responsible AI, regulatory frameworks (EU AI Act, NIST AI RMF) and privacy-preserving techniques such as differential privacy and secure computation.

- ✓ Estimate and reduce the energy and carbon cost of training and deploying models.
- ✓ Set up a reproducible Python environment and carry out complete, statistically sound machine learning and deep learning projects from data to deployment.

Prerequisites

This chapter assumes the material of Chapters 1 and 2. From Chapter 1 you should be comfortable with the supervised learning set-up, loss functions, gradient descent, the bias–variance trade-off, cross-validation, evaluation metrics, and classical algorithms such as linear and logistic regression, decision trees, ensembles, support vector machines and clustering. From Chapter 2 you should know how feed-forward, convolutional and recurrent networks are trained with backpropagation, how attention and transformers work, and how large language models are pre-trained and fine-tuned. Basic programming in Python and first-year university mathematics (algebra, elementary calculus and probability) are also assumed; the foundations sections of Part A revise and extend the mathematics as needed.

Chapter Roadmap

Table 3.1 lists the sections of the chapter with a brief statement of the purpose of each. Sections 1–20 form Part A and sections 21–36 form Part B.

Table 3.1 Roadmap of Chapter 3: Part A (advanced and integrated topics) and Part B (practical and research-oriented chapters).

Part	Section	Focus
A	1. Statistical Foundations for Machine Learning and Deep Learning	Estimation, MLE/MAP, tests, intervals
A	2. Linear Algebra and Calculus for AI	Matrices, SVD, gradients, chain rule
A	3. Probability and Statistics for AI	Distributions, Bayes, CLT
A	4. Information Theory and Entropy	Entropy, cross-entropy, KL, MI
A	5. Data Engineering for Machine Learning	Pipelines, storage, data quality
A	6. Big Data Analytics and Machine Learning	MapReduce, Spark, MLlib
A	7. Feature Stores and Data Versioning	Feast, DVC, training–serving skew
A	8. Model Selection and Automated Hyperparameter Optimization	AIC/BIC, Bayesian optimisation, Hyperband
A	9. Causal Machine Learning	Potential outcomes, DAGs, double ML
A	10. Federated Learning	FedAvg, non-IID data
A	11. Online and Incremental Learning	Regret, concept drift

Part	Section	Focus
A	12. Continual and Lifelong Learning	Forgetting, EWC, replay
A	13. Few-Shot and Zero-Shot Learning	Prototypical and Siamese networks, CLIP
A	14. Meta-Learning	MAML, learning to learn
A	15. Retrieval-Augmented Generation (RAG)	Retrieval, vector databases, evaluation
A	16. AI Agents and Tool-Using Models	ReAct, planning, memory, tools
A	17. Responsible AI and AI Governance	Principles, EU AI Act, NIST AI RMF
A	18. Privacy-Preserving Machine Learning	Differential privacy, HE, MPC
A	19. Green AI and Energy-Efficient Computing	Carbon accounting, efficiency
A	20. Real-World AI Case Studies and End-to-End Projects	Five applied case studies
B	21. Setting Up Python, Jupyter Notebook, and Google Colab	Working environment
B	22. Data Analysis with NumPy and Pandas	Arrays and data frames
B	23. Data Visualization with Matplotlib and Other Tools	Exploratory and explanatory plots
B	24. Building Machine Learning Models with Scikit-learn	Pipelines and model APIs
B	25. Machine Learning Workflows in KNIME	Visual, low-code workflows
B	26. Building Neural Networks with TensorFlow and Keras	Keras models and training
B	27. Building Neural Networks with PyTorch	Modules, autograd, training loops
B	28. Dataset Selection and Data Annotation	Sourcing and labelling data
B	29. Benchmark Datasets and Experimental Design	Fair, controlled experiments
B	30. Reproducibility and Experiment Tracking	Seeds, environments, tracking tools
B	31. Model Comparison and Statistical Significance Testing	Paired tests, corrections
B	32. Interpreting Experimental Results	Error analysis, ablations
B	33. Model Deployment Using APIs and Web Applications	Serving models
B	34. End-to-End Machine Learning Project	Tabular project from data to deployment
B	35. End-to-End Deep Learning Project	Deep learning project lifecycle
B	36. Research Trends, Open Problems, and Future Directions	Where the field is heading

PART A

PART A

Advanced and Integrated Topics

3.1 Statistical Foundations for Machine Learning and Deep Learning

Machine learning is, at its heart, statistical inference carried out by computers at scale. A model is fitted to a finite *sample* of data in the hope that it will generalise to the wider *population* that produced the sample. Every practical question in the field (How much data is enough? Is model A really better than model B? How uncertain is this prediction?) is a statistical question. This section reviews the statistical ideas that recur throughout the book: descriptive statistics, sampling, point and interval estimation, maximum likelihood and maximum a posteriori estimation, and hypothesis testing.

3.1.1 Descriptive Statistics and Sampling

Descriptive statistics summarise a dataset $x_1, \ldots, x_n$. Measures of **central tendency** describe a typical value; measures of **dispersion** describe spread; and measures of **shape** describe asymmetry and tail weight. The sample mean and the unbiased sample variance are defined in Equation (3.1). Dividing by $n - 1$ rather than n (Bessel's correction) compensates for the fact that deviations are measured from the sample mean, which is itself fitted to the data.

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i \qquad s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2 \tag{3.1}$$

where $\bar{x}$ is the sample mean, s^2 the unbiased sample variance and s its square root, the sample standard deviation.

Table 3.2 summarises the common descriptive measures and their roles in machine learning practice. The median and the interquartile range are *robust*: a single extreme outlier barely moves them, whereas it can move the mean and the standard deviation arbitrarily far. This is why robust scaling (subtract the median, divide by the IQR) is preferred for heavy-tailed features.

A **sample** is useful only if it represents the population. In **simple random sampling** every unit has the same chance of selection. **Stratified sampling** divides the population into strata (for example, classes of a label) and samples each separately; scikit-learn's `stratify=` option in `train_test_split` applies exactly this idea so that rare classes appear in both training and test sets. **Cluster sampling** samples whole groups (hospitals, schools) and is cheaper but introduces within-cluster correlation, which is

Table 3.2 Common descriptive statistics and their typical uses in machine learning.

Measure	Definition (in words)	Use in ML
Mean	Arithmetic average of the values	Standardisation, imputation, baselines
Median	Middle value after sorting	Robust imputation and scaling
Mode	Most frequent value	Imputing categorical features
Variance / std. dev.	Average squared deviation from the mean	Feature scaling, uncertainty
Interquartile range	Q3 minus Q1	Outlier detection (1.5 × IQR rule)
Skewness	Asymmetry of the distribution	Deciding on log or Box–Cox transforms
Kurtosis	Heaviness of the tails	Detecting heavy-tailed noise
Correlation	Standardised linear association	Feature selection, multicollinearity checks

why patient-level data from the same hospital should not be split across training and test sets. **Bootstrap sampling** draws n points *with replacement* from the sample itself; it underlies bagging and random forests (Chapter 1) and gives a general way to approximate the sampling distribution of any statistic.

> **D Definition 3.1 | Population, Sample, Parameter and Statistic**
>
> A **population** is the full set of units (or the data-generating distribution) about which we wish to reason. A **parameter** θ is a fixed but unknown property of the population, such as its mean μ. A **sample** is the finite set of observations actually collected, and a **statistic** is any function of the sample, such as $\bar{x}$. An **estimator** is a statistic used to estimate a parameter; its distribution over repeated samples is the **sampling distribution**.

3.1.2 Point Estimation: Bias, Variance and Consistency

An estimator θ is judged by its sampling distribution. Its **bias** is $\mathrm{E}[\theta] - \theta$ and its **variance** is $\mathrm{Var}(\theta)$. The mean squared error decomposes into these two components, as shown in Equation (3.2). This is the same bias–variance decomposition met in Chapter 1 for predictive models; there the 'estimator' is the fitted function itself.

$$\mathrm{MSE}(\hat{\theta}) = \mathbb{E}\left[(\hat{\theta} - \theta)^2\right] = \mathrm{Bias}(\hat{\theta})^2 + \mathrm{Var}(\hat{\theta}) \tag{3.2}$$

where the expectation is taken over random samples from the population.

An estimator is **consistent** if θ converges in probability to θ as $n \to \infty$, and **efficient** if it attains the lowest possible variance among unbiased estimators (the Cramér–Rao bound). The sample mean is unbiased and consistent for μ, with variance σ^2/n; its

standard deviation $\sigma/\sqrt{n}$ is called the **standard error**. The $\sqrt{n}$ in the denominator is one of the most important facts in applied statistics: halving the standard error requires four times as much data.

3.1.3 Maximum Likelihood and Maximum A Posteriori Estimation

Most models in this book are fitted by **maximum likelihood estimation** (MLE). Given independent observations from a model $p(x \mid \theta)$, the likelihood is the probability of the observed data viewed as a function of θ. Because products of many small numbers underflow, we maximise the log-likelihood instead, as in Equation (3.3).

$$\hat{\theta}_{\mathrm{MLE}} = \arg\max_{\theta} \prod_{i=1}^{n} p(x_i \mid \theta) = \arg\max_{\theta} \sum_{i=1}^{n} \log p(x_i \mid \theta) \tag{3.3}$$

where $p(x_i \mid \theta)$ is the probability (or density) of observation x_i under parameter θ.

Minimising mean squared error is MLE under Gaussian noise, and minimising cross-entropy is MLE for a categorical (softmax) model; Section 4 develops the second connection in detail. **Maximum a posteriori** (MAP) estimation adds a prior $p(\theta)$ that encodes beliefs held before seeing the data. By Bayes' theorem the posterior is proportional to likelihood times prior, giving Equation (3.4).

$$\hat{\theta}_{\mathrm{MAP}} = \arg\max_{\theta} \left[\sum_{i=1}^{n} \log p(x_i \mid \theta) + \log p(\theta) \right] \tag{3.4}$$

where $p(\theta)$ is the prior distribution over parameters.

The log-prior acts as a regulariser. A zero-mean Gaussian prior $\theta \sim \mathcal{N}(0, \tau^2 I)$ contributes $-\|\theta\|^2/(2\tau^2)$, which is exactly L2 (ridge) regularisation with $\lambda = 1/(2\tau^2)$; a Laplace prior gives L1 (lasso) regularisation. Weight decay in neural networks is therefore MAP estimation with a Gaussian prior. As n grows the likelihood term dominates the fixed prior and MAP converges to MLE.

E Worked Example 3.1 | MLE and MAP for a Coin

A coin is tossed $n = 10$ times and lands heads $k = 8$ times. The Bernoulli log-likelihood is $\ell(\theta) = k \log \theta + (n - k) \log(1 - \theta)$. Setting $d\ell/d\theta = k/\theta - (n - k)/(1 - \theta) = 0$ gives $\theta_{\mathrm{MLE}} = k/n = 8/10 = 0.80$.

Now place a Beta(α, β) prior with $\alpha = \beta = 5$, expressing a belief that the coin is roughly fair. The posterior is Beta$(k + \alpha, n - k + \beta) = \mathrm{Beta}(13, 7)$, whose mode is $(k + \alpha - 1)/(n + \alpha + \beta - 2) = 12/18 \approx 0.67$.

The prior has pulled the estimate towards 0.5. With $n = 1000$ tosses and 800 heads, the MAP

estimate would be $804/1008 \approx 0.798$, almost identical to the MLE: the data overwhelm the prior.

3.1.4 Hypothesis Testing and Confidence Intervals

A **hypothesis test** asks whether the data provide enough evidence to reject a default claim, the **null hypothesis** H_0 (for example, 'the two models have equal mean accuracy'), in favour of an **alternative** H_1. A **test statistic** is computed from the data, and the **p-value** is the probability, assuming H_0 is true, of obtaining a statistic at least as extreme as the one observed. If the p-value is below a pre-chosen **significance level** α (commonly 0.05), H_0 is rejected. For comparing a sample mean with a hypothesised value μ_0 when σ is unknown, the one-sample t-statistic Equation (3.5) is used.

$$t = \frac{\bar{x} - \mu_0}{s/\sqrt{n}} \tag{3.5}$$

where t follows a Student t-distribution with $n - 1$ degrees of freedom under H_0.

Two kinds of error are possible. A **Type I error** rejects a true H_0 (probability α); a **Type II error** fails to reject a false H_0 (probability β). The **power** $1 - \beta$ is the probability of detecting a real effect and grows with sample size and effect size. Table 3.3 lists tests that appear frequently in machine learning research; Section 31 in Part B applies them to model comparison.

Table 3.3 Frequently used statistical tests in machine learning research.

Test	Question answered	Typical ML use
One/two-sample t-test	Do means differ?	Compare a metric to a baseline value
Paired t-test	Do paired differences have zero mean?	Two models on the same CV folds
Wilcoxon signed-rank	Non-parametric paired comparison	Two models across many datasets
McNemar's test	Do two classifiers err on different cases?	Two classifiers on one test set
Chi-square test	Are two categorical variables independent?	Feature–label association
Kolmogorov–Smirnov	Do two samples share a distribution?	Data-drift detection
Friedman + Nemenyi	Do several models rank differently?	Many models over many datasets

A **confidence interval** (CI) gives a range of plausible values for a parameter. A 95% CI for a mean is constructed as in Equation (3.6). Its correct interpretation is about

the *procedure*: if we repeated the sampling many times, about 95% of the intervals so constructed would contain the true μ. Figure 3.1 illustrates this by drawing 40 samples from a population with known mean.

$$\bar{x} \pm t_{1-\alpha/2,\, n-1} \frac{s}{\sqrt{n}} \tag{3.6}$$

where $t_{1-\alpha/2,n-1}$ is the critical value of the t-distribution (about 2.09 for $n = 20$ and $\alpha = 0.05$).

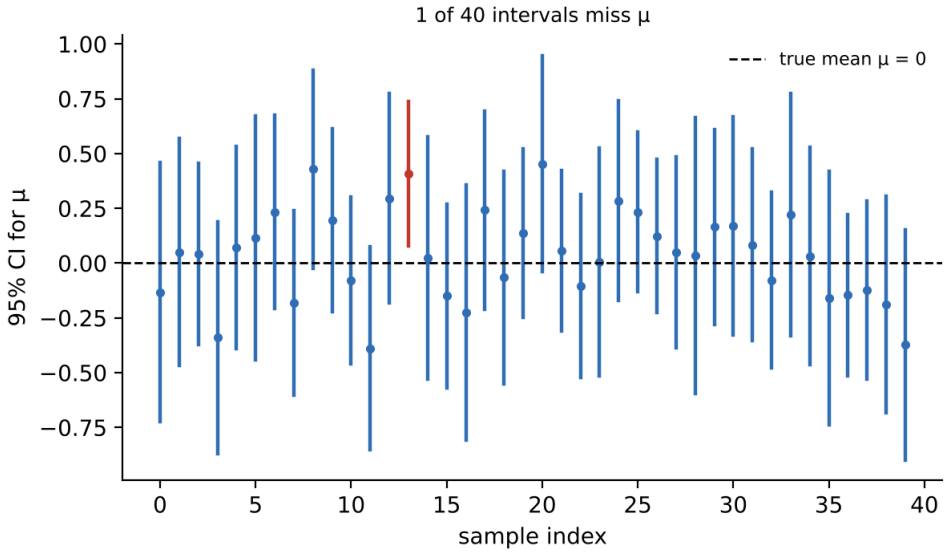


Figure 3.1 Forty 95% confidence intervals for a population mean $\mu = 0$, each computed from an independent sample of size 20. Intervals that miss μ are drawn in red; on average about 5% of them do.

Bootstrap confidence intervals avoid distributional assumptions: resample the test set B times (typically $B \geq 1000$), recompute the metric on each resample, and take the 2.5th and 97.5th percentiles. Listing 3.1 computes a t-interval, a bootstrap interval for accuracy and a paired t-test comparing two models across cross-validation folds.

Listing 3.1 Confidence intervals and a paired t-test with NumPy and SciPy.

```
import numpy as np
from scipy import stats

rng = np.random.default_rng(42)
# Accuracies of two models on the same 10 cross-validation folds
acc_a = np.array([0.81, 0.79, 0.84, 0.80, 0.83, 0.82, 0.78, 0.85, 0.80, 0.82])
acc_b = np.array([0.78, 0.77, 0.82, 0.79, 0.80, 0.81, 0.76, 0.83, 0.79, 0.80])

# 95% t-interval for the mean accuracy of model A
n = len(acc_a)
half = stats.t.ppf(0.975, n - 1) * acc_a.std(ddof=1) / np.sqrt(n)
```

```
print(f"Model A: {acc_a.mean():.3f} +/- {half:.3f}")

# Bootstrap 95% CI for test-set accuracy from per-example correctness
correct = rng.random(500) < 0.82          # stand-in for (y_pred == y_true)
boots = [rng.choice(correct, correct.size).mean() for _ in range(2000)]
print("Bootstrap CI:", np.percentile(boots, [2.5, 97.5]))

# Paired t-test: are the fold-wise differences centred on zero?
t, p = stats.ttest_rel(acc_a, acc_b)
print(f"t = {t:.2f}, p = {p:.4f}")
```

Note | Statistical Versus Practical Significance

With very large test sets, even a 0.1% accuracy difference can be 'statistically significant'. A small p-value says that an effect is unlikely to be zero, not that it is large or useful. Always report effect sizes and confidence intervals alongside p-values, and when many comparisons are made correct for multiple testing (for example, with the Bonferroni or Holm procedure).

3.2 Linear Algebra and Calculus for AI

Data, parameters and activations in machine learning are stored as vectors, matrices and higher-order tensors, and learning is carried out by following derivatives downhill. Linear algebra supplies the language for the first and multivariable calculus for the second. This section gathers the results used most often, with worked examples that connect each idea to a learning algorithm.

3.2.1 Vectors, Matrices and Norms

A **vector** $x \in \mathbb{R}^d$ is an ordered list of d numbers; in machine learning it usually holds the features of one example or the weights of a linear model. A **matrix** $A \in \mathbb{R}^{m \times n}$ holds m rows and n columns; a dataset is typically stored as a design matrix $X \in \mathbb{R}^{n \times d}$ with one example per row. The product Ax is a linear transformation of x, and a fully connected neural network layer computes $\sigma(Wx + b)$: a linear map followed by an element-wise non-linearity. The **dot product** $x^T y = \Sigma_i \, x_i y_i$ measures alignment and underlies cosine similarity, attention scores and linear classifiers.

A **norm** measures the size of a vector. The family of p-norms is defined in Equation (3.7); the three members listed in Table 3.4 have distinct roles in regularisation, distance metrics and robustness.

$$\|\mathbf{x}\|_p = \left(\sum_{i=1}^{d} |x_i|^p\right)^{1/p}, \quad p \geq 1 \tag{3.7}$$

where the special cases $p = 1, 2$ and $p \to \infty$ give the Manhattan, Euclidean and maximum norms.

Table 3.4 Vector and matrix norms used in machine learning.

Norm	Formula (in words)	Typical use
L1 (Manhattan)	Sum of absolute values	Lasso, sparsity, robust losses
L2 (Euclidean)	Square root of sum of squares	Ridge, weight decay, k-NN distance
L∞ (max)	Largest absolute entry	Adversarial perturbation budgets
Frobenius	Square root of sum of squared matrix entries	Matrix factorisation losses
Spectral	Largest singular value of a matrix	Lipschitz control, spectral normalisation
Nuclear	Sum of singular values	Low-rank matrix completion

Other matrix concepts recur constantly. The **transpose** A^T swaps rows and columns. A square matrix A is **invertible** if there is A^{-1} with $AA^{-1} = I$; this holds exactly when its **determinant** is non-zero, equivalently when its columns are linearly independent. The **rank** of A is the number of linearly independent columns; a rank-deficient design matrix (perfect multicollinearity) makes X^TX singular and the ordinary least-squares solution non-unique. A symmetric matrix A is **positive definite** if $x^TAx > 0$ for every non-zero x; covariance matrices are positive semi-definite, and a positive-definite Hessian certifies a strict local minimum.

3.2.2 Eigen-Decomposition and the Singular Value Decomposition

A non-zero vector v is an **eigenvector** of a square matrix A with **eigenvalue** λ if $Av = \lambda v$: A merely stretches v by λ without changing its direction. A real symmetric matrix (such as a covariance matrix) has a full set of orthonormal eigenvectors and real eigenvalues, so it can be factorised as in Equation (3.8).

$$A = Q\,\Lambda\,Q^{\top}, \quad Q^{\top}Q = I, \quad \Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_d) \tag{3.8}$$

where the columns of Q are eigenvectors of A and Λ is the diagonal matrix of eigenvalues.

Principal component analysis (PCA, Chapter 1) is precisely the eigen-decomposition of the data covariance matrix: eigenvectors give the principal directions and eigenvalues the variance along each. Eigenvalues also govern optimisation: for a quadratic loss

the ratio of the largest to the smallest Hessian eigenvalue (the **condition number**) determines how slowly gradient descent converges.

The **singular value decomposition** (SVD) generalises eigen-decomposition to any rectangular matrix. Every $A \in \mathbb{R}^{m \times n}$ can be written as in Equation (3.9). Geometrically, any linear map is a rotation, an axis-aligned scaling and another rotation, as illustrated in Figure 3.2.

$$A = U \Sigma V^{\top} = \sum_{i=1}^{r} \sigma_i \, \mathbf{u}_i \, \mathbf{v}_i^{\top} \tag{3.9}$$

where $U \in \mathbb{R}^{m \times m}$ and $V \in \mathbb{R}^{n \times n}$ are orthogonal, Σ is diagonal with singular values $\sigma_1 \geq \sigma_2 \geq \ldots \geq 0$, and r is the rank of A.

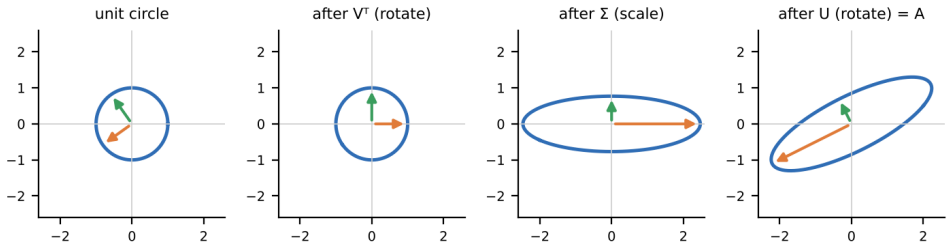


Figure 3.2 Geometric view of the SVD of a 2×2 matrix $A = U\Sigma V^{T}$: the unit circle is rotated by V^{T}, stretched along the axes by Σ and rotated by U into an ellipse whose semi-axes are the singular values.

Truncating the sum in Equation (3.9) after k terms gives the best rank-k approximation of A in both the Frobenius and spectral norms (the Eckart–Young theorem). This single fact underlies latent semantic analysis, image compression, matrix-factorisation recommenders, and parameter-efficient fine-tuning methods such as LoRA, which constrain weight updates to be low-rank.

E Worked Example 3.2 | Eigenvalues of a 2 × 2 Covariance Matrix

Let the covariance matrix of two standardised features be $A = [[2, 1], [1, 2]]$. Eigenvalues solve $\det(A - \lambda I) = (2 - \lambda)^2 - 1 = 0$, so $\lambda = 2 \pm 1$, giving $\lambda_1 = 3$ and $\lambda_2 = 1$.

For $\lambda_1 = 3$: $(A - 3I)v = 0$ gives $-v_1 + v_2 = 0$, so $v_1 = (1, 1)/\sqrt{2}$. For $\lambda_2 = 1$: $v_1 + v_2 = 0$, so $v_2 = (1, -1)/\sqrt{2}$.

The first principal component points along $(1, 1)/\sqrt{2}$ and explains $3/(3 + 1) = 75\%$ of the total variance. Projecting onto it alone keeps three quarters of the variance with half the dimensions.

3.2.3 Derivatives, Gradients, Jacobians and Hessians

For a scalar function of one variable, the derivative $f'(x)$ is the slope of the tangent line. For a scalar function of many variables $f: \mathbb{R}^d \to \mathbb{R}$ (such as a loss as a function of all model weights), each **partial derivative** $\partial f/\partial x_i$ measures the rate of change when only x_i varies. Collecting them gives the **gradient**, Equation (3.10), which points in the direction of steepest ascent. Gradient descent therefore moves in the direction $-\nabla f$.

$$\nabla f(\mathbf{x}) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \ldots, \frac{\partial f}{\partial x_d}\right)^\top \tag{3.10}$$

where $\nabla f(x) \in \mathbb{R}^d$ has the same shape as x.

For a vector-valued function $f: \mathbb{R}^n \to \mathbb{R}^m$ (such as a network layer), the first derivatives form the $m \times n$ **Jacobian** matrix, and for a scalar function the second derivatives form the $d \times d$ **Hessian**, both given in Equation (3.11). The Jacobian is what backpropagation multiplies layer by layer; the Hessian describes curvature and is used by Newton's method and in analyses of sharp versus flat minima.

$$\begin{aligned} J_{ij} &= \frac{\partial f_i}{\partial x_j}, \quad J \in \mathbb{R}^{m \times n} \\ H_{ij} &= \frac{\partial^2 f}{\partial x_i \, \partial x_j}, \quad H \in \mathbb{R}^{d \times d} \end{aligned} \tag{3.11}$$

where the Hessian of a twice continuously differentiable function is symmetric.

A second-order Taylor expansion, Equation (3.12), combines both. It explains why gradient descent works for small steps, why too large a learning rate diverges in directions of high curvature, and how Newton's method chooses the step $-H^{-1}\nabla f$ that minimises the quadratic model exactly.

$$f(\mathbf{x} + \delta) \approx f(\mathbf{x}) + \nabla f(\mathbf{x})^\top \delta + \tfrac{1}{2}\delta^\top H(\mathbf{x})\,\delta \tag{3.12}$$

where δ is a small displacement of the parameters.

Figure 3.3 shows the gradient field of an ill-conditioned quadratic and the zig-zag path that plain gradient descent follows across the narrow valley, a direct visual consequence of a large Hessian condition number.

3.2.4 The Chain Rule and Backpropagation

Neural networks are compositions of simple functions, and the **chain rule** is how their derivatives are computed. For a composition $y = f(g(x))$ of vector functions, the Jacobian of the composition is the product of the Jacobians, Equation (3.13). Backpropagation (Chapter 2) evaluates this product from the output backwards, which is efficient

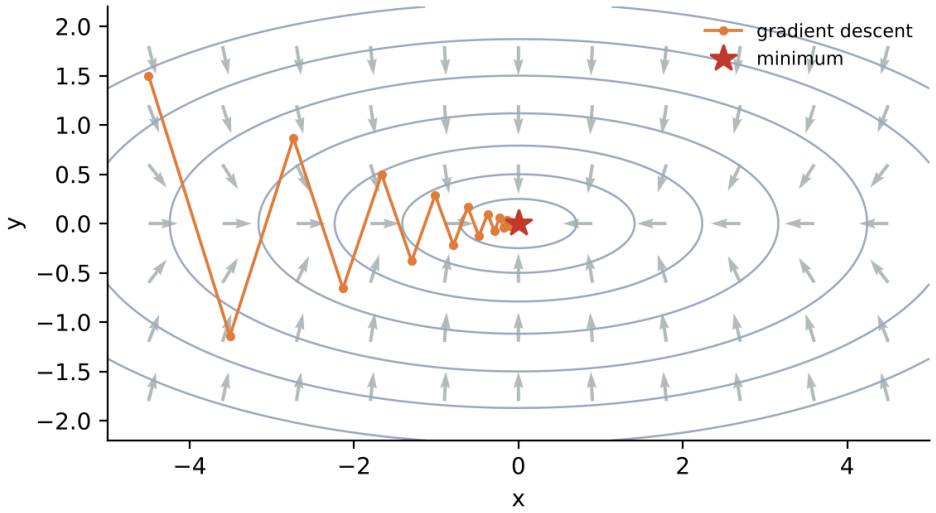


Figure 3.3 Contours of $f(x, y) = x^2 + 8y^2$ with negative-gradient direction arrows (unit length) and the zig-zag path of gradient descent ($\eta = 0.11$) from $(-4.5, 1.5)$. The Hessian eigenvalues 2 and 16 give a condition number of 8.

because the final output is a scalar loss: each step is a vector–Jacobian product rather than a full matrix–matrix product.

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \frac{\partial \mathbf{f}}{\partial \mathbf{g}} \frac{\partial \mathbf{g}}{\partial \mathbf{x}}$$
$$\frac{\partial L}{\partial \mathbf{x}} = \left(\frac{\partial \mathbf{g}}{\partial \mathbf{x}}\right)^{\top} \frac{\partial L}{\partial \mathbf{g}} \tag{3.13}$$

where the second line is the vector–Jacobian product used in reverse-mode automatic differentiation.

Table 3.5 lists matrix-calculus identities that appear repeatedly when deriving gradients by hand, for instance for linear regression, logistic regression and Gaussian models.

E Worked Example 3.3 | Backpropagation Through a Single Neuron

Consider $\hat{y} = \sigma(z)$ with $z = w_1 x_1 + w_2 x_2 + b$, the logistic loss $L = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$, and the values $x = (1, 2)$, $w = (0.5, -0.25)$, $b = 0.1$ and $y = 1$.

Forward pass: $z = 0.5 - 0.5 + 0.1 = 0.1$; $\hat{y} = \sigma(0.1) \approx 0.525$; $L = -\log 0.525 \approx 0.644$.

Backward pass: $\partial L/\partial z = \hat{y} - y = -0.475$ (the elegant simplification for sigmoid with cross-entropy). Then $\partial L/\partial w_1 = -0.475 \times 1 = -0.475$, $\partial L/\partial w_2 = -0.475 \times 2 = -0.950$ and $\partial L/\partial b = -0.475$.

A gradient step with $\eta = 0.1$ gives $w = (0.548, -0.155)$ and $b = 0.148$, increasing z and hence $\hat{y}$ towards the target 1.

Table 3.5 Useful matrix-calculus identities (vectors x, a; symmetric matrix A where noted).

Function f(x)	Gradient $\nabla_x f$	Where it appears
$a^T x$	a	Linear models
$x^T x = \|x\|^2$	2x	L2 regularisation
$x^T A x$	$(A + A^T)x$, or 2Ax if A symmetric	Quadratic forms, Gaussians
$\|y - Xw\|^2$ (in w)	$-2X^T(y - Xw)$	Least squares
$\log(1 + \exp(-y\, w^T x))$ (in w)	$-y\, \sigma(-y\, w^T x)\, x$	Logistic regression
softmax cross-entropy (in logits z)	$\text{softmax}(z) - y$	Classification output layer

Listing 3.2 verifies several of these results numerically: it computes an eigen-decomposition and an SVD, forms a low-rank approximation, and checks an analytic gradient against automatic differentiation in PyTorch.

Listing 3.2 Eigen-decomposition, SVD, low-rank approximation and gradient checking.

```python
import numpy as np
import torch

A = np.array([[2.0, 1.0], [1.0, 2.0]])
vals, vecs = np.linalg.eigh(A)              # symmetric -> eigh
print("eigenvalues:", vals)                 # [1. 3.]

rng = np.random.default_rng(0)
M = rng.normal(size=(100, 40))
U, S, Vt = np.linalg.svd(M, full_matrices=False)
k = 10
M_k = U[:, :k] @ np.diag(S[:k]) @ Vt[:k]    # best rank-10 approximation
rel_err = np.linalg.norm(M - M_k) / np.linalg.norm(M)
print(f"relative Frobenius error (rank {k}): {rel_err:.3f}")

# Gradient check: f(w) = ||y - Xw||^2, analytic gradient = -2 X^T (y - Xw)
X = rng.normal(size=(50, 3)); y = rng.normal(size=50); w = rng.normal(size=3)
g_analytic = -2 * X.T @ (y - X @ w)
wt = torch.tensor(w, requires_grad=True)
loss = ((torch.tensor(y) - torch.tensor(X) @ wt) ** 2).sum()
loss.backward()
print(np.allclose(g_analytic, wt.grad.numpy()))   # True
```

3.3 Probability and Statistics for AI

Probability theory is the mathematics of uncertainty, and nearly every modern learning system is probabilistic somewhere: classifiers output class probabilities, generative models learn data distributions, Bayesian methods maintain distributions over param-

eters, and stochastic gradient descent is itself a random process. Section 1 treated statistics from the viewpoint of estimation; this section develops the probabilistic toolkit behind those estimates.

3.3.1 Random Variables and Probability Distributions

A **random variable** X maps outcomes of a random experiment to numbers. A **discrete** random variable is described by a probability mass function (PMF) $P(X = x)$; a **continuous** one by a probability density function (PDF) $p(x)$, with probabilities given by areas: $P(a \le X \le b) = \int_a^b p(x)\,dx$. The **cumulative distribution function** $F(x) = P(X \le x)$ is defined for both. For several variables we work with a **joint** distribution $p(x, y)$, from which **marginals** are obtained by summing or integrating out variables (the *sum rule*) and **conditionals** by division (the *product rule*), as in Equation (3.14).

$$p(x) = \sum_{y} p(x, y) \quad \text{(sum rule)}$$
$$p(x, y) = p(y \mid x)\, p(x) \quad \text{(product rule)} \tag{3.14}$$

where integrals replace sums for continuous variables.

X and Y are **independent** if $p(x, y) = p(x)p(y)$. The weaker notion of **conditional independence**, $p(x, y \mid z) = p(x \mid z)p(y \mid z)$, is central to naive Bayes, hidden Markov models and the causal graphs of Section 9. Table 3.6 summarises the distributions that appear most often in machine learning.

Table 3.6 Common probability distributions in machine learning.

Distribution	Support / parameters	Mean; variance	Typical ML role
Bernoulli(p)	$\{0, 1\}$; $p \in [0, 1]$	p; $p(1-p)$	Binary labels, dropout masks
Binomial(n, p)	$\{0, \ldots, n\}$	np; $np(1-p)$	Number of correct predictions
Categorical(π)	$\{1, \ldots, K\}$; π on simplex	—	Softmax outputs, token sampling
Poisson(λ)	$\{0, 1, 2, \ldots\}$; $\lambda > 0$	λ; λ	Count data, event arrivals
Uniform(a, b)	$[a, b]$	$(a+b)/2$; $(b-a)^2/12$	Random search, initialisation
Gaussian $\mathcal{N}(\mu, \sigma^2)$	$\mathbb{R}$	μ; σ^2	Noise models, VAEs, weight priors
Exponential(λ)	$[0, \infty)$	$1/\lambda$; $1/\lambda^2$	Waiting times, survival analysis
Beta(α, β)	$[0, 1]$	$\alpha/(\alpha+\beta)$; —	Prior on probabilities, Thompson sampling
Dirichlet(α)	Probability simplex	$\alpha_k/\Sigma\alpha$; —	Topic models (LDA), priors on π
Laplace(μ, b)	$\mathbb{R}$	μ; $2b^2$	L1 priors, differential privacy noise

The Gaussian distribution deserves special attention. Its density, Equation (3.15), is fully determined by its mean and variance; its multivariate form uses a mean vector μ and a covariance matrix Σ. Figure 3.4 shows how the two parameters change the shape of the curve and how the familiar 68–95–99.7 rule describes the mass within one, two and three standard deviations.

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$
$$p(\mathbf{x}) = \frac{1}{(2\pi)^{d/2}|\Sigma|^{1/2}} \exp\left(-\tfrac{1}{2}(\mathbf{x}-\mu)^{\top}\Sigma^{-1}(\mathbf{x}-\mu)\right) \tag{3.15}$$

where μ is the mean, σ^2 the variance, Σ the $d \times d$ covariance matrix and $|\Sigma|$ its determinant.

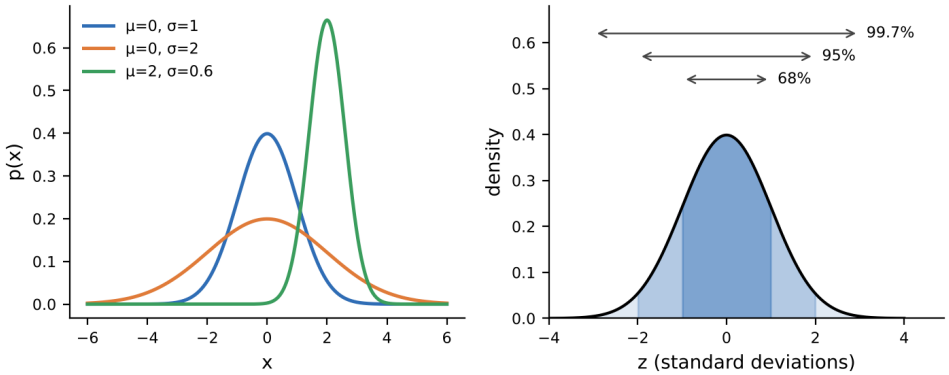


Figure 3.4 Left: Gaussian densities for different means and standard deviations. Right: the standard normal with the regions within one, two and three standard deviations, containing about 68%, 95% and 99.7% of the probability mass.

3.3.2 Bayes' Theorem

Combining the two forms of the product rule, $p(x, y) = p(y \mid x)p(x) = p(x \mid y)p(y)$, yields **Bayes' theorem**, Equation (3.16). It tells us how to update a **prior** belief about a hypothesis H in the light of evidence E, producing a **posterior**. The term $p(E \mid H)$ is the **likelihood**, and the denominator, the **evidence** or marginal likelihood, normalises the result.

$$P(H \mid E) = \frac{P(E \mid H)\,P(H)}{P(E)} = \frac{P(E \mid H)\,P(H)}{P(E \mid H)\,P(H) + P(E \mid \neg H)\,P(\neg H)} \tag{3.16}$$

where $\neg H$ denotes that the hypothesis is false.

E Worked Example 3.4 | Interpreting a Positive Screening Test

A disease has prevalence 1% ($P(D) = 0.01$). A screening model has sensitivity 95% ($P(+ \mid D) = 0.95$) and specificity 90%, so its false-positive rate is $P(+ \mid \neg D) = 0.10$. A patient tests positive. What is the probability that they have the disease?

Numerator: $P(+ \mid D)P(D) = 0.95 \times 0.01 = 0.0095$.

Evidence: $P(+) = 0.0095 + 0.10 \times 0.99 = 0.0095 + 0.099 = 0.1085$.

Posterior: $P(D \mid +) = 0.0095 / 0.1085 \approx 0.088$, that is, under 9%.

Despite a seemingly accurate model, more than 90% of positives are false alarms because the condition is rare. This is the **base-rate effect**, and it is why precision must be reported alongside recall for imbalanced medical and fraud-detection problems.

Bayes' theorem is also the foundation of a whole school of machine learning. **Bayesian inference** treats model parameters as random variables and computes the full posterior $p(\theta \mid D)$ rather than a single point estimate; predictions then average over that posterior, providing calibrated uncertainty. Exact posteriors are rarely tractable, so practical methods use Markov chain Monte Carlo, variational inference, Laplace approximations, or cheaper proxies such as deep ensembles and Monte Carlo dropout.

3.3.3 Expectation, Variance and Covariance

The **expectation** of a function $g(X)$ is its probability-weighted average; the **variance** measures spread around the mean; and the **covariance** measures how two variables vary together. Their definitions are collected in Equation (3.17).

$$\begin{aligned} \mathbb{E}[g(X)] &= \sum_x g(x)\,p(x) \quad \text{or} \quad \int g(x)\,p(x)\,dx \\ \operatorname{Var}(X) &= \mathbb{E}\big[(X - \mathbb{E}[X])^2\big] = \mathbb{E}[X^2] - \mathbb{E}[X]^2 \\ \operatorname{Cov}(X, Y) &= \mathbb{E}\big[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])\big] \end{aligned} \tag{3.17}$$

Expectation is linear, $E[aX + bY] = aE[X] + bE[Y]$, whether or not the variables are independent. For variances, $\operatorname{Var}(aX + bY) = a^2\operatorname{Var}(X) + b^2\operatorname{Var}(Y) + 2ab\,\operatorname{Cov}(X, Y)$, so the variance of a sum of independent variables is the sum of their variances. This identity explains why averaging the predictions of several weakly correlated models (bagging, ensembles) reduces variance. Dividing the covariance by the product of standard deviations gives the **Pearson correlation** $\rho \in [-1, 1]$. Note that zero correlation does not imply independence: if X is symmetric about zero and $Y = X^2$, then $\operatorname{Cov}(X, Y) = 0$ although Y is a deterministic function of X.

For a random vector x the **covariance matrix** $\Sigma = E[(x - \mu)(x - \mu)^T]$ collects all pairwise covariances. It is symmetric and positive semi-definite, its diagonal holds vari-

ances, and its eigen-decomposition (Section 2) gives the principal components. Under a linear transformation $y = Ax + b$, the mean becomes $A\mu + b$ and the covariance becomes $A\Sigma A^{T}$, a rule used repeatedly in Kalman filters, Gaussian processes and the reparameterisation trick of variational autoencoders.

3.3.4 Law of Large Numbers and Central Limit Theorem

Two limit theorems justify much of statistical practice. The **law of large numbers** states that the sample mean of independent, identically distributed (i.i.d.) variables converges to the true mean as $n \to \infty$; this is why the empirical risk on a large training set approximates the true risk, and why mini-batch gradients are unbiased estimates of the full gradient. The **central limit theorem** (CLT), Equation (3.18), says more: whatever the shape of the original distribution (provided its variance is finite), the distribution of the standardised sample mean approaches a standard normal.

$$\frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}} \longrightarrow \mathcal{N}(0,1) \quad \text{as} \quad n \to \infty \tag{3.18}$$

where $\bar{X}_n$ is the mean of n i.i.d. variables with mean μ and finite variance σ^2.

Figure 3.5 demonstrates the theorem for a strongly skewed exponential distribution: the histogram of sample means becomes increasingly bell-shaped and narrower as n grows. The CLT is the reason the t-tests and confidence intervals of Section 1 work for accuracy, loss and many other metrics even when individual observations are far from normal.

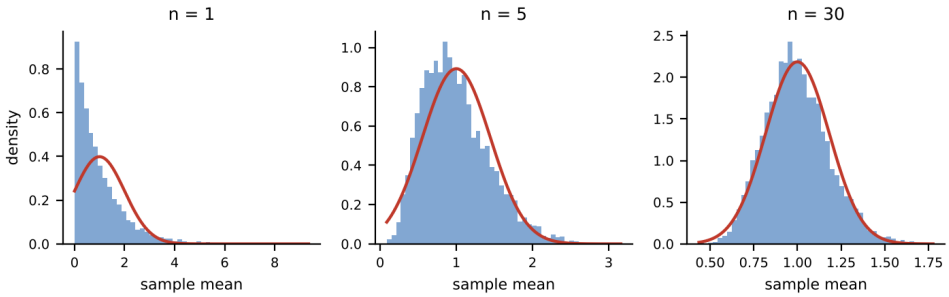


Figure 3.5 Central limit theorem in action. Histograms of 5,000 sample means drawn from a skewed Exponential(1) distribution for sample sizes $n = 1$, 5 and 30; the red curve is the normal approximation $\mathcal{N}(1, 1/n)$.

> **Key Points** | **Probability for AI**
>
> - The sum and product rules, and Bayes' theorem derived from them, are the only rules needed to manipulate any probabilistic model.

- Low base rates make even accurate detectors produce mostly false positives.
- Variances of independent quantities add; averaging n independent estimates divides variance by n.
- The law of large numbers justifies empirical risk minimisation; the CLT justifies normal-theory confidence intervals for averaged metrics.

3.4 Information Theory and Entropy

Information theory, founded by Claude Shannon in his 1948 paper 'A Mathematical Theory of Communication', quantifies uncertainty and information. It began as a theory of data compression and transmission, but its quantities now appear throughout machine learning: cross-entropy is the standard classification loss, KL divergence regularises variational autoencoders and aligns language models with reference policies, information gain chooses splits in decision trees, and mutual information drives feature selection and representation learning.

3.4.1 Entropy

The **self-information** of an outcome x with probability $\mathrm{p}(\mathrm{x})$ is $-\log \mathrm{p}(\mathrm{x})$: rare events are more surprising and carry more information. The **entropy** of a discrete random variable is its expected self-information, Equation (3.19). With base-2 logarithms it is measured in **bits**; with natural logarithms in **nats**.

$$H(X) = -\sum_{x} p(x) \log_2 p(x) \tag{3.19}$$

where the convention $0 \log 0 = 0$ is used.

Entropy is zero when the outcome is certain and maximal ($\log_2$ K bits) when all K outcomes are equally likely. Shannon's source-coding theorem gives it an operational meaning: $\mathrm{H}(\mathrm{X})$ is the minimum average number of bits per symbol needed to encode outcomes of X without loss. For a binary variable with $\mathrm{P}(\mathrm{X} = 1) = \mathrm{p}$, entropy reduces to the **binary entropy function** of Equation (3.20), plotted in Figure 3.6.

$$H_b(p) = -p\log_2 p - (1-p)\log_2 (1-p) \tag{3.20}$$

E Worked Example 3.5 | Entropy and Information Gain

A node contains 10 examples: 6 positive, 4 negative. Its entropy is $\mathrm{H} = -0.6 \log_2 0.6 - 0.4 \log_2 0.4 = 0.442 + 0.529 = 0.971$ bits.

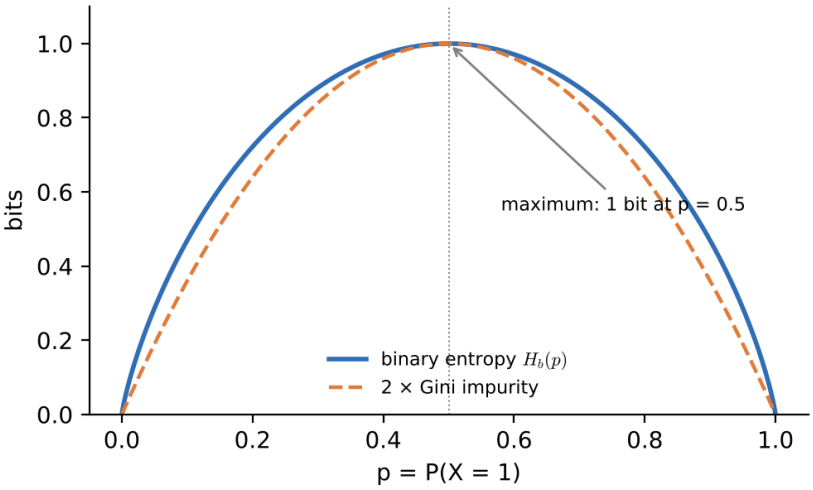


Figure 3.6 Binary entropy $\mathrm{H_b(p)}$ in bits, together with the Gini impurity $2\mathrm{p}(1-\mathrm{p})$ scaled by 2 for comparison. Both are maximal for a 50:50 split and zero for a pure node, which is why both serve as decision-tree impurity measures.

A candidate split sends 5 examples (5 positive, 0 negative) left and 5 (1 positive, 4 negative) right. Left entropy = 0 bits; right entropy = $-0.2\log_2 0.2 - 0.8\log_2 0.8 = 0.464 + 0.258 = 0.722$ bits.

Weighted child entropy = $0.5 \times 0 + 0.5 \times 0.722 = 0.361$ bits, so the **information gain** is $0.971 - 0.361 = 0.610$ bits. The ID3 and C4.5 tree algorithms (Chapter 1) choose the split with the largest such gain.

3.4.2 Joint and Conditional Entropy

For two variables, the **joint entropy** $\mathrm{H(X, Y)}$ measures the uncertainty of the pair, and the **conditional entropy** $\mathrm{H(Y \mid X)}$ measures the uncertainty remaining in Y once X is known. They are linked by the chain rule of entropy, Equation (3.21).

$$H(X,Y) = -\sum_{x,y} p(x,y)\log p(x,y)$$
$$H(Y\mid X) = -\sum_{x,y} p(x,y)\log p(y\mid x) = H(X,Y) - H(X) \tag{3.21}$$

Conditioning never increases entropy on average: $\mathrm{H(Y \mid X)} \le \mathrm{H(Y)}$, with equality exactly when X and Y are independent. Information gain in a decision tree is $\mathrm{H(Y)} - \mathrm{H(Y \mid split)}$, the reduction in label uncertainty achieved by learning which branch an example falls into.

3.4.3 Cross-Entropy and Kullback–Leibler Divergence

Suppose data come from a true distribution p but we encode them (or model them) using a distribution q. The **cross-entropy** $H(p, q)$ is the average number of bits needed, and the **Kullback–Leibler (KL) divergence** is the *extra* cost of using q instead of p. Both are defined in Equation (3.22).

$$H(p, q) = -\sum_{x} p(x) \log q(x)$$
$$D_{\mathrm{KL}}(p \| q) = \sum_{x} p(x) \log \frac{p(x)}{q(x)} = H(p, q) - H(p) \geq 0 \tag{3.22}$$

where $D_{\mathrm{KL}} = 0$ if and only if $p = q$.

In classification, p is the one-hot label distribution and q the model's softmax output. Because $H(p)$ does not depend on the model, minimising cross-entropy is the same as minimising KL divergence to the data distribution, and it is also the same as maximising the log-likelihood (Section 1). For a single example with true class c the loss reduces to $-\log q(c)$, as given in Equation (3.23).

$$\mathcal{L}_{\mathrm{CE}} = -\frac{1}{n} \sum_{i=1}^{n} \sum_{k=1}^{K} y_{ik} \log \hat{y}_{ik} = -\frac{1}{n} \sum_{i=1}^{n} \log \hat{y}_{i,\, c_i} \tag{3.23}$$

where y_{ik} is the one-hot label, $\hat{y}_{ik}$ the predicted probability of class k, and c_i the true class of example i.

KL divergence is **asymmetric**: $D_{\mathrm{KL}}(p\|q) \neq D_{\mathrm{KL}}(q\|p)$ in general. Minimising the 'forward' KL $D_{\mathrm{KL}}(p\|q)$ makes q cover all the modes of p (mass-covering), whereas minimising the 'reverse' KL $D_{\mathrm{KL}}(q\|p)$, as variational inference does, makes q lock on to a single mode (mode-seeking). Symmetric alternatives include the Jensen–Shannon divergence, used in the original GAN analysis, and the Wasserstein distance. Table 3.7 maps each information-theoretic quantity to its uses in machine learning.

E Worked Example 3.6 | Cross-Entropy and KL for Three Classes

Let the true distribution be $p = (0.7, 0.2, 0.1)$ and a model predict $q = (0.5, 0.3, 0.2)$. Using natural logarithms:

$H(p) = -(0.7 \ln 0.7 + 0.2 \ln 0.2 + 0.1 \ln 0.1) = 0.250 + 0.322 + 0.230 = 0.802$ nats.

$H(p, q) = -(0.7 \ln 0.5 + 0.2 \ln 0.3 + 0.1 \ln 0.2) = 0.485 + 0.241 + 0.161 = 0.887$ nats.

$D_{\mathrm{KL}}(p\|q) = 0.887 - 0.802 = 0.085$ nats: the model wastes 0.085 nats per example relative to the ideal predictor.

Table 3.7 Information-theoretic quantities and where they appear in machine learning.

Quantity	Intuition	Machine learning use
Entropy H(X)	Uncertainty of X	Tree impurity, entropy regularisation in RL, active learning
Conditional entropy H(Y|X)	Uncertainty left after observing X	Information gain in trees
Cross-entropy H(p, q)	Coding cost using the wrong model q	Classification and language-model loss
KL divergence	Extra cost of q over p	VAE regulariser, knowledge distillation, RLHF penalty
Jensen–Shannon divergence	Symmetrised, bounded KL	Analysis of GAN objectives
Mutual information I(X;Y)	Shared information between X and Y	Feature selection, contrastive learning
Perplexity 2^H	Effective number of choices	Evaluating language models

3.4.4 Mutual Information

Mutual information measures how much knowing one variable reduces uncertainty about another. It is the KL divergence between the joint distribution and the product of marginals, Equation (3.24), and is zero if and only if the variables are independent. Unlike correlation, it captures any form of dependence, linear or not.

$$I(X;Y) = \sum_{x,y} p(x,y) \log \frac{p(x,y)}{p(x)\,p(y)} = H(Y) - H(Y \mid X) = H(X) + H(Y) - H(X,Y) \tag{3.24}$$

Figure 3.7 shows the relationships between these quantities as an information diagram. In practice, mutual information between each feature and the label is a strong filter criterion for feature selection (scikit-learn's `mutual_info_classif` estimates it with nearest-neighbour methods). In representation learning, contrastive objectives such as InfoNCE maximise a lower bound on the mutual information between different views of the same input, and the *information bottleneck* principle seeks representations that keep information about the label while discarding the rest of the input.

Listing 3.3 Computing entropy, cross-entropy, KL divergence and mutual information.

```
import numpy as np
from scipy.stats import entropy
from sklearn.datasets import load_breast_cancer
from sklearn.feature_selection import mutual_info_classif

p = np.array([0.7, 0.2, 0.1]); q = np.array([0.5, 0.3, 0.2])
H_p = entropy(p)                        # natural log -> nats
KL = entropy(p, q)                      # D_KL(p || q)
CE = H_p + KL                           # cross-entropy H(p, q)
```

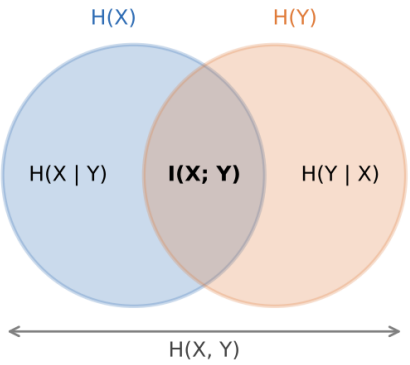


Figure 3.7 Information diagram for two variables. The circles represent $H(X)$ and $H(Y)$; their overlap is the mutual information $I(X;Y)$, the remaining crescents are the conditional entropies, and the union is the joint entropy $H(X, Y)$.

```
print(f"H(p)={H_p:.3f}  H(p,q)={CE:.3f}  KL={KL:.3f}")

# Rank features of a real dataset by estimated mutual information with the label
X, y = load_breast_cancer(return_X_y=True, as_frame=True)
mi = mutual_info_classif(X, y, random_state=0)
top = np.argsort(mi)[::-1][:5]
for j in top:
    print(f"{X.columns[j]:<25s} MI = {mi[j]:.3f} nats")
```

Listing 3.3 reproduces the three-class worked example and then ranks the features of the Wisconsin breast-cancer dataset by their estimated mutual information with the diagnosis. Features describing cell-nucleus size and concavity score highest, matching the intuition of pathologists.

Key Points | Information Theory

- Entropy measures uncertainty; it is maximal for uniform distributions.
- Minimising cross-entropy, minimising forward KL and maximising likelihood are the same objective for classification.
- KL divergence is asymmetric; the direction chosen determines mass-covering versus mode-seeking behaviour.
- Mutual information detects any statistical dependence and is zero only under independence.

3.5 Data Engineering for Machine Learning

A widely quoted observation among practitioners is that most of the effort in a production machine learning system goes into data rather than models. Sculley et al. (2015), in 'Hidden Technical Debt in Machine Learning Systems' (NeurIPS), showed that the model code is a small box surrounded by much larger components for data collection, verification, feature extraction, serving and monitoring. **Data engineering** is the discipline of building those components: reliable pipelines that move data from where it is produced to where it is analysed and learned from, in the right shape, at the right time and with known quality.

3.5.1 ETL and ELT Pipelines

A **data pipeline** is a sequence of automated steps that ingests raw data, transforms it and loads it into a target system. The classical pattern is **ETL** (extract, transform, load): data are extracted from operational sources such as transactional databases, application logs and third-party APIs, transformed on a separate processing server (cleaned, joined, aggregated, conformed to a schema), and only then loaded into an analytical store. Modern cloud warehouses are cheap to scale and powerful enough to transform data in place, which gave rise to **ELT** (extract, load, transform): raw data are loaded first and transformations are expressed as SQL inside the warehouse, often managed with tools such as dbt. Figure 3.8 contrasts the two patterns.

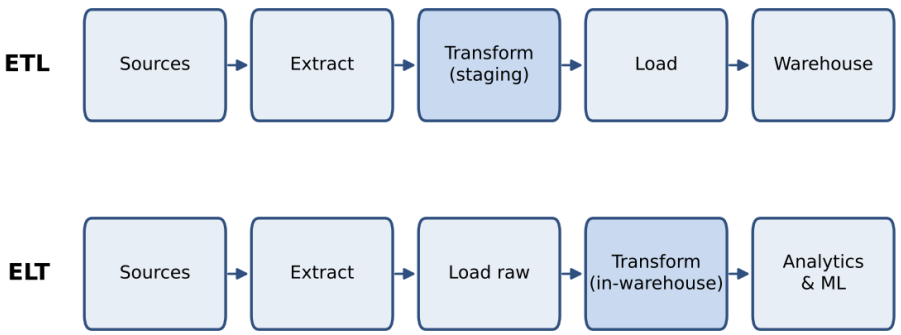


Figure 3.8 ETL transforms data before loading them into the target store, whereas ELT loads raw data first and transforms them inside a scalable warehouse or lakehouse, keeping the raw copy available for re-processing.

Whichever pattern is used, pipelines are expressed as **directed acyclic graphs** (DAGs) of tasks and run by an **orchestrator** such as Apache Airflow, Dagster, Prefect or Kubeflow Pipelines. The orchestrator schedules tasks, respects dependencies, retries failures and records lineage. Good pipelines are **idempotent** (running a task twice for the

same input yields the same output, so failed runs can be retried safely) and **incremental** (each run processes only new or changed data). Machine learning adds its own steps to the DAG: feature computation, training-set assembly, model training, evaluation and registration.

3.5.2 Batch and Streaming Processing

Batch processing handles data in large, bounded chunks on a schedule, for example nightly. It is simple, efficient and easy to reprocess, and it suffices for most model training and for predictions that can be pre-computed. **Stream processing** handles unbounded data continuously as events arrive, with latencies from milliseconds to seconds. It is needed when the value of a prediction decays quickly: fraud scoring at the moment of payment, real-time recommendations, or anomaly detection in sensor data. Streams are carried by durable logs such as Apache Kafka and processed by engines such as Apache Flink, Spark Structured Streaming or Kafka Streams, which provide windowing (tumbling, sliding and session windows), stateful aggregation and handling of late-arriving events through watermarks.

Two architectural styles combine the modes. The **Lambda architecture** runs a batch layer for accurate, complete results and a parallel speed layer for low-latency approximate results, merging them at query time; it is powerful but requires maintaining two code paths. The **Kappa architecture** treats everything as a stream and performs reprocessing by replaying the log, so only one code path exists. Table 3.8 compares batch and streaming processing.

Table 3.8 Batch versus stream processing for machine learning workloads.

Aspect	Batch processing	Stream processing
Data	Bounded, at rest	Unbounded, in motion
Latency	Minutes to hours	Milliseconds to seconds
Typical tools	Spark, SQL warehouses, dbt	Kafka, Flink, Spark Structured Streaming
ML use	Training, offline features, bulk scoring	Online features, real-time inference, drift alerts
Complexity	Lower; easy to backfill	Higher; state, ordering, late data
Failure recovery	Re-run the job	Checkpoints and exactly-once semantics

3.5.3 Data Lakes, Warehouses and Lakehouses

Analytical data are stored in one of three broad kinds of system. A **data warehouse** (for example, Snowflake, Google BigQuery, Amazon Redshift) stores structured, cleaned

data in tables with a schema enforced on write (*schema-on-write*), optimised for SQL analytics and business intelligence. A **data lake** stores raw data of any type (CSV, JSON, Parquet, images, audio, logs) cheaply in object storage such as Amazon S3, with structure imposed only when data are read (*schema-on-read*). Lakes suit machine learning, which often needs raw, unstructured data, but without governance they degrade into disorganised 'data swamps'.

The **lakehouse** combines the two: data live in open columnar files (Parquet) on cheap object storage, but an open *table format* such as Delta Lake, Apache Iceberg or Apache Hudi adds a transaction log that provides ACID transactions, schema enforcement and evolution, and **time travel** (querying a table as of an earlier version). Time travel is especially valuable for machine learning because it allows the exact training data of a past model to be reconstructed. Table 3.9 summarises the trade-offs.

Table 3.9 Comparison of data warehouses, data lakes and lakehouses.

Property	Data warehouse	Data lake	Lakehouse
Data types	Structured	Any (incl. unstructured)	Any
Schema	On write	On read	On write, with evolution
Storage cost	Higher	Low (object storage)	Low (object storage)
Transactions	ACID	None by default	ACID via table format
Best for	BI dashboards, SQL reporting	Raw archives, data science	Unified BI and ML
Examples	Snowflake, BigQuery, Redshift	S3, ADLS, HDFS	Delta Lake, Iceberg, Hudi

A common organising convention inside lakes and lakehouses is the **medallion architecture**: a *bronze* layer of raw ingested data, a *silver* layer of cleaned and conformed data, and a *gold* layer of business-level aggregates and model-ready feature tables. Each layer is produced from the previous one by pipeline tasks, so any downstream table can be rebuilt from bronze.

3.5.4 Data Quality and Validation

A model trained on defective data will faithfully learn the defects. **Data quality** is commonly assessed along several dimensions: *completeness* (missing values), *validity* (values conform to type, range and format), *accuracy* (values match reality), *consistency* (the same fact agrees across systems), *uniqueness* (no unintended duplicates) and *timeliness* (data are fresh enough for the use). In production these properties should be tested automatically at every pipeline stage, in the same way that software is unit-tested. Libraries such as Great Expectations, TensorFlow Data Validation and Pandera express 'expectations' about a dataset (for example, 'age is between 0 and 120', 'no

more than 1% of `country` is null') and fail the pipeline when they are violated. Algorithm 3.1 outlines a generic validation gate.

Algorithm 3.1 Data Validation Gate in an ML Pipeline

Input: New data batch B; schema S (types, ranges, allowed categories); reference statistics R from training data; tolerance thresholds τ

Output: Decision to accept or quarantine B; validation report

report ← empty list
for each column c **in** S **do**
  if type of B[c] differs from S[c].type **then** append (c, 'type error') **to** report
  m ← fraction of missing values **in** B[c]
  if m > S[c].max_missing **then** append (c, 'too many missing values') **to** report
  if any value of B[c] outside S[c].range **or** S[c].categories **then** append (c, 'invalid values')
  d ← distance(distribution of B[c], R[c]) // e.g. PSI, KS statistic
  if d > τ[c] **then** append (c, 'distribution drift') **to** report
if report contains any blocking error **then**
  quarantine B **and** alert data owners
else
  accept B **and** pass it **to** feature computation
return decision, report

Listing 3.4 implements a lightweight version of these checks in pandas, including the **population stability index** (PSI), a drift statistic widely used in credit scoring. A PSI below 0.1 is usually taken to indicate no significant shift, 0.1–0.25 a moderate shift and above 0.25 a major shift.

Listing 3.4 Simple data-quality checks and the population stability index in pandas.

```python
import numpy as np
import pandas as pd

def psi(expected, actual, bins=10):
    """Population stability index between a reference and a new sample."""
    edges = np.unique(np.quantile(expected, np.linspace(0, 1, bins + 1)))
    e = np.histogram(expected, edges)[0] / len(expected) + 1e-6
    a = np.histogram(np.clip(actual, edges[0], edges[-1]), edges)[0] / len(actual) +
    ↪ 1e-6
    return float(np.sum((a - e) * np.log(a / e)))

def validate(df, ref):
    issues = []
    if df["customer_id"].duplicated().any():
        issues.append("duplicate customer_id")
    if df["age"].isna().mean() > 0.01:
        issues.append("age: >1% missing")
    if not df["age"].dropna().between(18, 110).all():
        issues.append("age: out of range")
```

```
    if not set(df["country"].dropna()) <= set(ref["country"].unique()):
        issues.append("country: unseen categories")
    drift = psi(ref["income"].dropna(), df["income"].dropna())
    if drift > 0.25:
        issues.append(f"income: major drift (PSI={drift:.2f})")
    return issues

# issues = validate(new_batch, training_reference)
# if issues: raise ValueError(issues)
```

Practice Tip | Practical Tips for ML Data Pipelines

- Store raw data immutably and derive everything else from it with versioned code.
- Use columnar formats (Parquet) and partition large tables by date to make scans cheap.
- Record lineage: for every training set, know which sources, code version and time window produced it.
- Validate data at ingestion *and* before training *and* at serving time.
- Treat timestamps carefully: always use event time, store in UTC, and never let features use information from after the prediction time (temporal leakage).

3.6 Big Data Analytics and Machine Learning

'Big data' refers to datasets whose size, speed or variety exceeds what a single machine with conventional tools can store or process in reasonable time. The response of the 2000s, pioneered at Google and popularised by open-source Apache projects, was to distribute both storage and computation across clusters of commodity machines and to move the computation to the data rather than the other way round. This section introduces the defining characteristics of big data, the MapReduce model, and Apache Spark with its machine learning library MLlib.

3.6.1 The Five Vs of Big Data

Big data are often characterised by five Vs, summarised in Table 3.10. The original three (volume, velocity, variety) were described by Doug Laney in 2001; veracity and value were added later as the field matured.

3.6.2 Hadoop and the MapReduce Programming Model

Dean and Ghemawat (2004), in 'MapReduce: Simplified Data Processing on Large Clusters' (OSDI), described a programming model in which a computation is ex-

Table 3.10 The five Vs of big data and their implications for machine learning.

V	Meaning	Implication for ML	Example
Volume	Scale of data (TB–PB)	Distributed storage and training	Web-scale text corpora
Velocity	Speed of arrival	Streaming features, online learning	Click streams, IoT sensors
Variety	Heterogeneous formats	Multimodal models, flexible schemas	Text, images, logs, tables
Veracity	Uncertain quality	Cleaning, robust losses, label noise	Crowd-sourced labels
Value	Usefulness for decisions	Prioritise data that improve outcomes	Churn signals

pressed as two user functions. **Map** processes each input record independently and emits intermediate key–value pairs; the framework then **shuffles** the pairs so that all values for the same key arrive at the same worker; **Reduce** aggregates the values for each key. Apache Hadoop is the open-source implementation, combining MapReduce with the Hadoop Distributed File System (HDFS), which splits files into large blocks (128 MB by default) replicated across nodes, and later the YARN resource manager. Algorithm 3.2 gives the canonical word-count example and Figure 3.9 shows the data flow.

Algorithm 3.2 MapReduce Word Count

Input: Collection of documents split into blocks across M mapper nodes; R reducer nodes
Output: Count of every distinct word

```
// Map phase (runs in parallel on each block)
function MAP(doc_id, text)
    for each word w in tokenize(text) do
        emit (w, 1)
// Optional combiner: sum counts locally on each mapper to cut network traffic
// Shuffle phase (framework): partition keys by hash(w) mod R, sort, group
// Reduce phase (runs in parallel on each partition)
function REDUCE(w, list_of_counts)
    total ← 0
    for each c in list_of_counts do
        total ← total + c
    emit (w, total)
```

MapReduce made fault-tolerant parallel processing accessible, but it is poorly suited to iterative machine learning. Each iteration of, say, gradient descent or k-means becomes a separate job that reads its input from disk and writes its output back to disk, so a hundred iterations means a hundred full passes through HDFS.

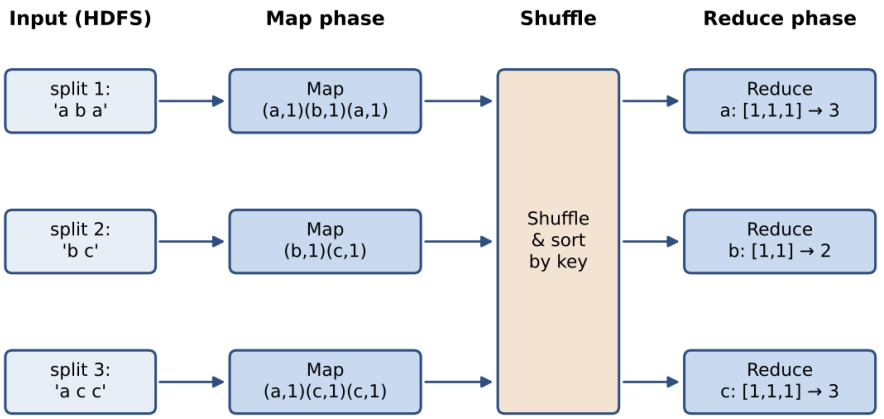


Figure 3.9 Data flow of a MapReduce job: input splits are processed by parallel mappers, intermediate key–value pairs are shuffled and sorted by key, and reducers aggregate each key's values.

3.6.3 Apache Spark and MLlib

Apache Spark, which originated at UC Berkeley's AMPLab (Zaharia et al., 2010 and 2012), addressed this limitation by keeping working data **in memory** across operations. Its original abstraction, the **resilient distributed dataset** (RDD), is an immutable, partitioned collection that records the *lineage* of transformations that produced it; if a partition is lost, Spark recomputes it from the lineage rather than relying on replication. Transformations (`map`, `filter`, `join`) are **lazy**: they build a DAG of operations that is optimised and executed only when an *action* (`count`, `collect`, `write`) is called. Higher-level **DataFrames** and Spark SQL add a schema and a query optimiser (Catalyst), and are now the preferred interface.

MLlib is Spark's machine learning library. It provides distributed implementations of common algorithms (linear and logistic regression, decision trees, random forests, gradient-boosted trees, k-means, alternating least squares for recommendation, PCA) and, importantly, a **Pipeline** API modelled on scikit-learn: a pipeline chains *Transformers* (which map one DataFrame to another) and *Estimators* (which are fitted to data and produce a Transformer). Listing 3.5 builds a complete pipeline in PySpark for a churn-prediction task.

Listing 3.5 A churn-prediction pipeline with PySpark MLlib.

```
from pyspark.sql import SparkSession
from pyspark.ml import Pipeline
from pyspark.ml.feature import StringIndexer, OneHotEncoder, VectorAssembler,
    StandardScaler
from pyspark.ml.classification import GBTClassifier
from pyspark.ml.evaluation import BinaryClassificationEvaluator
```

```

spark = SparkSession.builder.appName("churn").getOrCreate()
df = spark.read.parquet("s3://bucket/silver/customers/")      # distributed read

cat_cols, num_cols = ["plan", "region"], ["tenure", "monthly_spend", "support_calls"]
stages = [StringIndexer(inputCol=c, outputCol=c + "_idx", handleInvalid="keep") for c
↪  in cat_cols]
stages += [OneHotEncoder(inputCols=[c + "_idx" for c in cat_cols],
                         outputCols=[c + "_oh" for c in cat_cols])]
stages += [VectorAssembler(inputCols=num_cols + [c + "_oh" for c in cat_cols],
↪  outputCol="raw"),
           StandardScaler(inputCol="raw", outputCol="features"),
           GBTClassifier(labelCol="churned", featuresCol="features", maxIter=50)]

train, test = df.randomSplit([0.8, 0.2], seed=42)
model = Pipeline(stages=stages).fit(train)                    # trains across the
↪  cluster
auc = BinaryClassificationEvaluator(labelCol="churned").evaluate(model.transform(test))
print(f"Test AUC = {auc:.3f}")
model.write().overwrite().save("s3://bucket/models/churn_gbt")
```

For deep learning on large data, the same principles apply but the tools differ: data are typically stored as sharded files (TFRecord, WebDataset, Parquet) and streamed to GPUs, while training is distributed by data parallelism (each GPU processes a slice of every mini-batch and gradients are averaged with an all-reduce), model parallelism, or both. Frameworks such as Horovod, PyTorch Distributed Data Parallel, DeepSpeed and Ray Train handle the communication. Table 3.11 compares the main big-data processing frameworks.

Table 3.11 Big-data processing frameworks relevant to machine learning.

Framework	Processing model	Strength	ML support
Hadoop MapReduce	Disk-based batch	Very large, fault-tolerant batch jobs	Limited (Mahout, legacy)
Apache Spark	In-memory DAG, micro-batch streams	Iterative analytics, SQL	MLlib pipelines
Apache Flink	True streaming	Low-latency stateful streams	FlinkML, online features
Dask	Python task graphs	Scales pandas/NumPy code	Dask-ML, scikit-learn backend
Ray	Distributed actors and tasks	Python-native, GPU-friendly	Ray Train, Tune, Serve

Note | Do You Really Need a Cluster?

Many datasets described as 'big' fit comfortably on a single modern machine with tens or hundreds of gigabytes of RAM. Single-node engines such as DuckDB and Polars, which process columnar data with vectorised execution, are often faster than a small cluster because they avoid network shuffles. Reach for distributed processing when data or computation gen-

> uinely exceed one machine.

3.7 Feature Stores and Data Versioning

As organisations deploy many models, two recurring problems appear. First, the same features (for example, 'number of transactions by this customer in the last 7 days') are re-implemented by different teams, in different languages, for training and for serving, and the implementations drift apart. Second, nobody can say exactly which data a given model was trained on, so results cannot be reproduced or audited. **Feature stores** address the first problem and **data versioning** the second.

3.7.1 Training–Serving Skew

Training–serving skew is a difference between the data a model sees during training and the data it receives in production. It has three common causes: (i) *code skew*, where features are computed by a batch SQL query for training but re-implemented in an application language for serving, with subtle differences in rounding, time zones or missing-value handling; (ii) *temporal skew* or *leakage*, where training features accidentally include information that would not have been available at prediction time; and (iii) *distribution skew*, where the serving population differs from the training population. The first two are engineering defects that a feature store is designed to remove; the third is a monitoring problem (see Section 11 on concept drift).

> **D Definition 3.2 | Point-in-Time Correctness**
>
> A training example labelled at time t must be joined only with feature values whose timestamps are at or before t. A join that respects this rule is called **point-in-time correct** (or an 'as-of' join). Violating it leaks future information into training, producing optimistic offline metrics that collapse in production.

3.7.2 Feature Store Architecture

A **feature store** is a central system for defining, computing, storing and serving features. Its architecture, shown in Figure 3.10, has four parts. A **feature registry** holds declarative definitions (name, entity key, data type, source, transformation, owner, freshness). A **transformation layer** computes features from batch and streaming sources using a single definition. An **offline store** (a warehouse or lakehouse) keeps the full history of feature values with timestamps, used to build point-in-time-correct

training sets. An **online store** (a low-latency key–value database such as Redis or DynamoDB) keeps only the latest value per entity and serves it to models in milliseconds. A *materialisation* job copies fresh values from offline to online.

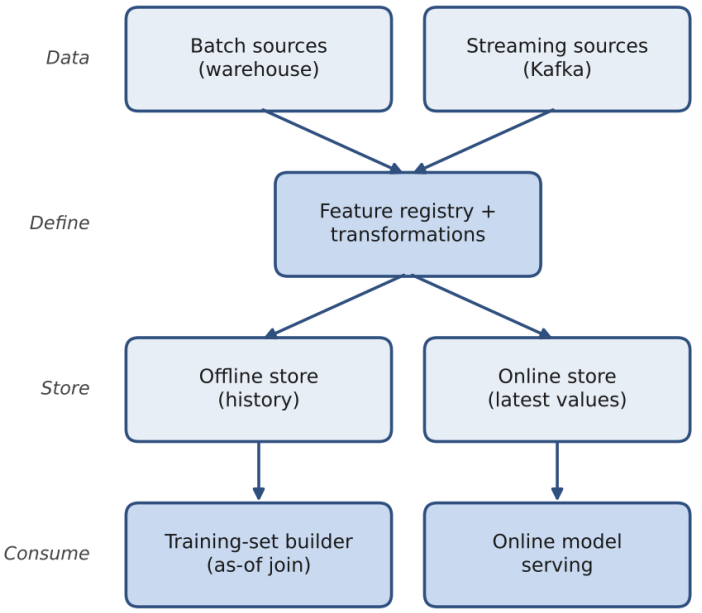


Figure 3.10 Feature store architecture: one set of feature definitions populates an offline store for point-in-time-correct training data and an online store for low-latency serving, eliminating training–serving skew.

Table 3.12 contrasts the two stores. Open-source **Feast** (originally developed at Gojek with Google Cloud) is the best-known standalone feature store; commercial and platform offerings include Tecton, Hopsworks, Databricks Feature Store, Amazon SageMaker Feature Store and Vertex AI Feature Store. Listing 3.6 shows how a feature view is declared in Feast and how the same definition yields both a historical training frame and online features.

Listing 3.6 Defining and retrieving features with Feast.

```
from datetime import timedelta
import pandas as pd
from feast import Entity, FeatureView, Field, FileSource, FeatureStore
from feast.types import Float32, Int64

customer = Entity(name="customer", join_keys=["customer_id"])
txn_source = FileSource(path="data/txn_features.parquet", timestamp_field="event_ts")

```

Table 3.12 Offline versus online feature stores.

Property	Offline store	Online store
Contents	Full time-stamped history	Latest value per entity key
Access pattern	Large scans and as-of joins	Point look-ups by key
Latency	Seconds to minutes	Single-digit milliseconds
Typical backend	BigQuery, Snowflake, Parquet/Delta	Redis, DynamoDB, Cassandra
Consumer	Training and batch scoring	Real-time inference

```
txn_stats = FeatureView(
    name="txn_stats", entities=[customer], ttl=timedelta(days=7), source=txn_source,
    schema=[Field(name="txn_count_7d", dtype=Int64), Field(name="avg_amount_7d",
    ↪ dtype=Float32)])

store = FeatureStore(repo_path=".")      # after `feast apply` registers the
↪ definitions

# Offline: point-in-time-correct training data for labelled events
labels = pd.DataFrame({"customer_id": [101, 102],
                       "event_timestamp": pd.to_datetime(["2026-03-01", "2026-03-02"]),
                       "is_fraud": [0, 1]})
train_df = store.get_historical_features(
    entity_df=labels, features=["txn_stats:txn_count_7d",
    ↪ "txn_stats:avg_amount_7d"]).to_df()

# Online: latest values for real-time scoring (after `feast materialize`)
online = store.get_online_features(
    features=["txn_stats:txn_count_7d", "txn_stats:avg_amount_7d"],
    entity_rows=[{"customer_id": 101}]).to_dict()
```

3.7.3 Data Versioning with DVC and Related Tools

Source code is versioned with Git, but Git is not designed for multi-gigabyte datasets or model files. **Data Version Control** (DVC) extends Git: large files are stored in a remote cache (S3, Google Cloud Storage, Azure, SSH or a local drive), and Git tracks only small `.dvc` pointer files containing content hashes. Checking out an old commit and running `dvc checkout` therefore restores the exact data that accompanied that code. DVC also defines **pipelines** in a `dvc.yaml` file, recording each stage's command, inputs and outputs, so `dvc repro` re-runs only the stages whose inputs changed. A typical workflow is: `dvc init`; `dvc add data/raw.csv`; `git add data/raw.csv.dvc .gitignore`; `git commit`; `dvc push`.

Other approaches suit other scales. **lakeFS** provides Git-like branches and commits over an entire object-store data lake; table formats such as **Delta Lake** and **Iceberg**

offer time travel at table level; and dataset hubs such as Hugging Face Datasets version datasets as Git repositories with large file storage. Table 3.13 compares them.

Table 3.13 Tools for data and model versioning.

Tool	Unit of versioning	Mechanism	Best suited to
Git LFS	Individual large files	Pointer files in Git	Small teams, moderate files
DVC	Files, directories, pipelines	Content hashes + remote cache	ML projects with code + data
lakeFS	Whole object-store buckets	Branches/commits over S3	Data lakes, many consumers
Delta Lake / Iceberg	Tables	Transaction log, snapshots	Lakehouse tables, time travel
MLflow Model Registry	Trained models	Versioned registry with stages	Model lifecycle management

Key Points | Feature Stores and Versioning

- Define each feature once and use the same definition for training and serving.
- Build training sets with point-in-time-correct joins to avoid leakage.
- Version data together with code so that any model can be reproduced exactly.
- A model is reproducible only if its code, data, configuration and environment are all versioned.

3.8 Model Selection and Automated Hyperparameter Optimization

Every learning algorithm has **hyperparameters**: settings that are fixed before training rather than learned from data, such as the regularisation strength of a ridge model, the depth of a decision tree or the learning rate and width of a neural network. **Model selection** is the task of choosing among candidate model families and hyperparameter values so as to maximise performance on unseen data. Chapter 1 introduced cross-validation, grid search and random search; this section adds analytic selection criteria and the automated methods (Bayesian optimisation, successive halving and Hyperband) used in modern AutoML systems.

3.8.1 Information Criteria: AIC and BIC

For probabilistic models fitted by maximum likelihood, information criteria trade goodness of fit against complexity without needing a held-out set. The **Akaike infor-**

mation criterion (Akaike, 1974) and the **Bayesian information criterion** (Schwarz, 1978) are defined in Equation (3.25); lower values are better.

$$\mathrm{AIC} = 2k - 2\log \hat{L}$$
$$\mathrm{BIC} = k\log n - 2\log \hat{L} \tag{3.25}$$

where k is the number of free parameters, n the number of observations and $\hat{L}$ the maximised likelihood.

AIC estimates expected predictive accuracy (relative KL divergence to the truth), and tends to choose somewhat larger models. BIC approximates the log marginal likelihood; its penalty grows with log n, so for $n > 7$ it penalises parameters more heavily and tends to choose more parsimonious models, being consistent for the true model when that model is among the candidates. Information criteria are standard for selecting the order of ARIMA models, the number of components in a Gaussian mixture, and variables in regression. They are not suitable for deep networks, whose effective number of parameters is far below the raw count, so cross-validation or a validation set is used instead.

E Worked Example 3.7 | Choosing the Number of Mixture Components

Gaussian mixture models with $K = 2$ and $K = 3$ components are fitted to $n = 500$ two-dimensional points. Each component has 2 means, 3 covariance entries and 1 weight, minus one weight constraint overall, so $k = 6K - 1$: $k_2 = 11$ and $k_3 = 17$. The maximised log-likelihoods are $\log \hat{L}_2 = -1520$ and $\log \hat{L}_3 = -1508$.

$\mathrm{AIC}_2 = 22 + 3040 = 3062$; $\mathrm{AIC}_3 = 34 + 3016 = 3050$, so AIC prefers $K = 3$.

$\mathrm{BIC}_2 = 11 \times \ln 500 + 3040 = 11 \times 6.215 + 3040 = 3108.4$; $\mathrm{BIC}_3 = 17 \times 6.215 + 3016 = 3121.7$, so BIC prefers $K = 2$. The 12-unit likelihood gain does not justify six extra parameters under BIC's stronger penalty.

3.8.2 Bayesian Optimisation

Grid and random search treat each trial independently. **Bayesian optimisation** (BO) instead builds a probabilistic **surrogate model** of the objective $f(\lambda)$ (for example, validation loss as a function of hyperparameters λ) from the trials so far, and uses it to choose the next trial intelligently. Because each evaluation of f may mean training a model for hours, spending a little computation to choose evaluations well is worthwhile. Snoek, Larochelle and Adams (2012), 'Practical Bayesian Optimization of Machine Learning Algorithms' (NeurIPS), popularised the approach for machine learning.

The classical surrogate is a **Gaussian process** (GP), which places a distribution over functions. Conditioned on observations, it gives at every candidate λ a predictive mean

$\mu(\lambda)$ and standard deviation $\sigma(\lambda)$. An **acquisition function** turns these into a score that balances *exploitation* (sampling where μ is good) and *exploration* (sampling where σ is large). The most widely used acquisition function is **expected improvement** (EI). For minimisation with current best value f*, EI has the closed form of Equation (3.26).

$$\begin{aligned} \mathrm{EI}(\lambda) &= \mathbb{E}[\max(0, f^* - f(\lambda))] \\ &= (f^* - \mu(\lambda) - \xi)\,\Phi(z) + \sigma(\lambda)\,\phi(z), \qquad z = \frac{f^* - \mu(\lambda) - \xi}{\sigma(\lambda)} \end{aligned} \tag{3.26}$$

where Φ and φ are the standard normal CDF and PDF, and $\xi \geq 0$ is a small exploration margin ($\mathrm{EI} = 0$ when $\sigma = 0$).

The first term rewards points predicted to be better than the incumbent; the second rewards uncertainty. Figure 3.11 shows a GP surrogate after five evaluations of a one-dimensional objective and the corresponding EI curve, whose maximum determines the next trial. Algorithm 3.3 states the loop. Alternative surrogates include random forests (SMAC) and the **tree-structured Parzen estimator** (TPE; Bergstra et al., 2011), which models the densities of good and bad configurations separately and scales well to conditional, mixed-type search spaces; TPE is Optuna's default sampler.

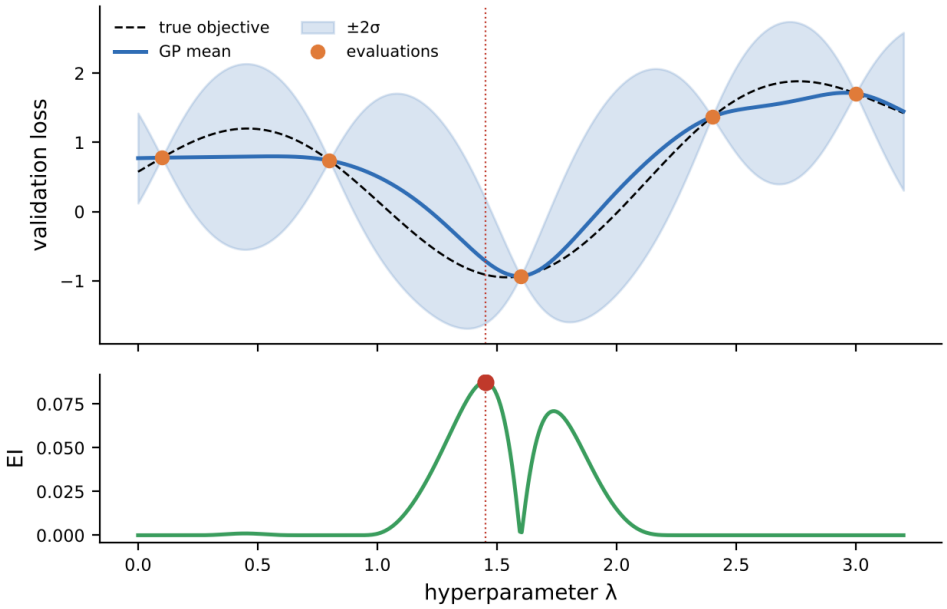


Figure 3.11 Bayesian optimisation in one dimension. Top: Gaussian-process posterior mean and $\pm 2\sigma$ band after five evaluations of the unknown objective (dashed). Bottom: expected improvement; its maximum (red marker) is the next hyperparameter value to evaluate.

Algorithm 3.3 Bayesian Optimisation with a Gaussian-Process Surrogate

Input: Objective f (e.g. validation loss); search space Λ; initial design size n_0; budget T
Output: Best configuration λ^* found

evaluate f at n_0 points $\lambda_1 \ldots \lambda_{n0}$ sampled at random (**or** by a Latin hypercube)
$D \leftarrow \{(\lambda_i, f(\lambda_i))\}$
for $t = n_0 + 1$ **to** T **do**
    fit GP surrogate **to** D (optimise kernel hyperparameters by marginal likelihood)
    $f^* \leftarrow$ min of observed values **in** D
    $\lambda_t \leftarrow$ argmax over $\lambda \in \Lambda$ of EI(λ) // inner optimisation is cheap
    $y_t \leftarrow f(\lambda_t)$ // expensive: train **and** validate a model
    $D \leftarrow D \cup \{(\lambda_t, y_t)\}$
return $\lambda^* \leftarrow$ argmin over D of observed value

3.8.3 Successive Halving and Hyperband

Bayesian optimisation chooses *which* configurations to try; **multi-fidelity** methods decide *how much* budget to spend on each. Most poor configurations are recognisably poor after a fraction of their full training budget (a few epochs, a subset of the data). **Successive halving** (Jamieson and Talwalkar, 2016) starts many configurations with a small budget, keeps the best $1/\eta$ fraction, multiplies their budget by η, and repeats. The difficulty is choosing how many configurations to start with: many configurations with tiny budgets risk discarding slow starters, while few configurations with large budgets explore poorly. **Hyperband** (Li et al., 2018, JMLR) hedges by running several successive-halving 'brackets' with different trade-offs, as described in Algorithm 3.4 and illustrated in Figure 3.12.

Algorithm 3.4 Hyperband (with Successive Halving as the inner loop)

Input: Maximum budget per configuration R (e.g. epochs); reduction factor η (typically 3); configuration sampler
Output: Best configuration found

$s_{max} \leftarrow \lfloor \log_\eta R \rfloor$; $B \leftarrow (s_{max} + 1)\, R$
for $s = s_{max}$ down **to** 0 **do** // one bracket per s
    $n \leftarrow \lceil (B / R) \cdot \eta^s / (s + 1) \rceil$ // number of configurations
    $r \leftarrow R \cdot \eta^{\wedge}(-s)$ // initial budget **each**
    C ← sample n configurations
    for $i = 0$ **to** s **do** // successive halving
        $n_i \leftarrow \lfloor n \cdot \eta^{\wedge}(-i) \rfloor$; $r_i \leftarrow r \cdot \eta^i$
        train **each** $c \in C$ with budget r_i **and** record validation loss L(c)
        C ← the $\lfloor n_i / \eta \rfloor$ configurations **in** C with lowest L
return configuration with lowest loss observed at any budget

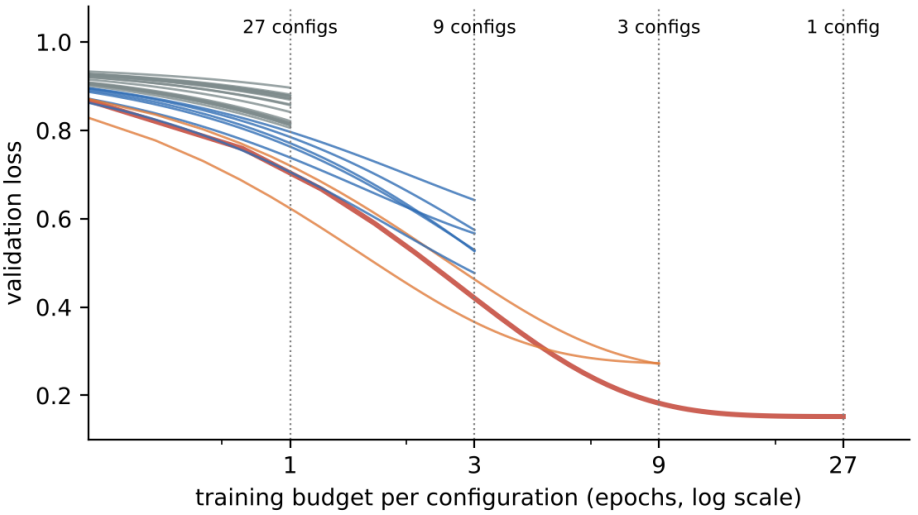


Figure 3.12 Successive halving with 27 configurations and $\eta = 3$: after each rung only the best third of configurations survive and receive three times the training budget, so most compute goes to promising candidates.

Combining the two ideas gives **BOHB** (Falkner et al., 2018), which replaces Hyperband's random sampling with a TPE-style model, and the **asynchronous successive halving algorithm** (ASHA), which promotes configurations as soon as enough results are available and therefore uses large parallel clusters efficiently. Table 3.14 compares the main hyperparameter-optimisation strategies.

Table 3.14 Hyperparameter optimisation strategies compared.

Method	Idea	Strengths	Weaknesses
Grid search	Evaluate every combination	Simple, exhaustive in low dimensions	Exponential cost; wastes trials
Random search	Sample configurations at random	Embarrassingly parallel; good baseline	Ignores past results
Bayesian optimisation (GP)	Surrogate + acquisition function	Sample-efficient for few, continuous HPs	Sequential; cubic GP cost
TPE	Model densities of good vs bad trials	Handles conditional, mixed spaces	Weaker at modelling interactions
Successive halving / Hyperband	Early-stop poor configurations	Large speed-ups, parallel	Assumes early ranking is informative
BOHB / ASHA	Model-based + multi-fidelity	State of the art for deep learning	More moving parts
Evolutionary / PBT	Mutate and select a population	Adapts schedules during training	Needs many workers

Listing 3.7 uses **Optuna** (Akiba et al., 2019) to tune a gradient-boosting classifier with the TPE sampler and a Hyperband pruner. Pruning requires the objective to report intermediate scores so that unpromising trials can be stopped early.

Listing 3.7 Hyperparameter optimisation with Optuna using TPE sampling and Hyperband pruning.

```python
import optuna
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.metrics import roc_auc_score

X, y = load_breast_cancer(return_X_y=True)
X_tr, X_va, y_tr, y_va = train_test_split(X, y, test_size=0.25, stratify=y,
    random_state=0)

def objective(trial):
    params = dict(learning_rate=trial.suggest_float("learning_rate", 1e-3, 0.3,
        log=True),
                  max_depth=trial.suggest_int("max_depth", 2, 10),
                  l2_regularization=trial.suggest_float("l2", 1e-8, 10.0, log=True),
                  min_samples_leaf=trial.suggest_int("min_samples_leaf", 5, 100))
    auc = 0.0
    for step, n_iter in enumerate([25, 75, 225]):            # increasing fidelity
        model = HistGradientBoostingClassifier(max_iter=n_iter, random_state=0,
            **params)
        auc = roc_auc_score(y_va, model.fit(X_tr, y_tr).predict_proba(X_va)[:, 1])
        trial.report(auc, step)
        if trial.should_prune():                              # Hyperband decides
            raise optuna.TrialPruned()
    return auc

study = optuna.create_study(direction="maximize",
                            sampler=optuna.samplers.TPESampler(seed=0),
                            pruner=optuna.pruners.HyperbandPruner(min_resource=1,
                                max_resource=3))
study.optimize(objective, n_trials=60)
print(study.best_value, study.best_params)
```

> **Note | Avoiding Optimistic Bias in Model Selection**
>
> The validation score of the best configuration is a biased estimate of its true performance, because the configuration was chosen *for* scoring well on that data. Report final performance on a separate test set that played no part in tuning, or use **nested cross-validation**: an inner loop tunes hyperparameters and an outer loop estimates the performance of the whole tuning procedure.

3.9 Causal Machine Learning

Standard supervised learning estimates associations: it learns $\mathrm{E}[Y \mid X]$, the expected outcome given what we observe. Many important questions, however, are about **interventions**: Will this drug reduce blood pressure? Will a discount make this customer

stay? Would this loan applicant have repaid if approved? Answering them requires estimating what *would* happen if we changed something, which is a causal question. **Causal machine learning** combines the formal frameworks of causal inference with flexible machine learning estimators to answer such questions from data.

3.9.1 Correlation Versus Causation

Two variables can be strongly associated without either causing the other. The most common reason is **confounding**: a third variable influences both. Ice-cream sales and drowning deaths are correlated because both rise in hot weather. In medicine, patients who receive an aggressive treatment may have *worse* outcomes simply because doctors reserve the treatment for the sickest patients; here disease severity confounds the treatment–outcome relationship. Other sources of spurious association include **reverse causation** (the outcome causes the 'treatment') and **selection bias** (the data were filtered on a variable affected by both).

E Worked Example 3.8 | Simpson's Paradox in Treatment Data

A hospital records recovery rates for two treatments, split by severity.

- Mild cases: Treatment A 81/87 = 93%, Treatment B 234/270 = 87%.
- Severe cases: Treatment A 192/263 = 73%, Treatment B 55/80 = 69%.
- Combined: Treatment A 273/350 = 78%, Treatment B 289/350 = 83%.

A is better within each severity group, yet B looks better overall, because A was given mostly to severe cases (263 of 350), which recover less often. Severity is a confounder: conditioning on it reveals the causal comparison, while the pooled figures mislead. (The numbers follow the classic kidney-stone study of Charig et al., 1986.)

A predictive model trained on such data will happily learn the misleading pooled association. That is fine for *prediction* (which patients will recover?) but disastrous for *decision-making* (which treatment should we give?). Causal inference makes explicit the assumptions under which an association can be interpreted causally.

3.9.2 Potential Outcomes and Treatment Effects

The **potential outcomes** framework, developed by Neyman (1923) and Rubin (1974), defines causal effects by comparing hypothetical worlds. For a binary treatment $T \in \{0, 1\}$, each unit i has two potential outcomes: $Y_i(1)$ if treated and $Y_i(0)$ if not. The **individual treatment effect** is $Y_i(1) - Y_i(0)$. The **fundamental problem of causal inference** is that we only ever observe one of the two outcomes for each unit; the other is *counterfactual*. We therefore target averages, defined in Equation (3.27).

$$\begin{aligned} \mathrm{ATE} &= \mathbb{E}[Y(1) - Y(0)] \\ \mathrm{CATE}(\mathbf{x}) &= \tau(\mathbf{x}) = \mathbb{E}[Y(1) - Y(0) \mid \mathbf{X} = \mathbf{x}] \end{aligned} \tag{3.27}$$

where ATE is the average treatment effect over the population and CATE the conditional average treatment effect for units with covariates x.

In a **randomised controlled trial** (RCT), treatment is assigned by coin flip, so treated and untreated groups are comparable on average and the ATE is simply the difference in mean outcomes. With **observational data** we need assumptions. The key ones are: (i) **unconfoundedness** (also called ignorability or 'no unmeasured confounders'): given the observed covariates X, treatment assignment is independent of the potential outcomes; (ii) **overlap** (positivity): every unit has a non-zero probability of receiving either treatment, $0 < \mathrm{P}(\mathrm{T}=1 \mid \mathrm{X}=\mathrm{x}) < 1$; and (iii) **consistency** and no interference between units (SUTVA). Under these assumptions the ATE is identified from observable quantities.

3.9.3 Causal Graphs, the do-Operator and Backdoor Adjustment

Judea Pearl's structural approach (Pearl, 2009, *Causality*, Cambridge University Press) represents causal assumptions as a **directed acyclic graph** (DAG) in which an arrow A → B means 'A is a direct cause of B'. Figure 3.13 shows the three elementary structures. In a *chain* (X → M → Y) and a *fork* (X ← Z → Y) the end variables are associated, and conditioning on the middle variable blocks the association. In a *collider* (X → C ← Y) the end variables are independent, but conditioning on the collider *creates* a spurious association; this is the graphical explanation of selection bias.

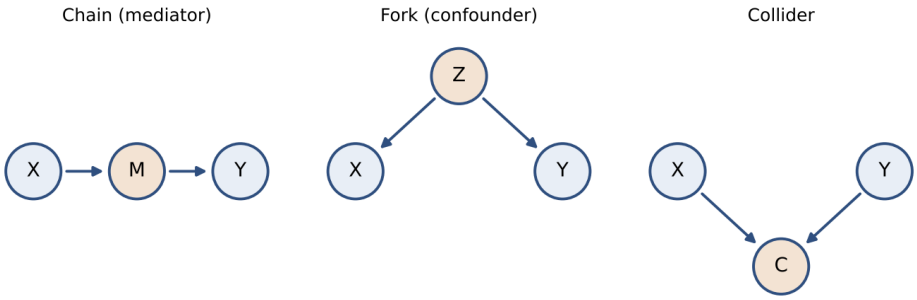


Figure 3.13 The three elementary causal structures. Conditioning on the mediator M (chain) or the confounder Z (fork) blocks the path between X and Y; conditioning on the collider C opens it.

Pearl's **do-operator** distinguishes observing from intervening. $\mathrm{P}(\mathrm{Y} \mid \mathrm{X}=\mathrm{x})$ is the distribution of Y among units that *happen* to have $\mathrm{X}=\mathrm{x}$; $\mathrm{P}(\mathrm{Y} \mid \mathrm{do}(\mathrm{X}=\mathrm{x}))$ is the distribution

if we *set* X to x for everyone, cutting all arrows into X. The **do-calculus** is a set of three rules for transforming interventional expressions into observational ones when the graph allows. Its most used consequence is the **backdoor criterion**: a set of variables Z satisfies it relative to (T, Y) if no element of Z is a descendant of T and Z blocks every path between T and Y that has an arrow *into* T. If Z satisfies the criterion, the causal effect is given by the **backdoor adjustment formula**, Equation (3.28).

$$P(Y \mid do(T=t)) = \sum_{z} P(Y \mid T=t, Z=z)\, P(Z=z) \tag{3.28}$$

where the sum runs over the values of the adjustment set Z (an integral for continuous Z).

Applied to the Simpson's paradox example, with Z = severity, the adjusted recovery rate for A is $0.93 \times \mathrm{P}(\text{mild}) + 0.73 \times \mathrm{P}(\text{severe})$, with $\mathrm{P}(\text{mild}) = 357/700 = 0.51$, which gives 0.83; for B it is $0.87 \times 0.51 + 0.69 \times 0.49 = 0.78$. Treatment A is causally better by about five percentage points. The formula also makes clear what can go wrong: adjusting for a mediator removes part of the effect we want to measure, and adjusting for a collider introduces bias. Causal graphs make such choices explicit and auditable.

3.9.4 Propensity Scores

When the adjustment set X is high-dimensional, stratifying on every combination of values is impossible. Rosenbaum and Rubin (1983) showed that it suffices to adjust for a single scalar, the **propensity score** $\mathrm{e}(\mathrm{x}) = \mathrm{P}(\mathrm{T} = 1 \mid \mathrm{X} = \mathrm{x})$, the probability of treatment given covariates. It can be estimated with any probabilistic classifier, such as logistic regression or gradient boosting. Units are then matched or stratified on $\hat{\mathrm{e}}(\mathrm{x})$, or reweighted by **inverse probability weighting** (IPW), Equation (3.29), which creates a pseudo-population in which treatment is independent of X.

$$\widehat{\mathrm{ATE}}_{\mathrm{IPW}} = \frac{1}{n} \sum_{i=1}^{n} \left[\frac{T_i Y_i}{\hat{e}(\mathbf{x}_i)} - \frac{(1-T_i)\, Y_i}{1-\hat{e}(\mathbf{x}_i)} \right] \tag{3.29}$$

where $\hat{\mathrm{e}}(\mathrm{x}_\mathrm{i})$ is the estimated propensity score of unit i.

IPW is unbiased if the propensity model is correct, but extreme scores near 0 or 1 produce huge weights and high variance; scores are therefore often clipped (for example to $[0.01, 0.99]$) and overlap should always be checked by plotting the propensity distributions of the two groups. **Doubly robust** estimators such as augmented IPW (AIPW) combine a propensity model with outcome models and remain consistent if *either* is correctly specified.

3.9.5 Uplift Modelling and Double Machine Learning

In marketing, the CATE is called **uplift**: the increase in purchase probability caused by contacting a particular customer. Uplift modelling divides customers into four groups: *persuadables* (buy only if contacted), *sure things* (buy anyway), *lost causes* (never buy) and *sleeping dogs* (contact makes them *less* likely to buy). A response model targets sure things and persuadables alike, wasting money on the former; an uplift model targets only persuadables. CATE estimators built on standard learners are known as **meta-learners**; Table 3.15 summarises the main ones (Künzel et al., 2019).

Table 3.15 Meta-learners for estimating conditional average treatment effects.

Learner	Construction	Remarks
S-learner	One model $\mu(x, t)$; $\tau(x) = \mu(x, 1) - \mu(x, 0)$	Simple; may shrink effect towards zero
T-learner	Separate models $\mu_1(x)$, $\mu_0(x)$ on treated and control	Flexible; noisy with small groups
X-learner	Impute individual effects, then model them, weight by $\hat{e}(x)$	Good with unbalanced groups
R-learner / DML	Regress outcome residuals on treatment residuals	Orthogonal; robust to nuisance errors
Causal forest	Random forest splitting on effect heterogeneity	Honest trees, valid confidence intervals

Double (or debiased) machine learning (DML; Chernozhukov et al., 2018, *The Econometrics Journal*) addresses a subtle problem: plugging flexible ML predictions into a causal formula transfers their regularisation bias into the effect estimate. Consider the partially linear model $Y = \theta T + g(X) + \varepsilon$, $T = m(X) + v$. DML (i) fits ML models for the *nuisance* functions $\ell(X) = E[Y \mid X]$ and $m(X) = E[T \mid X]$ using **cross-fitting** (each unit's predictions come from models trained on other folds), (ii) forms the residuals $\tilde{Y} = Y - \hat{\ell}(X)$ and $\tilde{T} = T - \hat{m}(X)$, and (iii) regresses one on the other, Equation (3.30).

$$\hat{\theta} = \frac{\sum_i \tilde{T}_i \tilde{Y}_i}{\sum_i \tilde{T}_i^2}, \qquad \tilde{Y}_i = Y_i - \hat{\ell}(\mathbf{x}_i), \qquad \tilde{T}_i = T_i - \hat{m}(\mathbf{x}_i) \tag{3.30}$$

where $\hat{\ell}$ and $\hat{m}$ are cross-fitted ML predictions of the outcome and treatment from the covariates.

This residual-on-residual regression is *Neyman orthogonal*: small errors in the nuisance models affect θ only to second order, so θ is $\sqrt{n}$-consistent with valid confidence intervals even when the nuisance models are random forests or neural networks. Algorithm 3.5 gives the procedure and Listing 3.8 applies DML with the EconML library.

Algorithm 3.5 Double Machine Learning with Cross-Fitting (Partially Linear Model)

Input: Data $\{(x_i, T_i, Y_i)\}_{i=1}^{n}$; ML regressors for ℓ and m; number of folds K
Output: Estimate θ of the treatment effect and its standard error

split indices into K folds $I_1, \ldots, I_K$
for $k = 1$ **to** K **do**
  fit $\hat{\ell}_{-k}$ on data outside I_k **to** predict Y from X
  fit $\hat{m}_{-k}$ on data outside I_k **to** predict T from X
  for i **in** I_k **do**
    $\tilde{Y}_i \leftarrow Y_i - \hat{\ell}_{-k}(x_i)$; $\tilde{T}_i \leftarrow T_i - \hat{m}_{-k}(x_i)$
$\theta \leftarrow \sum_i \tilde{T}_i \tilde{Y}_i \,/\, \sum_i \tilde{T}_i^2$
compute standard error from residuals $\psi_i = (\tilde{Y}_i - \theta \tilde{T}_i)\tilde{T}_i$
return θ, $\mathrm{SE}(\theta)$

Listing 3.8 Estimating heterogeneous treatment effects with double machine learning (EconML).

```python
import numpy as np
from econml.dml import LinearDML, CausalForestDML
from sklearn.ensemble import GradientBoostingRegressor, GradientBoostingClassifier

rng = np.random.default_rng(0)
n = 5000
X = rng.normal(size=(n, 5))                                  # customer covariates
propensity = 1 / (1 + np.exp(-(0.8 * X[:, 0] - 0.5 * X[:, 1])))
T = rng.binomial(1, propensity)                              # confounded treatment
↪ (discount)
tau = 1.0 + 2.0 * (X[:, 2] > 0)                              # true heterogeneous effect
Y = tau * T + X[:, 0] + np.sin(X[:, 1]) + rng.normal(size=n)

dml = LinearDML(model_y=GradientBoostingRegressor(),
↪  model_t=GradientBoostingClassifier(),
                discrete_treatment=True, cv=5, random_state=0)
dml.fit(Y, T, X=X)
print("ATE:", dml.ate(X), "95% CI:", dml.ate_interval(X))

cf = CausalForestDML(discrete_treatment=True, cv=5, random_state=0).fit(Y, T, X=X)
cate = cf.effect(X)                                          # per-customer uplift
print("mean CATE where X2>0:", cate[X[:, 2] > 0].mean())     # about 3
print("mean CATE where X2<=0:", cate[X[:, 2] <= 0].mean())   # about 1
```

Note | Assumptions Cannot Be Learned From Data Alone

No algorithm can discover from observational data whether an important confounder is missing. Causal conclusions rest on domain knowledge encoded in the graph or the unconfoundedness assumption. Good practice includes drawing the DAG with domain experts, checking covariate balance and overlap, and running **sensitivity analyses** that ask how strong an unmeasured confounder would have to be to overturn the conclusion.

3.10 Federated Learning

Machine learning traditionally assumes that training data are collected in one place. In many important settings this is impossible or undesirable: text typed on millions of phones, medical images held by separate hospitals, transaction records at different banks. Legal frameworks such as the EU General Data Protection Regulation and health-data regulations restrict sharing, and centralising sensitive data creates an attractive target for attackers. **Federated learning** (FL) trains a shared model across many data holders, called **clients**, without the raw data ever leaving them. The term and the FedAvg algorithm were introduced by McMahan et al. (2017), 'Communication-Efficient Learning of Deep Networks from Decentralized Data' (AISTATS), motivated by next-word prediction on Google's Android keyboard.

3.10.1 Architecture and Training Round

In the standard **client–server** architecture of Figure 3.14, a central server holds the global model. Each training **round** proceeds as follows: the server selects a subset of available clients and sends them the current model; each selected client trains the model for a few local epochs on its own data; clients send back only their model updates (weights or weight differences); and the server aggregates the updates into a new global model. Only parameters cross the network, never examples.

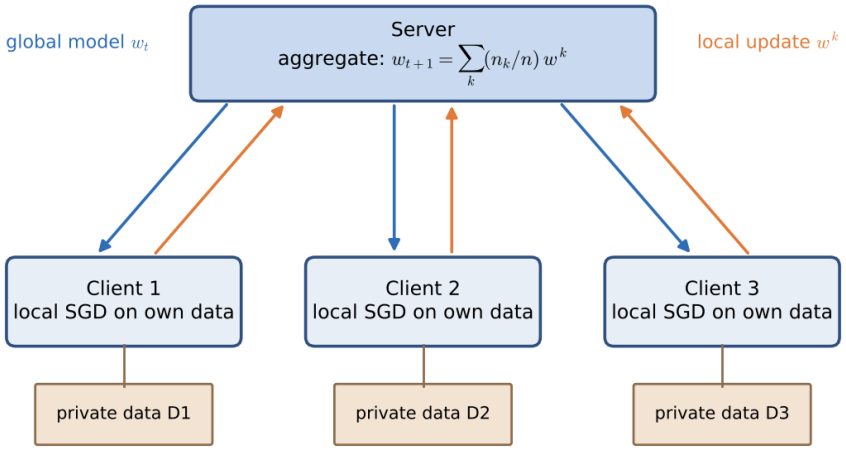


Figure 3.14 Federated learning architecture. The server broadcasts the global model w_t, clients train locally on private data, and the server aggregates the returned updates into w_{t+1}; raw data never leave the clients.

3.10.2 The FedAvg Algorithm

The federated objective is the data-weighted average of client objectives, Equation (3.31). **Federated averaging** (FedAvg) optimises it by letting each client run several steps of SGD before communicating, then averaging the resulting models in proportion to client data size, Equation (3.32). Local computation is cheap while communication over mobile networks is expensive, so performing E local epochs per round reduces the number of rounds needed by one to two orders of magnitude compared with sending a gradient after every mini-batch (FedSGD).

$$\min_{\mathbf{w}} F(\mathbf{w}) = \sum_{k=1}^{K} \frac{n_k}{n} F_k(\mathbf{w}), \qquad F_k(\mathbf{w}) = \frac{1}{n_k} \sum_{i \in D_k} \ell(\mathbf{w}; \mathbf{x}_i, y_i) \tag{3.31}$$

where client k holds n_k of the n examples in its local dataset D_k and ℓ is the per-example loss.

$$\mathbf{w}_{t+1} = \sum_{k \in S_t} \frac{n_k}{\sum_{j \in S_t} n_j} \mathbf{w}^k_{t+1} \tag{3.32}$$

where S_t is the set of clients selected in round t and $w^k{}_{t+1}$ is client k's model after local training starting from w_t.

Algorithm 3.6 Federated Averaging (FedAvg)

Input: K clients with local datasets D_k; fraction of clients per round C; local epochs E; local batch size B; learning rate η; rounds T

Output: Global model w_T

```
server initialises w_0
for t = 0 to T − 1 do
    m ← max(⌈C · K⌉, 1)
    S_t ← random subset of m clients
    for each client k ∈ S_t in parallel do
        w^k ← CLIENTUPDATE(k, w_t)
    w_{t+1} ← Σ_{k∈S_t} (n_k / Σ_{j∈S_t} n_j) · w^k
return w_T

function CLIENTUPDATE(k, w) // runs on device k
    for e = 1 to E do
        for each mini-batch b ⊂ D_k of size B do
            w ← w − η ∇ℓ(w; b)
    return w to server
```

3.10.3 Horizontal, Vertical and Cross-Silo Settings

Federated settings differ in how data are partitioned. In **horizontal** FL (sample-partitioned), clients share the same feature space but hold different examples, for instance hospitals with the same imaging protocol but different patients. In **vertical** FL (feature-partitioned), clients hold different features for the *same* entities, for example a bank and an e-commerce company with overlapping customers; training requires private set intersection to align entities and secure protocols to combine partial computations. **Federated transfer learning** handles cases with little overlap in either samples or features. Orthogonally, **cross-device** FL involves millions of unreliable, intermittently available devices, while **cross-silo** FL involves a handful of reliable organisations. Table 3.16 summarises the distinctions.

Table 3.16 Types of federated learning.

Type	Data partition	Typical example	Key technical need
Horizontal	Same features, different samples	Hospitals with similar records	Aggregation of model updates
Vertical	Same samples, different features	Bank + retailer, shared customers	Entity alignment, secure computation
Federated transfer	Little overlap in either	Cross-domain partners	Transfer of representations
Cross-device	Millions of small clients	Keyboard prediction on phones	Client sampling, compression, dropouts
Cross-silo	Few large organisations	Multi-hospital consortium	Governance, auditability, fairness

3.10.4 Challenges: Non-IID Data, Communication and Privacy

The defining difficulty of FL is **statistical heterogeneity**. Clients' data are rarely independent and identically distributed: one phone user types mostly in Tamil and another in English; one hospital sees mainly paediatric cases. With non-IID data, local models drift towards their own optima during the E local epochs (**client drift**), and averaging them can slow or destabilise convergence. Remedies include **FedProx** (Li et al., 2020), which adds a proximal term $(\mu/2)\|w - w_t\|^2$ to each local objective to keep local models close to the global one; **SCAFFOLD** (Karimireddy et al., 2020), which corrects drift with control variates; server-side adaptive optimisers (FedAdam); and **personalised FL**, which learns a shared representation plus client-specific heads, or fine-tunes the global model locally.

Systems heterogeneity (devices differ in speed, memory, battery and connectivity) is handled by sampling only available clients, tolerating stragglers and dropouts, and compressing updates through quantisation, sparsification or low-rank factorisation.

Finally, FL is *not* automatically private: model updates can leak information, and gradient-inversion attacks have reconstructed training images from shared gradients. Strong guarantees require combining FL with the techniques of Section 18: **secure aggregation**, so that the server sees only the sum of updates, and **differential privacy**, which clips and noises updates. Byzantine-robust aggregation rules (coordinate-wise median, trimmed mean, Krum) defend against malicious clients that send poisoned updates.

Listing 3.9 A minimal simulation of FedAvg for logistic regression on non-IID clients.

```python
import numpy as np
from sklearn.datasets import make_classification

rng = np.random.default_rng(0)
X, y = make_classification(n_samples=6000, n_features=20, random_state=0)
# Non-IID split: sort by label so each client is dominated by one class
order = np.argsort(y + 0.3 * rng.random(len(y)))
clients = np.array_split(order, 10)

sigmoid = lambda z: 1 / (1 + np.exp(-z))
def local_update(w, idx, epochs=2, lr=0.1, bs=32):
    w = w.copy()
    for _ in range(epochs):
        for b in np.array_split(rng.permutation(idx), max(1, len(idx) // bs)):
            grad = X[b].T @ (sigmoid(X[b] @ w) - y[b]) / len(b)
            w -= lr * grad
    return w

w = np.zeros(X.shape[1])
for rnd in range(30):
    chosen = rng.choice(len(clients), size=5, replace=False)       # C = 0.5
    sizes = np.array([len(clients[k]) for k in chosen])
    updates = [local_update(w, clients[k]) for k in chosen]
    w = np.average(updates, axis=0, weights=sizes)                  # FedAvg
    if rnd % 10 == 9:
        acc = ((sigmoid(X @ w) > 0.5) == y).mean()
        print(f"round {rnd + 1}: global accuracy = {acc:.3f}")
```

Listing 3.9 simulates FedAvg on ten deliberately non-IID clients. Production frameworks such as Flower, TensorFlow Federated, NVIDIA FLARE and PySyft provide the same logic together with networking, secure aggregation and differential privacy.

Case Study 3.1 | Federated Learning in Healthcare

Medical imaging models need data from many institutions to generalise across scanners and populations, yet patient data are tightly regulated. In a widely cited study (Dayan et al., 2021, *Nature Medicine*), twenty institutions across five continents trained the EXAM model to predict the oxygen needs of COVID-19 patients from chest X-rays and clinical data using federated learning. The federated model generalised better to external sites than models trained

at any single institution, without sharing patient records. Such consortia illustrate the cross-silo setting, where governance, auditing and fair contribution accounting matter as much as algorithms.

3.11 Online and Incremental Learning

The learning algorithms of Chapters 1 and 2 are mostly **batch** (or offline) learners: they assume the whole training set is available at once, train on it, and are then deployed unchanged. Many real systems instead receive an unending **stream** of data (clicks, transactions, sensor readings, news articles) and must update their models continually. **Online learning** processes examples one at a time (or in small mini-batches), updating the model after each and never revisiting old examples. **Incremental learning** more broadly refers to updating a model with new data without retraining from scratch. Both are essential when data are too large to store, when the model must adapt quickly, or when the underlying distribution changes.

3.11.1 The Online Learning Protocol and Online Gradient Descent

Online learning is formalised as a repeated game between a learner and an environment. At each round $t = 1, 2, \ldots, T$: the learner chooses a model w_t; the environment reveals an example (x_t, y_t) and the learner suffers loss $\ell_t(w_t) = \ell(w_t; x_t, y_t)$; the learner updates its model. No statistical assumptions about how examples are generated are required; they may even be chosen adversarially. The simplest and most widely used algorithm is **online gradient descent** (OGD; Zinkevich, 2003), Equation (3.33), which is exactly stochastic gradient descent applied to a stream, optionally followed by projection onto a feasible set.

$$\mathbf{w}_{t+1} = \Pi_{\mathcal{W}}\left(\mathbf{w}_t - \eta_t \nabla \ell_t(\mathbf{w}_t)\right) \tag{3.33}$$

where η_t is the step size at round t and $\Pi_{\mathcal{W}}$ projects back onto the feasible set $\mathcal{W}$ (omitted if unconstrained).

Algorithm 3.7 Online Gradient Descent on a Data Stream

Input: Stream of examples $(x_1, y_1), (x_2, y_2), \ldots$; loss ℓ; step-size schedule η_t; feasible set $\mathcal{W}$
Output: Sequence of models $w_1, w_2, \ldots$ and cumulative loss

1 **initialise** $w_1 \in \mathcal{W}$ (e.g. 0); $L \leftarrow 0$
2 **for** $t = 1, 2, \ldots$ **do**

receive x_t; predict $\hat{y}_t \leftarrow f(x_t; w_t)$
receive true label y_t; $L \leftarrow L + \ell(w_t; x_t, y_t)$ // prequential evaluation
$g_t \leftarrow \nabla_w \ell(w_t; x_t, y_t)$
$w_{t+1} \leftarrow \Pi_{\mathcal{W}}(w_t - \eta_t g_t)$
discard (x_t, y_t) // constant memory

Note the evaluation scheme embedded in Algorithm 3.7: each example is first used to *test* the current model and then to *train* it. This **prequential** (predictive sequential, or 'test-then-train') evaluation is the standard way to assess stream learners, because every prediction is made on data the model has not yet seen.

3.11.2 Regret

Because no distributional assumptions are made, online learners are judged by **regret**: the difference between the learner's cumulative loss and that of the best fixed model chosen in hindsight, Equation (3.34). An algorithm is called **no-regret** if R_t grows sublinearly, so that the average regret R_t/T tends to zero and the learner eventually performs as well as the best fixed model.

$$R_T = \sum_{t=1}^{T} \ell_t(\mathbf{w}_t) - \min_{\mathbf{w} \in \mathcal{W}} \sum_{t=1}^{T} \ell_t(\mathbf{w}) \qquad R_T^{\mathrm{OGD}} \le \tfrac{3}{2} G D \sqrt{T} \quad \text{with} \quad \eta_t = \tfrac{D}{G\sqrt{t}} \tag{3.34}$$

where, for convex losses, G bounds the gradient norms and D the diameter of $\mathcal{W}$.

The $O(\sqrt{T})$ bound for convex losses improves to $O(\log T)$ for strongly convex losses with step size $\eta_t \propto 1/t$. Regret analysis connects online learning to many other areas: multiplicative-weights algorithms for combining experts, multi-armed bandits (where only the loss of the chosen action is observed), and **online-to-batch conversion**, which turns any no-regret algorithm into a batch learner with a generalisation guarantee.

3.11.3 Concept Drift

A stream's distribution may change over time, a phenomenon called **concept drift**. Formally, the joint distribution $p_t(x, y)$ varies with t. If only $p(x)$ changes (for example, a new customer segment arrives) it is **covariate shift** or *virtual drift*; if $p(y \mid x)$ changes (the same symptoms now indicate a different disease, or fraudsters change tactics) it is **real drift**, and the decision boundary itself must move. Figure 3.15 illustrates the temporal patterns drift can follow: *sudden* (abrupt switch), *gradual* (the new concept

appears with increasing frequency), *incremental* (slow continuous change) and *recurring* (old concepts return, as with seasonal behaviour).

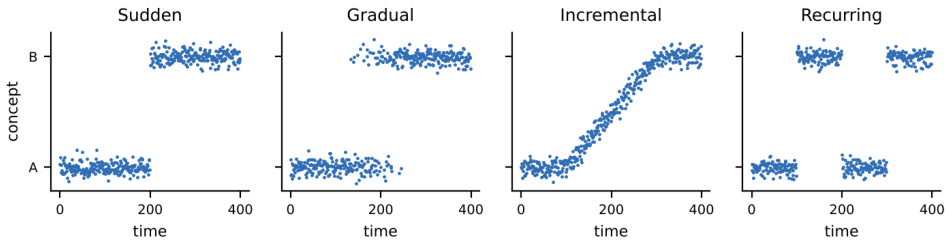


Figure 3.15 Temporal patterns of concept drift. Each panel plots the data distribution's mean over time as it moves from concept A to concept B.

Drift is handled in two complementary ways. **Passive** approaches adapt continuously: sliding windows that train only on recent data, exponential forgetting that down-weights old examples, and ensembles whose members are periodically replaced. **Active** approaches monitor the stream with a **drift detector** and trigger adaptation (resetting, retraining or growing a new sub-model) when change is detected. Well-known detectors include DDM (Gama et al., 2004), which watches the online error rate; ADWIN (Bifet and Gavaldà, 2007), which maintains an adaptive window and drops older data when two sub-windows differ significantly; and the Page–Hinkley test, a sequential change-point test. In production MLOps, drift is also monitored on input features with statistics such as PSI (Section 5) and the Kolmogorov–Smirnov test. Table 3.17 lists representative stream learners.

Table 3.17 Representative online and incremental learning algorithms.

Algorithm	Model type	Key idea
Perceptron / Passive–Aggressive	Linear classifier	Update only on mistakes or margin violations
SGD (online)	Linear and neural models	One gradient step per example or mini-batch
Hoeffding tree (VFDT)	Decision tree	Split once Hoeffding bound shows the best attribute
Adaptive random forest	Tree ensemble	Per-tree drift detectors replace stale trees
Online naive Bayes	Probabilistic classifier	Update count statistics incrementally
Mini-batch k-means	Clustering	Update centroids from small batches
Incremental PCA	Dimensionality reduction	Update the decomposition batch by batch

3.11.4 Incremental Learning in Practice

In scikit-learn, estimators that support incremental learning expose a `partial_fit` method, including `SGDClassifier`, `SGDRegressor`, `Perceptron`, `PassiveAggressiveClassifier`, the naive Bayes classifiers, `MiniBatchKMeans` and `IncrementalPCA`. The Python library **River** (Montiel et al., 2021) is dedicated to stream learning and includes Hoeffding trees, adaptive random forests and drift detectors. Listing 3.10 trains a classifier on a stream with sudden drift, evaluates it prequentially and uses ADWIN from River to detect the change.

Listing 3.10 Incremental learning with partial_fit, prequential evaluation and ADWIN drift detection.

```
import numpy as np
from sklearn.linear_model import SGDClassifier
from river.drift import ADWIN

rng = np.random.default_rng(0)
def stream(n=20000, drift_at=10000):
    for t in range(n):
        x = rng.normal(size=2)
        w = np.array([1.0, 1.0]) if t < drift_at else np.array([1.0, -1.0])   # real
        ↪ drift
        yield x, int(x @ w > 0)

clf = SGDClassifier(loss="log_loss", learning_rate="constant", eta0=0.05)
detector, correct, batch_X, batch_y = ADWIN(), [], [], []
for t, (x, y) in enumerate(stream()):
    if t > 0:                                          # test-then-train
        hit = int(clf.predict(x[None])[0] == y)
        correct.append(hit)
        detector.update(1 - hit)                       # monitor error stream
        if detector.drift_detected:
            print(f"drift detected at t={t}")
    batch_X.append(x); batch_y.append(y)
    if len(batch_X) == 16 or t == 0:                   # small mini-batches
        clf.partial_fit(np.array(batch_X), batch_y, classes=[0, 1])
        batch_X, batch_y = [], []
print("prequential accuracy:", np.mean(correct))
```

Practice Tip | Choosing Between Retraining and Online Updates

- If drift is slow and labels arrive late, scheduled batch retraining (for example weekly) on a sliding window is simpler and easier to validate.
- If the environment changes within hours and labels arrive quickly (clicks, fraud chargebacks), online updates pay off.
- Always keep a monitoring and rollback path: online learners can be corrupted by bad data or deliberate poisoning just as quickly as they adapt.

3.12 Continual and Lifelong Learning

Humans learn new skills throughout life without forgetting old ones: learning to drive does not erase the ability to ride a bicycle. Neural networks trained by gradient descent behave very differently. When a network trained on task A is subsequently trained on task B, its performance on A typically collapses. **Continual learning** (also called lifelong or incremental learning) studies how to learn from a sequence of tasks or a non-stationary stream while retaining previously acquired knowledge. Whereas the online learning of Section 11 emphasises fast adaptation to the *current* distribution, continual learning emphasises *accumulation*: performing well on all tasks seen so far.

3.12.1 Catastrophic Forgetting and the Stability–Plasticity Dilemma

The collapse of old-task performance is called **catastrophic forgetting** (McCloskey and Cohen, 1989). Its cause is that all tasks share the same parameters, and gradients for the new task move parameters that were important for the old one without any signal that they should be preserved. The underlying tension is the **stability–plasticity dilemma**: a learner must be plastic enough to acquire new knowledge but stable enough not to overwrite old knowledge. Figure 3.16 shows the typical behaviour of naive sequential fine-tuning compared with a continual learning method.

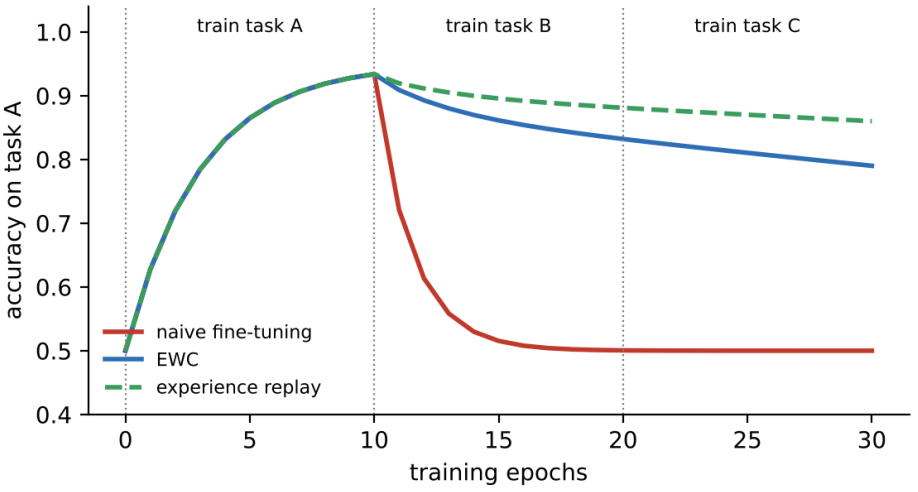


Figure 3.16 Accuracy on task A while a network is trained sequentially on tasks A, B and C. Naive fine-tuning drops to chance level on task A, whereas EWC and experience replay retain much of it at a small cost in plasticity.

Continual learning is studied in three standard scenarios (van de Ven and Tolias, 2019).

In **task-incremental** learning the task identity is given at test time, so a separate output head per task can be used. In **domain-incremental** learning the label set is fixed but the input distribution changes (for example, the same digits under new rotations). In **class-incremental** learning new classes arrive over time and the model must discriminate among all classes seen so far without being told the task; this is the hardest and most realistic scenario.

3.12.2 Regularisation-Based Methods: Elastic Weight Consolidation

Regularisation methods add a penalty that discourages changes to parameters important for earlier tasks. The best-known is **elastic weight consolidation** (EWC; Kirkpatrick et al., 2017, *PNAS*). Taking a Bayesian view, the posterior after task A, $p(\theta \mid D_A)$, becomes the prior for task B. EWC approximates this posterior with a Gaussian centred on the task-A solution θ_A, *whose precision is given by the diagonal of the Fisher information matrix** F. The resulting loss for task B is Equation (3.35).

$$\mathcal{L}(\theta) = \mathcal{L}_B(\theta) + \sum_i \frac{\lambda}{2} F_i \left(\theta_i - \theta^*_{A,i}\right)^2 \qquad F_i = \mathbb{E}_{\mathbf{x} \sim D_A,\ y \sim p_\theta(y \mid \mathbf{x})} \left[\left(\frac{\partial \log p_\theta(y \mid \mathbf{x})}{\partial \theta_i} \right)^2 \right] \tag{3.35}$$

where $\mathcal{L}_B$ is the loss on the new task, θ^*_A the parameters after training on task A, F_i the diagonal Fisher information of parameter i (its importance for task A) and λ the strength of consolidation.

Parameters with large Fisher information, those to which task A's predictions are sensitive, are held close to their old values like stiff springs, while unimportant parameters remain free to learn task B. For several tasks, one penalty is added per previous task, or a running Fisher estimate is kept (online EWC). Related methods include **synaptic intelligence** (Zenke et al., 2017), which accumulates importance along the training trajectory, and **learning without forgetting** (Li and Hoiem, 2017), which uses knowledge distillation from the old model's outputs instead of parameter penalties.

3.12.3 Replay and Parameter-Isolation Methods

Replay (rehearsal) methods store a small **episodic memory** of examples from earlier tasks and mix them into training on new tasks, so that gradients continue to reflect old objectives. Despite its simplicity, experience replay with a modest buffer is one of the strongest baselines, especially for class-incremental learning. Variants differ in how the buffer is filled (reservoir sampling keeps a uniform sample of the whole stream; herding selects examples near class means, as in iCaRL by Rebuffi et al., 2017) and how it is

used (GEM by Lopez-Paz and Ranzato, 2017, projects gradients so that loss on memory examples does not increase). Where storing raw data is prohibited, **generative replay** trains a generative model to produce pseudo-examples of old tasks. Algorithm 3.8 gives a generic replay procedure with reservoir sampling.

Algorithm 3.8 Experience Replay with a Reservoir Buffer

Input: Sequence of tasks with data streams $D_1, \ldots, D_T$; model f_θ; buffer capacity M; replay batch size b
Output: Model θ trained on all tasks

buffer $\leftarrow$ empty; seen $\leftarrow 0$
for task $\tau = 1$ **to** T **do**
  for each mini-batch (X, Y) from D_τ **do**
    if buffer is non-empty **then**
      $(X_m, Y_m) \leftarrow$ random sample of b items from buffer
      $(X, Y) \leftarrow (X \cup X_m, Y \cup Y_m)$
    $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}(f_\theta(X), Y)$
    for each new example (x, y) **in** the original mini-batch **do** // reservoir sampling
      seen $\leftarrow$ seen $+ 1$
      if $|\text{buffer}| < M$ **then** add (x, y) **to** buffer
      else
        $j \leftarrow$ uniform random integer **in** $[1, \text{seen}]$
        if $j \leq M$ **then** replace buffer$[j]$ with (x, y)
return θ

Parameter-isolation (architectural) methods allocate different parameters to different tasks so that learning one cannot interfere with another. **Progressive neural networks** (Rusu et al., 2016) add a new column of layers for each task with lateral connections to frozen earlier columns; **PackNet** (Mallya and Lazebnik, 2018) prunes the network after each task and frees the pruned weights for the next; and **adapter**- or prompt-based approaches freeze a large pre-trained backbone and learn small task-specific modules, a strategy that has become dominant with foundation models. These methods avoid forgetting by construction but grow with the number of tasks and usually need task identity at test time. Table 3.18 compares the three families.

Table 3.18 Families of continual learning methods.

Family	Representative methods	Advantages	Limitations
Regularisation	EWC, SI, MAS, LwF	No stored data; fixed model size	Weak in class-incremental settings
Replay	ER, iCaRL, GEM, DER, generative replay	Strong, simple, scenario-agnostic	Memory and privacy cost
Parameter isolation	Progressive nets, PackNet, adapters, prompts	No interference by design	Growing size; needs task ID

Continual learners are evaluated with an accuracy matrix R in which $R_{\tau,j}$ is the accuracy on task j after training on task τ. Common summaries are average accuracy after the final task, $(1/T)\ \Sigma_j\ R_{T,j}$, and **backward transfer**, $(1/(T-1))\ \Sigma_j\ (R_{T,j} - R_{j,j})$, which is negative when forgetting occurs; forward transfer measures how earlier tasks help later ones.

Listing 3.11 Computing a diagonal Fisher estimate and the EWC penalty in PyTorch.

```python
import torch
import torch.nn.functional as F

def fisher_diagonal(model, loader, n_batches=50):
    """Empirical diagonal Fisher information on the old task's data."""
    fisher = {n: torch.zeros_like(p) for n, p in model.named_parameters() if
    ↪  p.requires_grad}
    model.eval()
    for i, (x, y) in enumerate(loader):
        if i == n_batches:
            break
        model.zero_grad()
        F.nll_loss(F.log_softmax(model(x), dim=1), y).backward()
        for n, p in model.named_parameters():
            if p.grad is not None:
                fisher[n] += p.grad.detach() ** 2 / n_batches
    return fisher

def ewc_penalty(model, fisher, old_params, lam=1000.0):
    loss = 0.0
    for n, p in model.named_parameters():
        if n in fisher:
            loss = loss + (fisher[n] * (p - old_params[n]) ** 2).sum()
    return 0.5 * lam * loss

# After task A:
# fisher = fisher_diagonal(model, loader_A)
# old = {n: p.detach().clone() for n, p in model.named_parameters()}
# While training task B:
# loss = F.cross_entropy(model(x_b), y_b) + ewc_penalty(model, fisher, old)
```

Listing 3.11 shows how EWC is implemented in practice: after finishing task A, the squared gradients of the log-likelihood are averaged to estimate the diagonal Fisher information, and the quadratic penalty of Equation (3.35) is then added to the task-B loss.

Key Points | Continual Learning

- Sequential training on new tasks overwrites shared parameters, causing catastrophic forgetting.
- Regularisation (EWC) protects important weights; replay rehearses old data; parameter isolation assigns separate capacity.

- Class-incremental learning without task labels is the hardest scenario; replay-based methods are usually strongest there.
- Large pre-trained models with parameter-efficient adapters have made continual adaptation more practical, but forgetting of pre-trained knowledge remains an active research topic.

3.13 Few-Shot and Zero-Shot Learning

Deep networks usually need thousands of labelled examples per class, yet a child can recognise a new animal from a single picture. Many practical problems resemble the child's situation: rare diseases, new product categories, low-resource languages and personalised recognition all offer only a handful of labelled examples. **Few-shot learning** aims to recognise new classes from very few examples, and **zero-shot learning** from none at all, using side information such as attributes or text descriptions. Both rely on transferring knowledge learned from abundant data on *other* classes.

3.13.1 The N-Way K-Shot Problem

Few-shot classification is usually framed as a set of **episodes**. Each episode is an **N-way K-shot** task: N classes are sampled, and K labelled examples per class form the **support set** S ($N \times K$ examples in total); further examples of the same N classes form the **query set** Q, which the model must classify using only S. Typical benchmarks use 5-way 1-shot and 5-way 5-shot tasks on datasets such as Omniglot (1,623 handwritten characters), miniImageNet (100 ImageNet classes) and Meta-Dataset. Crucially, the classes used to construct test episodes are disjoint from those used in training, so the model must learn *how to learn new classes* rather than the classes themselves. Training with episodes that mimic the test condition is called **episodic training** (Vinyals et al., 2016).

D Definition 3.3 | Support Set, Query Set and Episode

An **episode** is a small classification task consisting of a **support set** $S = \{(x_i, y_i)\}$ containing K labelled examples for each of N classes and a **query set** Q of unlabelled examples from the same N classes. The learner must predict labels for Q using only S and whatever prior knowledge it acquired from earlier episodes on other classes.

3.13.2 Metric-Based Approaches: Siamese and Prototypical Networks

Metric-based methods learn an embedding space in which simple distance comparisons classify well. **Siamese networks** (Bromley et al., 1993; Koch et al., 2015 for one-shot image recognition) pass two inputs through the *same* network with shared weights and predict whether they belong to the same class from the distance between their embeddings. They are trained on pairs with a **contrastive loss**, Equation (3.36), or on triplets (anchor, positive, negative) with a triplet loss, as popularised by FaceNet (Schroff et al., 2015) for face verification. At test time a query is assigned the class of the most similar support example.

$$\mathcal{L} = y\, d^2 + (1-y)\max(0,\; m - d)^2, \qquad d = \lVert f_\phi(\mathbf{x}_1) - f_\phi(\mathbf{x}_2) \rVert_2 \tag{3.36}$$

where $y = 1$ for a same-class pair and 0 otherwise, f_φ is the shared embedding network and $m > 0$ is a margin beyond which dissimilar pairs incur no loss.

Prototypical networks (Snell, Swersky and Zemel, 2017, NeurIPS) are simpler and often stronger. Each class is represented by a **prototype**, the mean of its support embeddings, and a query is classified by a softmax over negative squared Euclidean distances to the prototypes, as in Equation (3.37). The whole network is trained end to end by minimising the negative log-probability of the true class of query examples across many episodes. Figure 3.17 illustrates the embedding space for a 3-way 5-shot episode.

$$\begin{aligned} \mathbf{c}_k &= \frac{1}{|S_k|} \sum_{(\mathbf{x}_i,\, y_i) \in S_k} f_\phi(\mathbf{x}_i) \\ p_\phi(y = k \mid \mathbf{x}) &= \frac{\exp\left(-\lVert f_\phi(\mathbf{x}) - \mathbf{c}_k \rVert^2\right)}{\sum_{k'} \exp\left(-\lVert f_\phi(\mathbf{x}) - \mathbf{c}_{k'} \rVert^2\right)} \end{aligned} \tag{3.37}$$

where S_k is the set of support examples of class k and $\mathbf{c}_k$ is its prototype in embedding space.

Other metric learners include **matching networks** (Vinyals et al., 2016), which use attention over the support set, and **relation networks** (Sung et al., 2018), which learn the similarity function itself. A notable finding of the late 2010s (for example, Chen et al., 2019; Tian et al., 2020) was that a simple baseline, pre-training a good feature extractor on all base classes with ordinary cross-entropy and then fitting a linear or nearest-centroid classifier on the support set, matches or beats many elaborate few-shot algorithms. Representation quality matters more than the episodic machinery. Listing 3.12 shows the core of a prototypical-network episode in PyTorch.

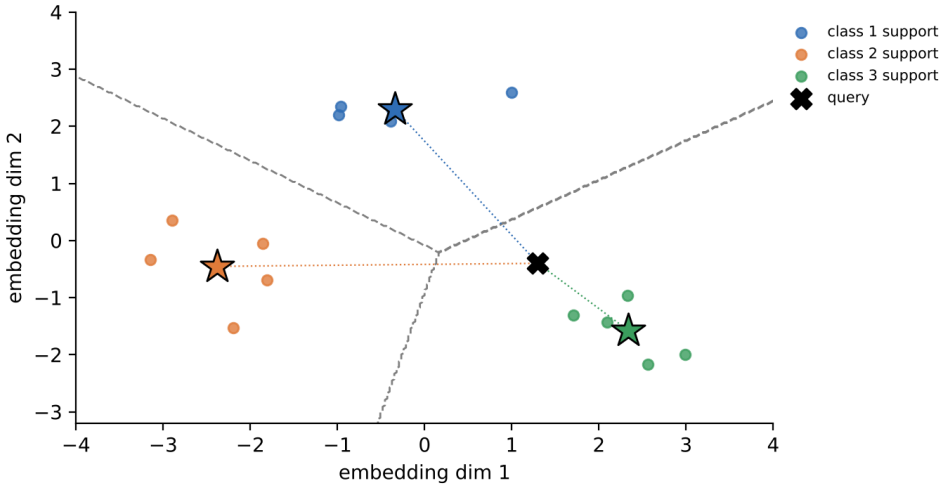


Figure 3.17 Prototypical network in a learned two-dimensional embedding space (3-way 5-shot). Support embeddings (dots) are averaged into class prototypes (stars); a query (black cross) is assigned to the nearest prototype, and the dashed lines show the resulting decision boundaries.

Listing 3.12 Loss for one prototypical-network episode in PyTorch.

```python
import torch
import torch.nn.functional as F

def prototypical_loss(encoder, support_x, support_y, query_x, query_y, n_way):
    """support_y and query_y contain labels 0..n_way-1 for this episode."""
    z_s = encoder(support_x)                       # (N*K, D)
    z_q = encoder(query_x)                         # (Q, D)
    prototypes = torch.stack([z_s[support_y == k].mean(0) for k in range(n_way)])  # (N, D)
    dists = torch.cdist(z_q, prototypes) ** 2      # squared Euclidean, (Q, N)
    log_p = F.log_softmax(-dists, dim=1)
    loss = F.nll_loss(log_p, query_y)
    acc = (log_p.argmax(1) == query_y).float().mean()
    return loss, acc

# Training loop over episodes sampled from base classes:
# for episode in sampler:
#     loss, acc = prototypical_loss(encoder, *episode, n_way=5)
#     optimiser.zero_grad(); loss.backward(); optimiser.step()
```

3.13.3 Zero-Shot Learning

Zero-shot learning recognises classes for which no training images exist, by relating them to seen classes through a shared **semantic space**. Early work (Lampert, Nickisch and Harmeling, 2009) used human-defined **attributes**: if a model has learned to detect ‘striped’, ‘four-legged’ and ‘horse-shaped’ from seen animals, it can recognise a ze-

bra from the description ‘striped, four-legged, horse-shaped’ without ever seeing one. Later methods replaced attributes with word embeddings of class names and learned a compatibility function between image features and class embeddings.

The breakthrough for practical zero-shot recognition came from large-scale **vision–language pre-training**. **CLIP** (Radford et al., 2021, ‘Learning Transferable Visual Models From Natural Language Supervision’, ICML) trains an image encoder and a text encoder jointly on about 400 million image–caption pairs with a contrastive objective that pulls matching pairs together and pushes mismatched pairs apart. To classify an image zero-shot, each candidate class name is inserted into a prompt such as ‘a photo of a {class}’, the prompts are embedded, and the image is assigned to the class whose text embedding has the highest cosine similarity, Equation (3.38). Large language models perform the analogous feat in text: given only a natural-language instruction, they classify sentiment, extract entities or translate without task-specific training examples (zero-shot prompting) or with a few examples in the prompt (few-shot or in-context learning; Brown et al., 2020).

$$\hat{y} = \arg\max_{k} \frac{\mathbf{v}^{\top}\mathbf{t}_k}{\|\mathbf{v}\|\,\|\mathbf{t}_k\|}, \qquad \mathbf{v} = f_{\text{img}}(\mathbf{x}), \qquad \mathbf{t}_k = f_{\text{text}}(\text{prompt}_k) \tag{3.38}$$

where f_{img} and f_{text} are the image and text encoders and prompt_k describes class k.

Table 3.19 Approaches to learning from few or no labelled examples.

Approach	Mechanism	Examples	Needs
Metric learning	Compare query with support in an embedding	Siamese, prototypical, matching nets	Episodic or pair training
Optimisation-based meta-learning	Learn an initialisation that adapts fast	MAML, Reptile	Second-order or approximate gradients
Transfer + fine-tuning	Pre-train, then fit a small head	Linear probe, adapters	Good pre-trained backbone
Data augmentation / generation	Synthesise extra examples	Hallucination, diffusion augmentation	Generative model
Attribute-based zero-shot	Map images to attribute space	DAP (Lampert et al.)	Attribute annotations
Vision–language zero-shot	Match image and text embeddings	CLIP, ALIGN	Large paired pre-training corpus
In-context learning	Condition an LLM on instructions or examples	GPT-style prompting	Large language model

Table 3.19 summarises the approaches. A realistic variant, **generalised zero-shot learning**, tests on a mixture of seen and unseen classes; models then tend to be biased towards seen classes, and calibration of scores between the two groups becomes important.

3.14 Meta-Learning

Meta-learning, or *learning to learn*, trains a system on a distribution of tasks so that it can learn new tasks from that distribution quickly. Ordinary learning finds parameters that solve one task; meta-learning finds something more general (an initialisation, an optimiser, an embedding or a learning rule) that makes *future* learning efficient. Few-shot classification (Section 13) is the most common test bed, but meta-learning is also used for fast adaptation in reinforcement learning, hyperparameter optimisation, and personalisation across users.

3.14.1 Problem Formulation and Taxonomy

Meta-learning assumes a distribution over tasks $p(\mathcal{T})$. Each task $\mathcal{T}_i$ comes with a small training (support) set D_i^tr and a test (query) set D_i^ts. Learning happens at two levels: an **inner loop** adapts to a single task using its support set, and an **outer loop** updates the shared meta-parameters so that inner-loop adaptation yields low loss on the query sets across many tasks. Hospedales et al. (2021, *IEEE TPAMI*) and others classify methods into three families, summarised in Table 3.20.

Table 3.20 Taxonomy of meta-learning methods.

Family	What is meta-learned	Representative methods	Inner-loop adaptation
Metric-based	An embedding and similarity	Siamese, matching, prototypical, relation nets	Non-parametric comparison
Model-based (black-box)	A network that ingests the support set	MANN, SNAIL, in-context learning in LLMs	Forward pass / hidden state
Optimisation-based	Initialisation, learning rates or optimiser	MAML, FOMAML, Reptile, Meta-SGD	Gradient descent steps

3.14.2 Model-Agnostic Meta-Learning (MAML)

MAML (Finn, Abbeel and Levine, 2017, ICML) seeks an initialisation θ from which a few gradient steps on any new task produce good performance. It is 'model-agnostic' because it applies to any model trained by gradient descent. For each sampled task, the inner loop takes one (or a few) gradient steps from θ on the support set, Equation (3.39). The outer loop then evaluates the *adapted* parameters on the query set and updates θ by differentiating through the inner step, Equation (3.40).

$$\theta_i' = \theta - \alpha \nabla_\theta \mathcal{L}_{\mathcal{T}_i}^{\text{tr}}(\theta) \tag{3.39}$$

where α is the inner (task-level) learning rate and $\mathcal{L}$^tr the loss on the support set of task $\mathcal{T}_i$.

$$\theta \leftarrow \theta - \beta \nabla_\theta \sum_{\mathcal{T}_i \sim p(\mathcal{T})} \mathcal{L}^{\text{ts}}_{\mathcal{T}_i}\big(\theta - \alpha \nabla_\theta \mathcal{L}^{\text{tr}}_{\mathcal{T}_i}(\theta)\big)$$
$$\nabla_\theta \mathcal{L}^{\text{ts}}(\theta_i') = \big(I - \alpha \nabla^2_\theta \mathcal{L}^{\text{tr}}(\theta)\big) \nabla_{\theta'} \mathcal{L}^{\text{ts}}(\theta_i') \tag{3.40}$$

where β is the meta learning rate and $\mathcal{L}$^ts the loss on the query set; the second line shows the Hessian term produced by differentiating through the inner update.

The Hessian in the meta-gradient is expensive. **First-order MAML** (FOMAML) drops it, using ∇_θ′$\mathcal{L}$^ts(θ′$_i$) directly, and performs almost as well in practice. **Reptile** (Nichol, Achiam and Schulman, 2018) is simpler still: run several SGD steps on a task to obtain θ′$_i$ and move θ towards it, $\theta \leftarrow \theta + \varepsilon(\theta'_i - \theta)$. Algorithm 3.9 gives the full MAML procedure and Figure 3.18 shows the geometric intuition: θ is placed where small task-specific steps reach each task's optimum.

Algorithm 3.9 Model-Agnostic Meta-Learning (MAML) for Few-Shot Supervised Learning

Input: Task distribution p($\mathcal{T}$); model f_θ; inner learning rate α; meta learning rate β; meta-batch size B; inner steps s

Output: Meta-learned initialisation θ

randomly **initialise** θ
while not converged **do**
  sample a batch of tasks $\mathcal{T}_1$, ..., $\mathcal{T}$_B ~ p($\mathcal{T}$)
  for each task $\mathcal{T}_i$ **do**
    sample support set D_i^{tr} **and** query set D_i^{ts} from $\mathcal{T}_i$
    θ′$_i$ ← θ
    repeat s times: θ′$_i$ ← θ′$_i$ − α ∇ $\mathcal{L}$(f_θ′$_i$; D_i^{tr}) // inner loop
    compute query loss $\mathcal{L}_i$ ← $\mathcal{L}$(f_θ′$_i$; D_i^{ts})
  θ ← θ − β ∇_θ Σ$_i$ $\mathcal{L}_i$ // outer loop, backprop through inner steps
return θ

// Meta-test: **for** a new task, run the inner loop from θ on its support set **and** predict on its queries

E Worked Example 3.9 | One MAML Step on Quadratic Tasks

Let each task i have loss $\mathcal{L}_i(\theta) = (\theta - a_i)^2$ for a scalar parameter, with two tasks $a_1 = 1$ and $a_2 = 3$, α = 0.25 and β = 0.1, starting from θ = 0. Use the same loss for support and query to keep the arithmetic simple.

Inner step: $\nabla\mathcal{L}_i = 2(\theta - a_i)$, so $\theta'_1 = 0 - 0.25 \times 2(0 - 1) = 0.5$ and $\theta'_2 = 0 - 0.25 \times 2(0 - 3) = 1.5$.

Meta-gradient: $\partial\mathcal{L}_i(\theta'_i)/\partial\theta = 2(\theta'_i - a_i) \times (1 - 2\alpha) = 2(\theta'_i - a_i) \times 0.5$. For task 1: $2(0.5 - 1) \times 0.5 = -0.5$; for task 2: $2(1.5 - 3) \times 0.5 = -1.5$. Sum = −2.0.

Outer update: $\theta \leftarrow 0 - 0.1 \times (-2.0) = 0.2$. Repeating moves θ towards 2, the point from which

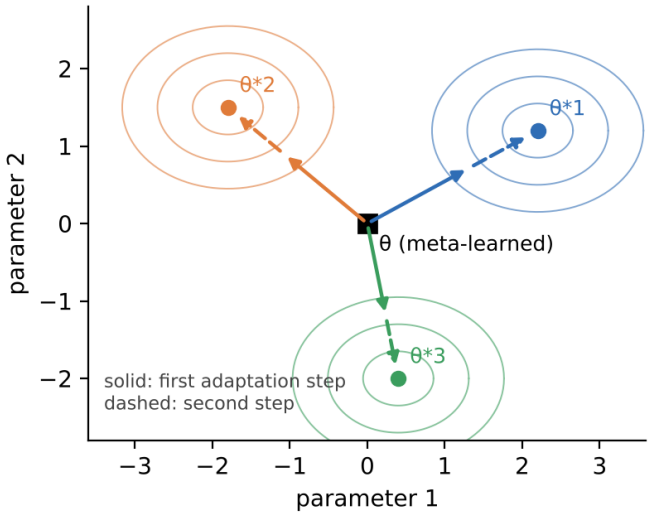


Figure 3.18 Geometric intuition for MAML. The meta-learned initialisation θ sits where one or two gradient steps (arrows) reach the optimum θ^*_i of each task's loss, unlike an initialisation tuned to a single task.

> one step reaches both tasks' optima equally well. The factor $(1 - 2\alpha)$ is the second-order term; FOMAML would omit it.

3.14.3 Strengths, Limitations and Connections

Meta-learning provides a principled route to sample-efficient adaptation and has been applied to few-shot image and text classification, fast motor-skill adaptation in robotics, personalised models on devices, drug-discovery tasks with few labelled molecules, and learned optimisers. Its limitations are also real. Optimisation-based methods are expensive and sometimes unstable (MAML++ by Antoniou et al., 2019, proposed several fixes); performance depends on how well meta-training tasks resemble meta-test tasks; and, as noted in Section 13, strong pre-trained representations often close much of the gap. Large language models blur the boundaries further: their in-context learning can be viewed as model-based meta-learning that emerged from pre-training on a vast and varied distribution of text, with the prompt playing the role of the support set.

3.15 Retrieval-Augmented Generation (RAG)

Large language models (LLMs; Chapter 2) store a remarkable amount of knowledge in their parameters, but that knowledge is frozen at training time, cannot easily be updated or attributed to sources, excludes private documents, and is prone to **hallucination**: fluent statements that are false. **Retrieval-augmented generation** (RAG) addresses these problems by retrieving relevant passages from an external corpus at query time and supplying them to the model as context. The term was introduced by Lewis et al. (2020), 'Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks' (NeurIPS), which combined a dense retriever with a sequence-to-sequence generator; today it most often denotes a pipeline that places retrieved text into the prompt of a general-purpose LLM.

3.15.1 The RAG Pipeline

A RAG system has an offline **indexing** stage and an online **retrieval-and-generation** stage, shown in Figure 3.19. During indexing, documents are loaded, cleaned, split into chunks, converted to vector **embeddings** by an embedding model, and stored in a vector index together with their text and metadata. At query time, the user's question is embedded with the same model, the most similar chunks are retrieved (optionally re-ranked), and a prompt is assembled containing instructions, the retrieved context and the question. The LLM generates an answer grounded in that context, ideally citing the chunks it used.

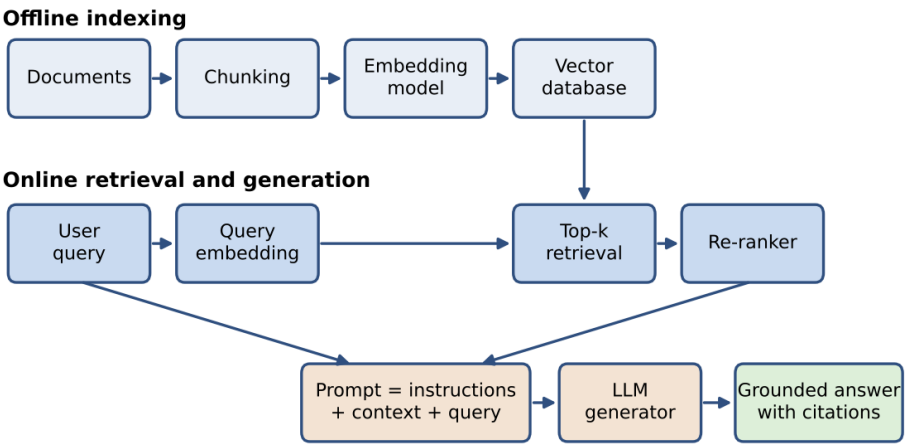


Figure 3.19 Retrieval-augmented generation. Offline, documents are chunked, embedded and indexed in a vector database; online, the query is embedded, similar chunks are retrieved and re-ranked, and the LLM generates an answer grounded in the retrieved context.

3.15.2 Chunking, Embeddings and Similarity

Chunking determines the unit of retrieval. Chunks that are too large dilute relevance and waste the model's context window; chunks that are too small lose the context needed to interpret them. Common strategies are fixed-size windows of a few hundred tokens with an overlap of 10–20%, recursive splitting along document structure (sections, paragraphs, sentences), and semantic chunking that starts a new chunk where the topic shifts. Attaching metadata (title, section heading, date, access rights) enables filtering and better citations.

Embedding models map text to dense vectors such that semantically similar texts are close together. They are typically transformer encoders trained contrastively on large collections of (query, relevant passage) pairs, in the spirit of dense passage retrieval (DPR; Karpukhin et al., 2020) and Sentence-BERT (Reimers and Gurevych, 2019). Similarity is usually measured by **cosine similarity**, Equation (3.41), which equals the dot product for unit-normalised vectors.

$$\operatorname{sim}(\mathbf{q}, \mathbf{d}) = \cos\theta = \frac{\mathbf{q}^\top \mathbf{d}}{\|\mathbf{q}\|_2 \, \|\mathbf{d}\|_2} \tag{3.41}$$

where q and d are the embeddings of the query and a document chunk; values range from -1 to 1.

Dense retrieval excels at paraphrase and semantic matching but can miss exact identifiers, rare names and codes, which classic **sparse** lexical retrieval such as BM25 (Robertson and Zaragoza, 2009) handles well. **Hybrid retrieval** combines both, for example by reciprocal rank fusion, Equation (3.42). A **cross-encoder re-ranker**, which reads the query and each candidate passage together, then re-scores the top few dozen candidates more accurately than the bi-encoder used for first-stage retrieval.

$$\mathrm{RRF}(d) = \sum_{r \in R} \frac{1}{k + \mathrm{rank}_r(d)} \tag{3.42}$$

where R is the set of rankers (e.g. BM25 and dense), rank_r(d) the position of document d in ranker r's list, and k a constant (commonly 60).

3.15.3 Vector Databases

Finding the exact nearest neighbours of a query among millions of vectors is too slow, so vector databases use **approximate nearest neighbour** (ANN) indexes that trade a little recall for orders of magnitude in speed. The most popular is **HNSW** (hierarchical navigable small-world graphs; Malkov and Yashunin, 2018), which searches a multi-layer proximity graph greedily from coarse to fine. **IVF** (inverted file) indexes cluster

vectors with k-means and search only the nearest clusters, and **product quantisation** compresses vectors into short codes to save memory. Libraries such as FAISS (Johnson, Douze and Jégou, 2019) implement these indexes; database systems such as Milvus, Weaviate, Qdrant, Pinecone, Chroma and the pgvector extension for PostgreSQL add persistence, metadata filtering, replication and access control. Table 3.21 compares index types.

Table 3.21 Approximate nearest-neighbour index types used in vector databases.

Index	Principle	Strengths	Trade-offs
Flat (brute force)	Compare with every vector	Exact; no training	Linear query time
IVF	Search only nearest k-means cells	Fast, simple	Recall depends on cells probed
HNSW	Greedy search on layered proximity graph	High recall at low latency	Memory-hungry; slow to build
Product quantisation (PQ)	Compress vectors into sub-codebooks	Small memory footprint	Lossy distances
IVF-PQ	IVF cells with PQ-compressed vectors	Billion-scale search	Tuning complexity

3.15.4 Evaluating and Improving RAG Systems

A RAG system can fail at retrieval (the right passage is not found), at generation (the passage is found but ignored or misread), or both, so each stage should be evaluated separately. Retrieval is measured with standard information-retrieval metrics such as recall@k, precision@k, mean reciprocal rank and nDCG on a set of questions with known relevant passages. Generation is assessed for **faithfulness** (groundedness: is every claim supported by the retrieved context?), **answer relevance** (does it address the question?) and **correctness** against reference answers. Because human evaluation is costly, 'LLM-as-a-judge' frameworks such as RAGAS automate these judgements, but they should be validated against human ratings on a sample. Table 3.22 lists the core metrics.

Common improvements include query rewriting (expanding or decomposing questions), hypothetical-document embeddings (HyDE), parent–child chunking (retrieve small chunks but return their larger parent section), knowledge-graph-enhanced retrieval (GraphRAG), and **agentic RAG**, in which the model decides when and what to retrieve over several steps (Section 16). Listing 3.13 implements a minimal RAG pipeline with Sentence-Transformers and FAISS; the final generation call can use any LLM API.

Table 3.22 Evaluation metrics for retrieval-augmented generation.

Stage	Metric	What it measures
Retrieval	Recall@k	Fraction of relevant passages found in the top k
Retrieval	MRR / nDCG	Whether relevant passages are ranked near the top
Retrieval	Context precision	Fraction of retrieved passages that are relevant
Generation	Faithfulness (groundedness)	Claims in the answer supported by retrieved context
Generation	Answer relevance	How directly the answer addresses the question
Generation	Correctness	Agreement with a reference answer
System	Latency and cost	Time and tokens per query

Listing 3.13 A minimal retrieval-augmented generation pipeline with Sentence-Transformers and FAISS.

```python
import faiss
import numpy as np
from sentence_transformers import SentenceTransformer

def chunk(text, size=120, overlap=20):
    words = text.split()
    return [" ".join(words[i:i + size]) for i in range(0, len(words), size - overlap)]

docs = {"policy.txt": open("policy.txt").read(), "faq.txt": open("faq.txt").read()}
chunks, sources = [], []
for name, text in docs.items():
    for c in chunk(text):
        chunks.append(c); sources.append(name)

embedder = SentenceTransformer("all-MiniLM-L6-v2")
E = embedder.encode(chunks, normalize_embeddings=True).astype("float32")
index = faiss.IndexFlatIP(E.shape[1])            # inner product = cosine on unit vectors
index.add(E)

def retrieve(query, k=4):
    q = embedder.encode([query], normalize_embeddings=True).astype("float32")
    scores, ids = index.search(q, k)
    return [(chunks[i], sources[i], float(s)) for i, s in zip(ids[0], scores[0])]

def build_prompt(query):
    ctx = "\n\n".join(f"[{j+1}] ({src}) {txt}" for j, (txt, src, _) in enumerate(retrieve(query)))
    return ("Answer using ONLY the context below. Cite passages as [n]. "
            "If the answer is not in the context, say you do not know.\n\n"
            f"Context:\n{ctx}\n\nQuestion: {query}\nAnswer:")

prompt = build_prompt("How many days of annual leave do new employees get?")
# answer = llm.generate(prompt)   # any chat/completions API
```

> **Note | RAG Reduces but Does Not Eliminate Hallucination**
>
> Retrieved context can itself be outdated, contradictory or malicious. Documents containing hidden instructions can hijack the model (**indirect prompt injection**). Production systems should filter sources, respect document-level access rights at retrieval time, ask the model to abstain when the context is insufficient, and monitor faithfulness continuously.

3.16 AI Agents and Tool-Using Models

An LLM on its own maps a prompt to text. An **AI agent** wraps the model in a loop that lets it pursue goals over multiple steps: it observes the state of its environment, reasons about what to do, takes **actions** through tools (search engines, code interpreters, databases, web browsers, other software APIs), observes the results and continues until the task is complete. The concept of an agent, an entity that perceives its environment and acts upon it, is foundational in AI (Russell and Norvig, *Artificial Intelligence: A Modern Approach*); what is new is that LLMs provide a general-purpose policy that can be instructed in natural language.

3.16.1 The Agent Loop and the ReAct Pattern

Figure 3.20 shows the basic agent loop. The LLM receives the goal, the conversation and tool results so far, and any retrieved memory; it decides on either a tool call or a final answer; the runtime executes the tool and appends its output to the context; and the loop repeats. The **ReAct** pattern (Yao et al., 2023, 'ReAct: Synergizing Reasoning and Acting in Language Models', ICLR) interleaves free-text **reasoning** traces ('Thought') with **actions** ('Action') and their results ('Observation'). Reasoning helps the model plan and track progress, while actions ground the reasoning in real information, reducing hallucination compared with reasoning alone. Algorithm 3.10 formalises the loop.

Algorithm 3.10 ReAct-Style Tool-Using Agent Loop

Input: User goal g; LLM policy π; tool set $\mathcal{T}$ with descriptions; step limit N; memory store M

Output: Final answer (or escalation to a human)

```
context ← [system instructions, tool descriptions, g, relevant items retrieved from M]
for step = 1 to N do
    output ← π(context) // thought + proposed action
    if output is a final answer then
        store useful facts in M; return answer
```

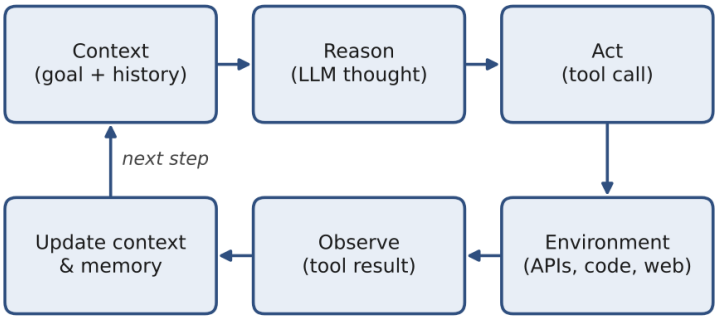


Figure 3.20 The agent loop. An LLM policy reasons over the goal, context and memory, selects a tool action, observes the result returned by the environment and iterates until it produces a final answer.

```
    (tool, arguments) ← parse(output)
    if tool ∉ 𝒯 or arguments fail validation then
        observation ← error message
    else if tool is high-risk then
        observation ← execute only after human approval
    else
        observation ← execute(tool, arguments) // sandboxed
    append (output, observation) to context
return 'step limit reached' and escalate to a human
```

3.16.2 Tool Calling, Planning and Memory

Tool calling (function calling) is the interface between the model and software. Each tool is described to the model by a name, a natural-language description and a JSON schema for its arguments; the model emits a structured call, the runtime validates and executes it, and the result is returned as a message. Toolformer (Schick et al., 2023) showed that models can learn when to call tools in a self-supervised way, and current LLMs are fine-tuned explicitly for reliable structured tool use. Emerging standards such as the Model Context Protocol (MCP) standardise how tools and data sources are exposed to models. Listing 3.14 shows a minimal tool-calling loop.

Listing 3.14 A minimal tool-calling agent loop (provider-agnostic sketch).

```python
import json

def get_weather(city: str) -> dict:
    return {"city": city, "forecast": "light rain", "max_c": 29}     # stub

def calculator(expression: str) -> dict:
    import ast, operator as op
    ops = {ast.Add: op.add, ast.Sub: op.sub, ast.Mult: op.mul, ast.Div: op.truediv}
    def ev(n):
        if isinstance(n, ast.Constant): return n.value
        if isinstance(n, ast.BinOp): return ops[type(n.op)](ev(n.left), ev(n.right))
        raise ValueError("unsupported expression")
    return {"result": ev(ast.parse(expression, mode="eval").body)}   # no eval()

TOOLS = {"get_weather": get_weather, "calculator": calculator}

def run_agent(llm, user_goal, max_steps=8):
    messages = [{"role": "user", "content": user_goal}]
    for _ in range(max_steps):
        reply = llm(messages, tools=TOOLS)          # returns text or a tool call
        if reply.get("tool_call") is None:
            return reply["content"]                 # final answer
        name, args = reply["tool_call"]["name"], reply["tool_call"]["arguments"]
        try:
            result = TOOLS[name](**json.loads(args))
        except Exception as e:                      # report errors back to the model
            result = {"error": str(e)}
        messages += [reply, {"role": "tool", "name": name, "content":
        ↪  json.dumps(result)}]
    return "Stopped: step limit reached."
```

Planning lets agents tackle long tasks. Simple agents plan implicitly, step by step; others first produce an explicit plan and then execute it (plan-and-execute), revise plans when observations contradict expectations, or search over alternative reasoning paths, as in Tree of Thoughts (Yao et al., 2023). **Reflection** techniques such as Reflexion (Shinn et al., 2023) have the agent critique failed attempts and store lessons for the next try. **Memory** is usually divided into *short-term* memory, the current context window, and *long-term* memory stored externally (often in a vector database, as in Section 15) and retrieved when relevant; long-term memory may be *episodic* (past interactions), *semantic* (facts) or *procedural* (learned skills and tool-use routines).

3.16.3 Multi-Agent Systems

Complex workflows are often decomposed across several agents with distinct roles, for example a planner, a researcher, a coder and a reviewer, coordinated by an orchestrator or communicating through a shared message board. Multi-agent designs can improve modularity and allow specialised prompts, tools and permissions for each

role; debate-style set-ups in which agents critique each other can improve factuality. They also multiply cost and introduce new failure modes, such as agents looping, miscommunicating or amplifying each other's errors, so a single well-equipped agent is often the better starting point. Frameworks such as LangGraph, AutoGen, CrewAI and vendor agent SDKs provide building blocks for both single- and multi-agent systems. Table 3.23 summarises the main design components.

Table 3.23 Components of LLM-based agent systems.

Component	Role	Design options
Policy (LLM)	Decides next thought or action	Model choice, system prompt, temperature
Tools	Act on the world	APIs, code execution, browser, retrieval, MCP servers
Planning	Decompose and sequence work	ReAct, plan-and-execute, tree search, reflection
Memory	Persist information across steps and sessions	Context window, vector store, summaries
Orchestration	Control flow and multi-agent coordination	Graphs/state machines, supervisor agents
Guardrails	Constrain and monitor behaviour	Permissions, sandboxing, approvals, output filters
Evaluation	Measure task success and safety	Benchmarks (SWE-bench, WebArena), trajectory review

3.16.4 Risks and Safeguards

Because agents take actions, their errors have consequences beyond a wrong sentence: deleting files, sending messages, spending money or leaking data. Key risks include **prompt injection** (malicious instructions hidden in web pages, emails or documents that the agent reads), **excessive agency** (tools with broader permissions than the task needs), **compounding errors** over long trajectories, **unreliable self-assessment** (agents may report success without achieving it), and cost or latency blow-ups from runaway loops. Sound engineering practice applies the principle of least privilege to tools, runs code in sandboxes, requires human confirmation for irreversible or high-impact actions, validates tool arguments, logs complete trajectories for audit, sets step and budget limits, and evaluates agents on realistic task suites before deployment.

> **K Key Points | AI Agents**
>
> - An agent is an LLM in a loop of reasoning, tool use and observation.

- ReAct interleaves reasoning with actions so that conclusions are grounded in observations.
- Tools are described by schemas; the runtime, not the model, executes them and must validate inputs.
- Treat all tool outputs and retrieved content as untrusted; require approval for consequential actions.

3.17 Responsible AI and AI Governance

As AI systems make or shape decisions about credit, employment, medical care, policing and access to information, their failures become social as well as technical problems. Documented cases include recidivism-risk scores with unequal error rates across racial groups, commercial face-analysis systems far less accurate for darker-skinned women (Buolamwini and Gebru, 2018), and a recruiting model that learned to penalise CVs mentioning women's organisations. **Responsible AI** is the practice of designing, building and deploying AI so that it is fair, safe, transparent and accountable; **AI governance** is the set of organisational processes, standards and laws that make those properties enforceable.

3.17.1 Principles of Responsible AI

Dozens of organisations have published AI principles, including the OECD AI Principles (2019), UNESCO's Recommendation on the Ethics of AI (2021) and the EU High-Level Expert Group's Ethics Guidelines for Trustworthy AI (2019). They converge on a common core, summarised in Table 3.24 together with the technical practices that put each principle into effect.

Fairness deserves particular care because its definitions conflict. **Demographic parity** requires equal positive-prediction rates across groups; **equalised odds** (Hardt, Price and Srebro, 2016) requires equal true-positive and false-positive rates; **calibration within groups** requires that a score of 0.7 means a 70% outcome rate in every group. Kleinberg, Mullainathan and Raghavan (2017) and Chouldechova (2017) proved that, when base rates differ between groups, calibration and equal error rates cannot generally hold simultaneously. Choosing a fairness criterion is therefore a value judgement that must be made explicitly with stakeholders, not a purely technical decision. Demographic parity, for example, is expressed in Equation (3.43).

$$\left| P(\hat{Y}=1 \mid A=a) - P(\hat{Y}=1 \mid A=b) \right| \leq \epsilon \tag{3.43}$$

Table 3.24 Core principles of responsible AI and corresponding technical practices.

Principle	Meaning	Technical practices
Fairness	No unjustified disparate treatment or impact	Group fairness metrics, bias audits, reweighting, constrained training
Transparency	Stakeholders know when and how AI is used	Model cards, datasheets, disclosure of AI use
Explainability	Decisions can be understood and contested	SHAP, LIME, counterfactual explanations, interpretable models
Privacy	Personal data are protected	Data minimisation, differential privacy, federated learning
Robustness and safety	Reliable under shift and attack	Stress tests, adversarial evaluation, monitoring, fallbacks
Accountability	Clear responsibility and redress	Audit trails, sign-offs, incident reporting, human oversight
Human agency	Humans remain in meaningful control	Human-in-the-loop review, override and appeal mechanisms
Sustainability	Environmental and social costs considered	Energy and carbon reporting (Section 19)

where A is a protected attribute (such as sex or ethnicity), a and b are two of its values, and ε is a tolerance.

3.17.2 Regulation and Standards: The EU AI Act and NIST AI RMF

The **EU Artificial Intelligence Act** (Regulation (EU) 2024/1689), which entered into force on 1 August 2024 with obligations phased in over the following years, is the first comprehensive horizontal AI law. It takes a **risk-based approach**, illustrated in Figure 3.21. *Unacceptable-risk* practices, such as social scoring by public authorities and manipulative techniques that exploit vulnerabilities, are prohibited. *High-risk* systems, including AI used in critical infrastructure, education, employment, credit scoring, law enforcement, migration and safety components of regulated products, must meet requirements for risk management, data governance, technical documentation, logging, transparency, human oversight, accuracy, robustness and cybersecurity, and undergo conformity assessment. *Limited-risk* systems such as chatbots and generators of synthetic content carry transparency obligations (users must know they are interacting with AI, and deepfakes must be labelled). *Minimal-risk* systems, such as spam filters, face no new obligations. Separate rules apply to providers of general-purpose AI models, with additional duties for models posing systemic risk.

In the United States, the **NIST AI Risk Management Framework** (AI RMF 1.0, January 2023) is a voluntary framework organised around four functions: **Govern** (establish

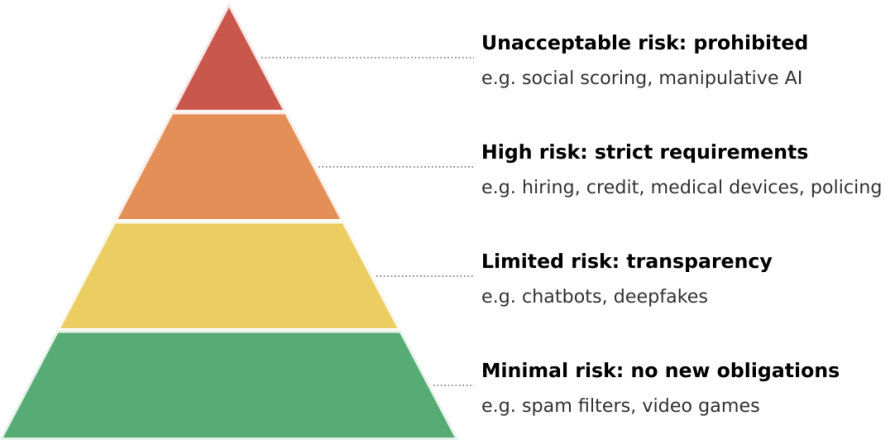


Figure 3.21 The four risk tiers of the EU AI Act. Obligations increase towards the top of the pyramid; the top tier is prohibited outright.

a risk-aware culture, policies and accountability), **Map** (understand the context, intended use and potential impacts), **Measure** (assess and track risks with quantitative and qualitative methods) and **Manage** (prioritise and treat risks, and respond to incidents). A Generative AI Profile (2024) extends it to risks specific to generative models. The international management-system standard **ISO/IEC 42001:2023** provides a certifiable framework for organisations' AI governance. Table 3.25 compares these instruments.

Table 3.25 Major AI governance frameworks.

Instrument	Type	Approach	Key elements
EU AI Act (2024)	Binding regulation	Risk tiers	Prohibitions, high-risk requirements, GPAI duties, fines
NIST AI RMF 1.0 (2023)	Voluntary framework	Lifecycle risk management	Govern, Map, Measure, Manage
ISO/IEC 42001 (2023)	Certifiable standard	Management system	Policies, roles, risk assessment, continual improvement
OECD AI Principles (2019)	Intergovernmental principles	Values-based	Inclusive growth, human rights, transparency, robustness

3.17.3 Documentation: Model Cards and Datasheets

Documentation is the most practical everyday tool of AI governance. **Model cards** (Mitchell et al., 2019, FAT) *are short documents accompanying a trained model that describe its intended uses and out-of-scope uses, training and evaluation data, performance metrics broken down by relevant groups and conditions, ethical considerations, and caveats. Datasheets for datasets** (Gebru et al., 2021, *Communications of the ACM*) play the same role for

data: motivation, composition, collection process, pre-processing and labelling, recommended uses, distribution and maintenance. Model and dataset hubs such as Hugging Face have adopted cards as standard, and the technical documentation required for high-risk systems under the EU AI Act covers much of the same ground.

Practice Tip | A Minimal Model Card Checklist

- **Model details:** owner, version, date, architecture, licence.
- **Intended use:** primary users and uses; explicitly out-of-scope uses.
- **Data:** training and evaluation sources, time span, known gaps.
- **Metrics:** overall and disaggregated by subgroup, with confidence intervals.
- **Limitations and risks:** failure modes, fairness findings, required human oversight.
- **Maintenance:** monitoring plan, retraining triggers, contact for issues.

3.18 Privacy-Preserving Machine Learning

Models trained on personal data can leak it. **Membership inference** attacks determine whether a given record was in the training set (Shokri et al., 2017); **model inversion** and gradient-inversion attacks reconstruct representative or actual inputs; and large language models have been shown to memorise and regurgitate verbatim training sequences, including personal information (Carlini et al., 2021). Anonymisation by removing names is insufficient: Sweeney showed that ZIP code, birth date and sex uniquely identify most Americans. **Privacy-preserving machine learning** (PPML) provides techniques with formal guarantees. This section covers the three main families: differential privacy, homomorphic encryption and secure multi-party computation.

3.18.1 Differential Privacy

Differential privacy (DP), introduced by Dwork, McSherry, Nissim and Smith (2006), is a mathematical guarantee about an algorithm, not a dataset. Intuitively, an algorithm is differentially private if its output distribution barely changes when any single individual's record is added or removed, so an observer cannot tell whether that person participated. Formally, a randomised mechanism M satisfies (ε, δ)-differential privacy if, for all pairs of **neighbouring datasets** D and D′ differing in one record and all sets of outputs S, the inequality in Equation (3.44) holds.

$$P[M(D) \in S] \leq e^{\epsilon}\, P[M(D') \in S] + \delta \tag{3.44}$$

where ε (the privacy budget) bounds the multiplicative change in probabilities and δ is a small probability of failure (δ = 0 gives pure ε-DP).

Smaller ε means stronger privacy; values below about 1 are considered strong, while values of 8–10 are common in deep learning practice and provide weaker but still meaningful protection. DP has three properties that make it practical: it is immune to **post-processing** (any computation on a DP output remains DP), it **composes** (running k mechanisms with budgets ε_i costs at most $\Sigma\varepsilon_i$, with tighter advanced-composition and Rényi-DP accounting available), and it holds regardless of the attacker's side information.

Definition 3.4 | Sensitivity

The **ℓ_1-sensitivity** of a numeric query f is $\Delta f = \max \|f(D) - f(D')\|_1$ over all neighbouring datasets D and D′: the most that one individual can change the answer. A counting query has $\Delta f = 1$; the sum of values clipped to $[0, B]$ has $\Delta f = B$. Noise must be calibrated to sensitivity.

The simplest DP algorithm, the **Laplace mechanism**, Equation (3.45), adds noise drawn from a Laplace distribution with scale proportional to the sensitivity and inversely proportional to ε. The **Gaussian mechanism** adds Gaussian noise with σ calibrated to the ℓ_2-sensitivity and gives (ε, δ)-DP. Figure 3.22 shows the output distributions of the Laplace mechanism for neighbouring datasets: their ratio is bounded by e^ε everywhere, which is exactly the DP condition.

$$M(D) = f(D) + \mathrm{Lap}\left(\frac{\Delta f}{\epsilon}\right), \quad \mathrm{Lap}(z \mid b) = \frac{1}{2b}\exp\left(-\frac{|z|}{b}\right) \tag{3.45}$$

where Δf is the sensitivity of f and $b = \Delta f/\varepsilon$ is the noise scale.

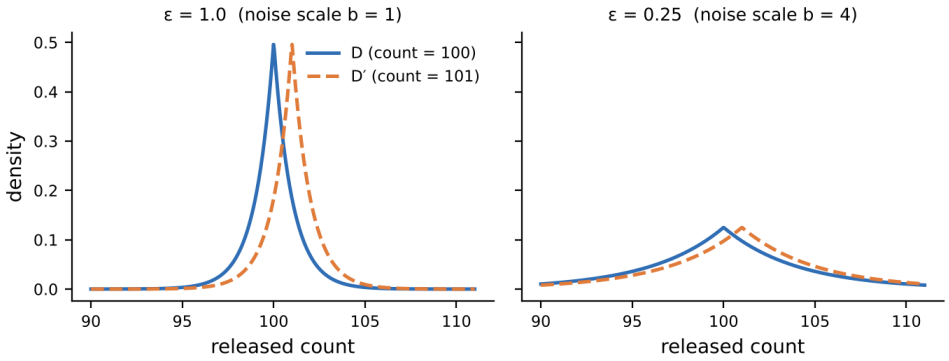


Figure 3.22 Laplace mechanism for a counting query (Δf = 1). Output densities for neighbouring datasets with true counts 100 and 101 overlap heavily; smaller ε means more noise and less distinguishable outputs.

E Worked Example 3.10 | A Private Count

A hospital wants to release the number of patients with a rare condition. The true count is 37 and the sensitivity of a count is $\Delta f = 1$.

With $\varepsilon = 0.5$ the noise scale is $b = 1/0.5 = 2$. The standard deviation of Laplace noise is $\sqrt{2} \cdot b \approx 2.83$, so a released value such as 39.4 is typically within about ± 3 of the truth.

With $\varepsilon = 0.1$, $b = 10$ and the standard deviation is about 14: much stronger privacy, but the count is now only roughly informative. This privacy–utility trade-off is central to every DP deployment.

3.18.2 Differentially Private Training: DP-SGD

To train neural networks with DP, Abadi et al. (2016), 'Deep Learning with Differential Privacy' (ACM CCS), proposed **DP-SGD**, given in Algorithm 3.11. Two modifications to SGD bound and then mask each individual's influence: per-example gradients are **clipped** to a maximum ℓ_2 norm C, which bounds the sensitivity, and Gaussian **noise** with standard deviation σC is added to the summed gradients. A privacy accountant (the moments accountant or Rényi-DP accountant) tracks the cumulative ε over all training steps, benefiting from *privacy amplification by subsampling* because each example appears in only a fraction of batches.

Algorithm 3.11 Differentially Private Stochastic Gradient Descent (DP-SGD)

Input: Dataset of N examples; loss ℓ; learning rate η; noise multiplier σ; clipping norm C; sampling rate $q = L/N$; steps T; target δ
Output: Model θ_T and privacy guarantee (ε, δ)

initialise θ_0
for $t = 1$ **to** T **do**
  form batch L_t by including **each** example independently with probability q // Poisson sampling
  for each example i **in** L_t **do**
    $g_i \leftarrow \nabla_\theta \, \ell(\theta_{t-1}; x_i)$ // per-example gradient
    $\bar{g}_i \leftarrow g_i \,/\, \max(1, \|g_i\|_2 \,/\, C)$ // clip **to** norm $\le C$
  $\tilde{g} \leftarrow (1 / L) \, (\Sigma_i \, \bar{g}_i + \mathcal{N}(0, \sigma^2 C^2 \, I))$ // add calibrated noise
  $\theta_t \leftarrow \theta_{t-1} - \eta \, \tilde{g}$
$\varepsilon \leftarrow$ privacy accountant(q, σ, T, δ)
return θ_T, (ε, δ)

DP-SGD typically costs some accuracy, more for small datasets and large models, and requires per-example gradients, which libraries such as Opacus (PyTorch) and TensorFlow Privacy compute efficiently. Listing 3.15 wraps a standard PyTorch training set-up with Opacus. DP has been deployed at scale, for example in the 2020 US Census disclosure-avoidance system and in telemetry collection by major technology compa-

nies.

Listing 3.15 Differentially private training with Opacus.

```python
import torch
from torch import nn
from opacus import PrivacyEngine

model = nn.Sequential(nn.Linear(30, 64), nn.ReLU(), nn.Linear(64, 2))
optimiser = torch.optim.SGD(model.parameters(), lr=0.05)
# train_loader: a standard torch DataLoader over the private training data

engine = PrivacyEngine()
model, optimiser, train_loader = engine.make_private_with_epsilon(
    module=model, optimizer=optimiser, data_loader=train_loader,
    target_epsilon=3.0, target_delta=1e-5, epochs=20, max_grad_norm=1.0)  # clipping C
    ↪ = 1

for epoch in range(20):
    for x, y in train_loader:
        optimiser.zero_grad()
        nn.functional.cross_entropy(model(x), y).backward()   # per-example grads,
        ↪ clipped
        optimiser.step()                                       # noise added here
print("spent epsilon:", engine.get_epsilon(delta=1e-5))
```

3.18.3 Homomorphic Encryption and Secure Multi-Party Computation

Differential privacy protects what a model *reveals*; cryptographic techniques protect data while it is being *computed on*. **Homomorphic encryption** (HE) allows computations on encrypted data: the result, once decrypted, equals the result of the same computation on the plaintext. Partially homomorphic schemes support one operation (Paillier encryption supports addition); **fully homomorphic encryption** (FHE), first constructed by Gentry (2009), supports arbitrary computation. Schemes such as CKKS support approximate arithmetic on real numbers and make encrypted inference of small neural networks feasible: a client encrypts its input, the server evaluates the model on the ciphertext, and only the client can decrypt the prediction. The cost is high, often thousands of times slower than plaintext computation, and non-linear activations must be approximated by polynomials.

Secure multi-party computation (MPC), founded on work by Yao (1982) and Goldreich, Micali and Wigderson (1987), lets several parties jointly compute a function of their private inputs while learning nothing beyond the output. A basic building block is **additive secret sharing**: a value x is split into random shares $x_1 + x_2 + \dots + x_k = x$ (modulo a large number), each held by a different party; no single share reveals anything about x, yet parties can add shared values locally and multiply them with a little

interaction. Secure aggregation in federated learning (Bonawitz et al., 2017) uses such techniques so that the server learns only the sum of client updates. **Trusted execution environments** (hardware enclaves such as Intel SGX and confidential-computing GPUs) offer a faster but hardware-trust-based alternative. Table 3.26 compares the techniques.

Table 3.26 Comparison of privacy-preserving machine learning techniques.

Technique	Protects against	Guarantee	Main cost
Differential privacy	Inference about individuals from outputs	Statistical (ε, δ)	Accuracy loss
Federated learning	Central collection of raw data	Architectural only	Communication, heterogeneity
Homomorphic encryption	Server seeing inputs or outputs	Cryptographic	Very high computation
Secure MPC	Parties seeing each other's inputs	Cryptographic	Communication rounds
Trusted execution environments	Host OS and operators	Hardware trust	Side-channel risk, hardware
Synthetic data	Release of real records	None unless generated with DP	Fidelity loss

K Key Points | Privacy-Preserving ML

- Trained models can leak training data through membership inference, inversion and memorisation.
- Differential privacy bounds any individual's influence; DP-SGD clips per-example gradients and adds noise.
- HE and MPC protect data during computation at significant computational or communication cost.
- Techniques combine: federated learning + secure aggregation + DP is a common strong design.

3.19 Green AI and Energy-Efficient Computing

The computation used to train the largest AI models has grown by orders of magnitude over the past decade, and with it energy use and carbon emissions. Strubell, Ganesh and McCallum (2019, ACL) estimated the emissions of training NLP models including neural architecture search, drawing wide attention to the issue; Schwartz et al. (2020, *Communications of the ACM*) coined the term **Green AI** for research that treats efficiency as an evaluation criterion alongside accuracy, in contrast to 'Red AI' that buys accuracy with ever more computation. Inference matters as well: a model served billions of times can consume far more energy over its lifetime than its training did.

3.19.1 Estimating Energy Use and Carbon Footprint

A practical estimate of the operational carbon footprint of a training or inference job multiplies the energy consumed by the hardware, the data centre's overhead and the carbon intensity of the electricity grid, as in Equation (3.46) (following the methodology of Lacoste et al., 2019, and Patterson et al., 2021).

$$E = t \times \left(\sum_{g=1}^{G} P_g \, u_g + P_{\text{CPU}} + P_{\text{mem}} \right) \times \text{PUE}$$
$$\text{CO}_2\text{e} = E \times I_{\text{grid}} \tag{3.46}$$

where t is runtime in hours, P_g the power draw of accelerator g (kW) at average utilisation u_g, PUE the data-centre power usage effectiveness (typically 1.1–1.6), E the energy in kWh, and I_{grid} the grid carbon intensity in kg CO_2e per kWh.

E Worked Example 3.11 | Carbon Cost of a Fine-Tuning Run

A model is fine-tuned on 8 GPUs, each drawing about 0.4 kW at 80% average utilisation, for 24 hours. CPU and memory add about 0.3 kW, and the data centre's PUE is 1.2.

Power: $8 \times 0.4 \times 0.8 + 0.3 = 2.56 + 0.3 = 2.86$ kW. Energy: $E = 24 \times 2.86 \times 1.2 \approx 82.4$ kWh.

On a grid with intensity 0.7 kg CO_2e/kWh (coal-heavy), emissions are $82.4 \times 0.7 \approx 57.7$ kg CO_2e. On a grid at 0.05 kg CO_2e/kWh (largely hydro or nuclear), they are about 4.1 kg: a fourteen-fold difference for the same computation, which is why location and timing matter.

Figure 3.23 makes the same point for a larger training job across grids with different carbon intensities. Tools such as CodeCarbon and the ML CO_2 Impact calculator automate these estimates, and many venues now encourage authors to report compute and energy alongside results. A complete life-cycle view also includes **embodied** emissions from manufacturing hardware and building data centres, and water used for cooling.

3.19.2 Strategies for Energy-Efficient AI

Efficiency can be improved at every level of the stack, as summarised in Table 3.27. At the *model* level, smaller architectures, **knowledge distillation** (training a small student to mimic a large teacher), **pruning** and **quantisation** to 8-bit or 4-bit arithmetic reduce computation with little accuracy loss, and sparse **mixture-of-experts** models activate only a fraction of their parameters per token. At the *training* level, **mixed-precision** arithmetic, efficient attention kernels, parameter-efficient fine-tuning (LoRA, adapters) instead of full fine-tuning, transfer learning instead of training from scratch, and compute-optimal scaling (Hoffmann et al., 2022, showed that many large language models were under-trained for their size) all cut energy. At the *experimentation* level, multi-fidelity hyperparameter search (Section 8) and early stopping

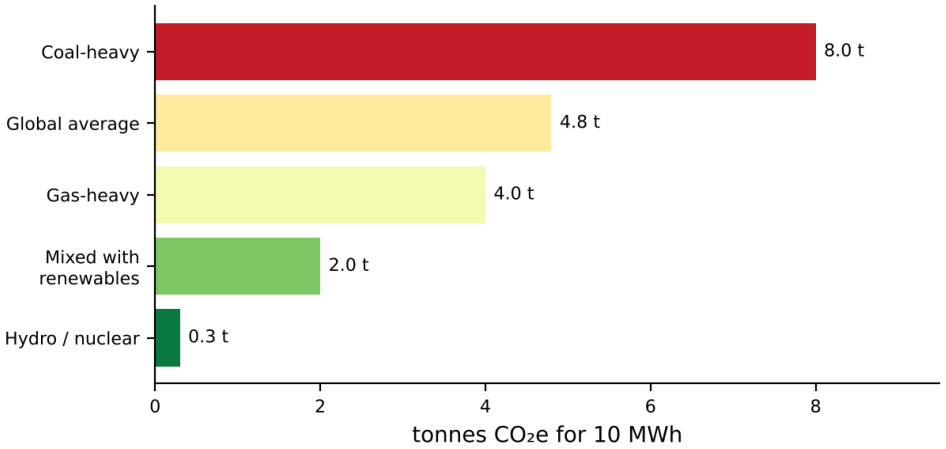


Figure 3.23 Estimated emissions of a 10 MWh training run under different grid carbon intensities (illustrative values). The same computation can differ in footprint by more than an order of magnitude.

avoid wasted runs. At the *infrastructure* level, efficient accelerators, high-utilisation scheduling, low-PUE data centres and **carbon-aware scheduling** (running flexible jobs when and where electricity is cleanest) reduce emissions per unit of computation.

Table 3.27 Strategies for reducing the energy and carbon cost of AI.

Level	Strategy	Typical effect
Model	Distillation, pruning, quantisation (INT8/INT4)	2–10× cheaper inference
Model	Mixture-of-experts, efficient architectures	Fewer FLOPs per prediction
Training	Mixed precision, fused kernels	Faster training, lower energy
Training	Transfer learning, LoRA/adapters	Orders of magnitude less compute than pre-training
Experimentation	Hyperband/ASHA, early stopping, smaller pilots	Fewer wasted GPU-hours
Infrastructure	Low-PUE data centres, efficient accelerators	Lower overhead per FLOP
Scheduling	Carbon-aware timing and location	Lower CO_2e for the same energy
Reporting	Track energy (CodeCarbon), report compute	Accountability and comparability

Listing 3.16 Measuring the energy and emissions of a training run with CodeCarbon.

```python
from codecarbon import EmissionsTracker
from sklearn.datasets import make_classification
from sklearn.ensemble import RandomForestClassifier

X, y = make_classification(n_samples=200_000, n_features=50, random_state=0)

tracker = EmissionsTracker(project_name="rf-baseline", measure_power_secs=5)
tracker.start()
model = RandomForestClassifier(n_estimators=300, n_jobs=-1, random_state=0).fit(X, y)
emissions_kg = tracker.stop()          # estimated kg CO2e; details written to
↪ emissions.csv
print(f"Estimated emissions: {emissions_kg * 1000:.1f} g CO2e")
```

Listing 3.16 shows how little code is needed to measure a job's footprint: CodeCarbon samples hardware power counters and looks up the regional grid intensity. Reporting such numbers alongside accuracy makes efficiency a visible, comparable property of research.

3.20 Real-World AI Case Studies and End-to-End Projects

The preceding sections introduced techniques one at a time. Real projects integrate many of them: a clear problem definition, reliable data pipelines, appropriate models, careful evaluation, deployment, monitoring, and attention to fairness, privacy and cost. This section presents five representative case studies drawn from common application domains. They are composites of well-documented industry and research practice rather than accounts of a single organisation, and each highlights the decisions that matter most in its domain. Part B (Sections 34 and 35) then walks through complete projects in code.

3.20.1 A Common End-to-End Lifecycle

Despite differences in domain, successful projects follow a recognisable lifecycle, shown in Figure 3.24. It begins with framing a business or scientific problem as a learning task with a measurable success criterion and a simple baseline, and ends with a monitored system that feeds new data and lessons back into the next iteration. The arrows back to earlier stages are the norm, not the exception: error analysis frequently reveals data problems, and monitoring frequently reveals drift.

Table 3.28 summarises the five case studies that follow.

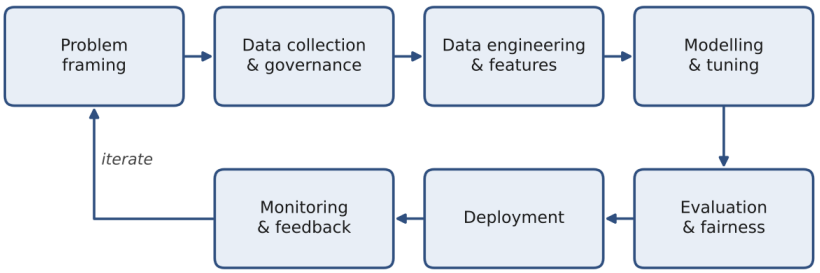


Figure 3.24 The end-to-end lifecycle of an applied AI project. Monitoring closes the loop by triggering data collection, retraining or re-framing.

Table 3.28 Summary of the five case studies.

Domain	Task type	Key techniques	Primary metric	Main risk
Healthcare diagnosis	Image classification	Transfer learning, calibration, XAI	Sensitivity at fixed specificity	Dataset shift across sites
Fraud detection	Imbalanced classification, streaming	Gradient boosting, graph features, online updates	Precision–recall AUC, cost	Adversarial adaptation
Demand forecasting	Time-series regression	Global GBMs, deep forecasters, quantiles	WAPE, pinball loss	Shocks and promotions
Recommendation	Ranking	Two-tower retrieval, re-ranking, A/B tests	Online engagement, NDCG	Feedback loops, filter bubbles
Agriculture	Image classification, regression	Lightweight CNNs, edge deployment, remote sensing	Accuracy in field conditions	Domain gap lab→field

3.20.2 Case Studies

Case Study 3.2 | Case Study 1: Diagnostic Support from Medical Images

Problem. Screening for diabetic retinopathy from retinal fundus photographs, where specialist graders are scarce. Gulshan et al. (2016, *JAMA*) showed that a deep convolutional network could match ophthalmologists on referable retinopathy, and such systems have since been deployed in screening programmes.

Pipeline. Images are de-identified and graded by several experts; adjudicated labels reduce noise. An ImageNet-pretrained CNN is fine-tuned with augmentation. The operating threshold is chosen for high sensitivity at an acceptable specificity, and probabilities are calibrated (Section 3's base-rate effect makes positive predictive value depend strongly on prevalence).

Lessons. Performance often drops at new clinics because of different cameras, image quality and patient populations, so external validation at independent sites is essential. Saliency maps help clinicians trust and audit outputs, and the system is positioned as decision *support* with human review, a design that also aligns with the high-risk requirements of Section 17.

Case Study 3.3 | Case Study 2: Real-Time Payment Fraud Detection

Problem. Score each card transaction within tens of milliseconds and block or challenge likely fraud. Fraud rates are typically well below 1%, labels (chargebacks) arrive weeks later, and fraudsters adapt to the model.

Pipeline. Streaming features from a feature store (Section 7), such as spend velocity over the last hour and distance from the usual location, feed a gradient-boosted tree model; graph features capture rings of linked accounts and devices. Class imbalance is handled with weighting and evaluation by precision–recall curves and expected monetary cost rather than accuracy.

Lessons. Concept drift (Section 11) is constant, so models are retrained frequently and monitored for drift. Point-in-time-correct training data prevent leakage from post-transaction information. Explanations of declined transactions support customer service and regulatory obligations.

Case Study 3.4 | Case Study 3: Retail Demand Forecasting

Problem. Forecast daily sales of tens of thousands of products in hundreds of stores to drive replenishment; under-forecasting causes stock-outs and over-forecasting causes waste.

Pipeline. Rather than one model per series, a single *global* model (gradient-boosted trees such as LightGBM, or deep forecasters such as DeepAR or the Temporal Fusion Transformer) is trained across all series with features for lags, rolling statistics, calendar effects, prices and promotions. The M5 forecasting competition (Makridakis et al., 2022) found tree-based global models among the most accurate approaches. Quantile forecasts trained with the pinball loss support inventory decisions at chosen service levels.

Lessons. Validation must respect time (rolling-origin back-testing, never random splits). External shocks such as pandemics break historical patterns; monitoring and human override are required.

Case Study 3.5 | Case Study 4: Large-Scale Recommendation

Problem. Choose a handful of items (videos, products, articles) for each user from a catalogue of millions, in real time.

Pipeline. A multi-stage architecture (as described for YouTube by Covington, Adams and Sargin, 2016) first retrieves hundreds of candidates with a two-tower embedding model and approximate nearest-neighbour search (the same vector-search technology as Section 15), then ranks them with a richer model using user, item and context features, and finally re-ranks for diversity, freshness and business rules.

Lessons. Offline metrics such as NDCG correlate imperfectly with user satisfaction, so decisions rest on online A/B tests. Recommendations influence future training data, creating feedback loops and popularity bias; exploration and causal evaluation (Section 9) mitigate them.

Case Study 3.6 | Case Study 5: Crop Disease Detection for Smallholder Farmers

Problem. Help farmers identify plant diseases early from smartphone photographs, where

agronomists are scarce and connectivity is poor.

Pipeline. Public datasets such as PlantVillage (over 50,000 leaf images; Mohanty, Hughes and Salathé, 2016) enable transfer learning, but lab images on uniform backgrounds differ greatly from field photos, so locally collected and annotated field images are essential. Lightweight networks (for example MobileNet or EfficientNet variants), quantised for on-device inference, run offline on low-cost phones. At larger scale, satellite and drone imagery support yield prediction and irrigation planning.

Lessons. Accuracy on held-out lab images can exceed 99% yet fall sharply in the field; evaluation must use realistic conditions. Advice must be locally appropriate and delivered in local languages, and few-shot learning (Section 13) helps add new diseases with few labelled images.

3.20.3 Cross-Cutting Lessons

Several lessons recur across the case studies. First, **data quality and representativeness** dominate model choice: in every domain, the largest failures came from training data that differed from deployment conditions. Second, **evaluation must mirror the decision**: sensitivity at a clinical threshold, monetary cost of fraud, service-level-specific forecast error, online engagement. Third, **deployment is the beginning, not the end**: drift monitoring, retraining and incident response are permanent responsibilities. Fourth, **humans remain in the loop** for high-stakes decisions, and systems should be designed to support their judgement. Finally, responsible-AI considerations (fairness across sites and groups, privacy of personal data, energy cost) are integral engineering requirements rather than afterthoughts.

Key Points | End-to-End Projects

- Frame the problem with a measurable objective and a simple baseline before choosing models.
- Validate on data that reflect deployment conditions: external sites, future time periods, field images.
- Choose metrics and thresholds that reflect the real costs of errors.
- Plan monitoring, retraining and human oversight as part of the initial design.

PART B

PART B

Practical and Research-Oriented Chapters

3.21 Setting Up Python, Jupyter Notebook, and Google Colab

Part A of this chapter dealt with the conceptual and mathematical machinery of modern machine learning. Part B turns to practice: the tools, workflows and research habits that turn an idea into a working, trustworthy model. Every subsequent section assumes a working Python environment, so we begin by building one carefully. A surprising fraction of failed projects, irreproducible papers and lost afternoons can be traced to a poorly managed environment: two projects that need incompatible versions of a library, a notebook that silently imports a different package from the one the author intended, or a GPU driver that does not match the deep learning framework. A small amount of discipline at the start prevents these problems.

3.21.1 The Python Data Science Stack

Python has become the dominant language for machine learning not because it is the fastest language but because it is a pleasant *glue* language around fast compiled libraries. NumPy wraps optimised C and Fortran routines for array arithmetic; pandas provides labelled tabular data; Matplotlib draws figures; scikit-learn implements classical learning algorithms behind a uniform interface; and TensorFlow and PyTorch provide automatic differentiation and GPU acceleration for neural networks. Jupyter Notebook (and its successor interface JupyterLab) offers an interactive, cell-based document in which code, output, figures and prose live together, which makes it ideal for exploration and teaching. Google Colaboratory (Colab) is a hosted Jupyter service that runs in the browser and gives free, time-limited access to GPUs and TPUs.

The stack is layered, as Figure 3.25 illustrates. At the bottom sits the operating system and, optionally, the GPU driver and CUDA toolkit; above that, a Python interpreter isolated in a virtual environment; then the scientific libraries; and finally the interactive front end. Problems at a lower layer propagate upwards, so when something breaks it is usually worth checking the layers from the bottom up.

3.21.2 Local Installation Step by Step

There are two mainstream routes to a local installation. The first uses the official Python distribution from python.org with the built-in `venv` module and the `pip` package installer. The second uses **conda**, the package and environment manager distributed with Anaconda and its minimal variant Miniconda (or the community-driven Miniforge). Conda can install non-Python dependencies such as compilers, BLAS libraries

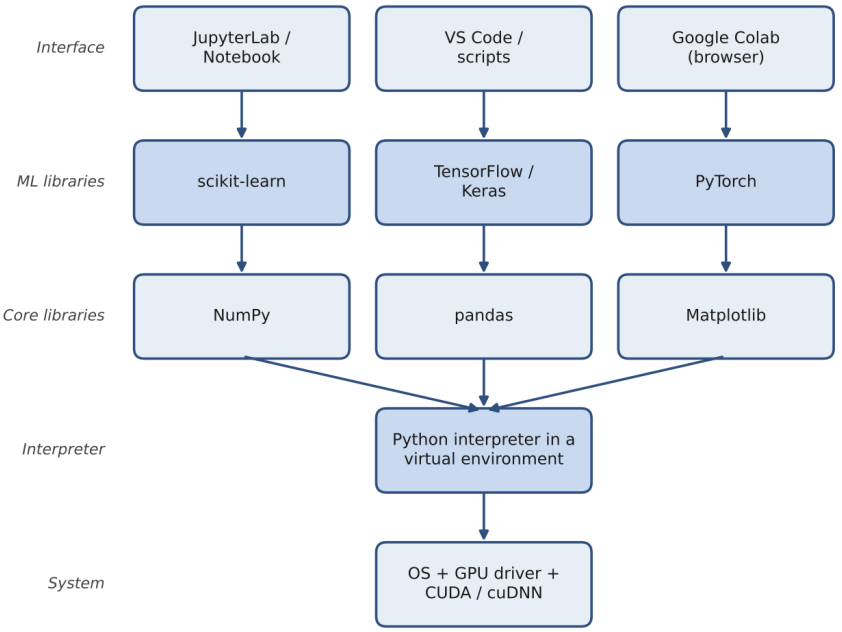


Figure 3.25 Layers of a typical machine learning environment. Each layer depends on the one beneath it, and isolating the interpreter in a virtual environment keeps projects independent.

and CUDA runtimes, which makes it attractive on Windows and for GPU work; pip with `venv` is lighter and is the default in most production containers. Table 3.29 summarises a sequence that works on all three major operating systems.

3.21.3 Virtual Environments and Conda

> **D Definition 3.5 | Virtual Environment**
>
> A **virtual environment** is a self-contained directory holding a Python interpreter and its own set of installed packages, so that the dependencies of one project cannot conflict with those of another or with the system Python.

Listing 3.17 shows a typical conda workflow. An `environment.yml` file lists the channels and packages; anyone can recreate the same environment with a single command. The `ipykernel` registration step makes the environment selectable as a kernel inside Jupyter, which avoids the common mistake of launching Jupyter from one environment while expecting packages from another.

Table 3.29 Recommended installation sequence for a local machine learning environment.

Step	Action	Command or check
1	Install a Python distribution (3.10–3.12) or Miniconda	`python --version` or `conda --version`
2	Create an isolated environment for the project	`python -m venv .venv` or `conda create -n ml python=3.11`
3	Activate the environment	`source .venv/bin/activate` / `conda activate ml`
4	Upgrade the installer	`python -m pip install --upgrade pip`
5	Install core libraries	`pip install numpy pandas matplotlib scikit-learn`
6	Install a deep learning framework	follow the selector on pytorch.org or tensorflow.org
7	Install Jupyter and register the kernel	`pip install jupyterlab ipykernel`
8	Freeze the environment for reproducibility	`pip freeze > requirements.txt`
9	Verify the GPU (if any)	`nvidia-smi`; `torch.cuda.is_available()`

Listing 3.17 Creating, exporting and registering a conda environment (shell commands).

```
# environment.yml
# name: ml-book
# channels: [conda-forge]
# dependencies: [python=3.11, numpy, pandas, matplotlib, scikit-learn, jupyterlab, ipykernel]

conda env create -f environment.yml        # build the environment from the file
conda activate ml-book                     # switch into it
python -m ipykernel install --user --name ml-book --display-name "Python (ml-book)"
pip install torch torchvision              # pip inside conda is fine for packages absent from conda-forge
conda env export --from-history > environment.lock.yml   # record what was explicitly requested
jupyter lab                                # launch the notebook interface
```

Practice Tip | Choosing Between pip and Conda

- Use **conda** (preferably the conda-forge channel) when you need compiled scientific libraries, GPU runtimes, or you work on Windows.
- Use **venv + pip** for lightweight projects and inside Docker images.

- Avoid mixing `conda install` and `pip install` for the same package; install everything possible with conda first, then the remainder with pip.
- Newer tools such as `uv` and `poetry` resolve and lock dependencies faster, but the principles of isolation and pinning are identical.

3.21.4 Working in Jupyter Notebook

A notebook is an ordered list of cells. Code cells are executed by a *kernel*, a separate process that holds program state; markdown cells contain formatted text and equations. The kernel's state depends on the order in which cells were *run*, not the order in which they appear. A notebook that has been edited and re-run out of order can therefore display results that no top-to-bottom execution would reproduce. Before sharing a notebook, always choose *Restart Kernel and Run All Cells*. Useful magics include `%timeit` for micro-benchmarks, `%matplotlib inline` for embedded figures (the default in recent versions) and `%load_ext autoreload` with `%autoreload 2` to pick up edits to imported modules without restarting.

Notebooks are excellent for exploration but poor for long-lived code: they are hard to diff in version control, encourage global state and cannot easily be unit-tested. A healthy practice is to prototype in a notebook, then move stable functions into a Python package (for example `src/`) that the notebook imports. Tools such as `jupytext` keep a plain-text `.py` mirror of each notebook for clean version control.

3.21.5 Google Colab: GPU Runtimes and Drive Mounting

Colab removes installation entirely: a notebook opens in the browser, with NumPy, pandas, scikit-learn, TensorFlow and PyTorch preinstalled. To use an accelerator, choose *Runtime* → *Change runtime type* and select a GPU or TPU. Sessions are ephemeral: the virtual machine is recycled after a period of inactivity or a maximum session length, and every file written to the local disk is lost. Persistent storage is therefore provided by mounting Google Drive. Listing 3.18 shows the standard opening cell of a Colab notebook.

Listing 3.18 Typical first cell in Google Colab: check the GPU, mount Drive and install extras.

```
import torch, subprocess, sys

# 1. Confirm the accelerator assigned to this runtime
print("CUDA available:", torch.cuda.is_available())
if torch.cuda.is_available():
    print("Device:", torch.cuda.get_device_name(0))
print(subprocess.run(["nvidia-smi", "--query-gpu=name,memory.total",
                      "--format=csv"], capture_output=True, text=True).stdout)
```

```

# 2. Mount Google Drive for persistent storage
from google.colab import drive
drive.mount("/content/drive")
DATA_DIR = "/content/drive/MyDrive/ml-book/data"
CKPT_DIR = "/content/drive/MyDrive/ml-book/checkpoints"

# 3. Install packages that are not preinstalled (lost when the runtime resets)
subprocess.run([sys.executable, "-m", "pip", "install", "-q", "mlflow==2.*"])

# 4. Copy a large dataset to the fast local disk before training
subprocess.run(["cp", "-r", f"{DATA_DIR}/images", "/content/images"])
```

Note | Colab Pitfalls

- Reading thousands of small files directly from Drive is slow; copy or unzip them to `/content` first.
- Save model checkpoints to Drive regularly, because a disconnected runtime loses all local files.
- Free-tier GPU type and session length vary with demand; record the GPU name in your experiment log.
- Never paste API keys into notebook cells that you share; use Colab's *Secrets* panel instead.

3.22 Data Analysis with NumPy and Pandas

Almost every machine learning project begins with numerical arrays and tables. NumPy supplies the array, the fundamental data structure of scientific Python; pandas builds labelled, heterogeneous tables on top of it. Fluency with both libraries is the single most valuable practical skill for a data scientist, because it determines how quickly ideas can be tested. This section concentrates on the three ideas that make NumPy fast — homogeneous arrays, broadcasting and vectorisation — and then on the pandas operations used daily in data preparation.

3.22.1 NumPy Arrays

Definition 3.6 | ndarray

A NumPy **ndarray** is a fixed-size, homogeneous, n-dimensional grid of values stored in one contiguous block of memory, described by a data type (`dtype`), a `shape` tuple and `strides` that give the byte step along each axis.

Because all elements share one type and sit contiguously in memory, operations can be executed in compiled loops that exploit CPU caches and SIMD instructions. Reshaping and transposing usually return *views* that share memory with the original array by adjusting the strides, so they cost almost nothing; boolean and integer-array ("fancy") indexing, by contrast, return *copies*. Knowing which operations copy is important when arrays are large.

3.22.2 Broadcasting and Vectorisation

Vectorisation means expressing a computation as whole-array operations rather than explicit Python loops. **Broadcasting** is the set of rules that lets NumPy combine arrays of different shapes without copying data. Two shapes are compatible if, comparing dimensions from the right, each pair is equal or one of them is 1; the size-1 dimension is then virtually stretched. Figure 3.26 illustrates the most common case in machine learning: subtracting a per-feature mean vector of shape (d,) from a data matrix of shape (n, d). The standardisation in Equation (3.47) is therefore a single line of NumPy.

$$Z_{ij} = \frac{X_{ij} - \mu_j}{\sigma_j}, \quad \mu_j = \frac{1}{n}\sum_{i=1}^{n} X_{ij}, \quad \sigma_j = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(X_{ij} - \mu_j)^2} \tag{3.47}$$

where X is the n × d data matrix, μ_j and σ_j are the mean and standard deviation of feature j, and Z is the standardised matrix; in NumPy, `Z = (X - X.mean(0)) / X.std(0)`.

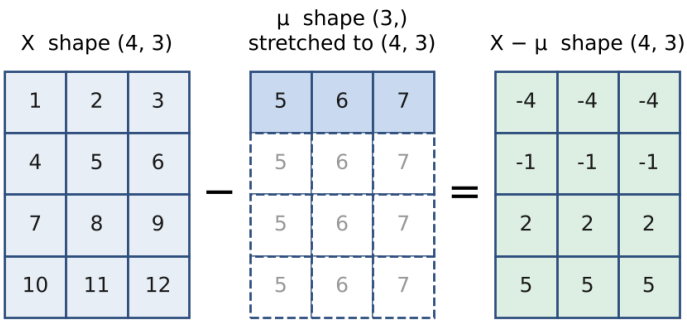


Figure 3.26 Broadcasting a (4, 3) matrix with a (3,) vector: the vector is virtually repeated along the first axis, so the subtraction is element-wise without any copying.

Listing 3.19 contrasts a loop-based and a vectorised computation of pairwise Euclidean distances, a core operation in k-nearest neighbours and clustering. The vectorised version uses the identity $\|a - b\|^2 = \|a\|^2 + \|b\|^2 - 2a^{T}b$ together with broadcasting, and is typically two to three orders of magnitude faster than the double loop for a few thousand points.

Listing 3.19 Array creation, broadcasting and a vectorised pairwise-distance computation.

```python
import numpy as np

rng = np.random.default_rng(42)            # modern, reproducible random generator
X = rng.normal(size=(2000, 16))            # 2,000 samples, 16 features
C = rng.normal(size=(50, 16))              # 50 centroids

# Standardise every column in one line (broadcasting (n,d) with (d,))
Z = (X - X.mean(axis=0)) / X.std(axis=0)

# Slow: explicit Python loops
def dist_loop(X, C):
    D = np.empty((len(X), len(C)))
    for i in range(len(X)):
        for j in range(len(C)):
            D[i, j] = np.sqrt(((X[i] - C[j]) ** 2).sum())
    return D

# Fast: ||x||^2 + ||c||^2 - 2 x.c, using broadcasting of (n,1) + (1,k)
def dist_vec(X, C):
    sq = (X**2).sum(1)[:, None] + (C**2).sum(1)[None, :] - 2 * X @ C.T
    return np.sqrt(np.maximum(sq, 0.0))     # clip tiny negatives from round-off

assert np.allclose(dist_loop(X[:100], C), dist_vec(X[:100], C))
nearest = dist_vec(X, C).argmin(axis=1)    # index of closest centroid per sample
print(np.bincount(nearest, minlength=50)[:10])
```

E Worked Example 3.12 | Checking Broadcast Compatibility

Shapes $(8, 1, 6)$ and $(7, 1)$ — align from the right: $(8, 1, 6)$ vs $(\cdot, 7, 1)$.

- Last axis: 6 vs 1 → compatible, result 6.
- Middle axis: 1 vs 7 → compatible, result 7.
- First axis: 8 vs (missing, treated as 1) → result 8.

The result has shape $(8, 7, 6)$. By contrast $(4, 3)$ and $(4,)$ are *incompatible*: $3 \neq 4$ on the last axis. Writing `x[:, None]` to obtain shape $(4, 1)$ fixes it, giving a $(4, 3)$ result.

3.22.3 Pandas DataFrames

A pandas **DataFrame** is a two-dimensional table whose columns may have different types, with labelled row and column indices. Each column is a **Series**. DataFrames support selection by label (`.loc`) and by position (`.iloc`), boolean filtering, handling of missing values (`isna`, `fillna`, `dropna`), type conversion, string and date-time accessors (`.str`, `.dt`), and, crucially, the *split–apply–combine* pattern implemented by `groupby`. Tables are combined with `merge` (SQL-style joins) and `concat` (stacking), and reshaped with `pivot_table` (long to wide) and `melt` (wide to long). Table 3.30 lists the operations used most often.

Table 3.30 Frequently used pandas operations and their purpose.

Task	pandas idiom	Notes
Load / save	`pd.read_csv`, `read_parquet`, `to_parquet`	Parquet is columnar, typed and much faster
Inspect	`df.info()`, `df.describe()`, `df.head()`	Check dtypes and missing counts first
Select	`df.loc[rows, cols]`, `df.iloc[i, j]`	Label-based vs position-based
Filter	`df[df.age > 30]`, `df.query('age > 30')`	Combine conditions with & and |
New column	`df.assign(bmi=w / h**2)`	Chainable, avoids SettingWithCopy warnings
Missing data	`isna().sum()`, `fillna`, `dropna`	Impute inside a pipeline for modelling
Aggregate	`groupby(key).agg(...)`	Split–apply–combine
Group transform	`groupby(key)[c].transform('mean')`	Returns a column aligned to the original rows
Join	`pd.merge(a, b, on=key, how='left')`	Use `validate='m:1'` to catch duplicate keys
Reshape	`pivot_table`, `melt`, `stack`	Long ↔ wide formats
Time series	`resample('W')`, `rolling(7)`	Requires a DatetimeIndex
Categories	`astype('category')`, `pd.get_dummies`	Saves memory; one-hot encoding

Listing 3.20 shows these operations working together on a small retail dataset: a join of transactions to customers, a group-wise aggregation and a pivot table summarising revenue by region and month.

Listing 3.20 A typical pandas workflow: merge, derive, group, aggregate and pivot.

```
import pandas as pd

tx = pd.read_csv("transactions.csv", parse_dates=["date"])   # id, customer_id, date,
↪  amount
cust = pd.read_csv("customers.csv")                          # customer_id, region,
↪  segment

df = (tx.merge(cust, on="customer_id", how="left", validate="m:1")
```

```
        .assign(month=lambda d: d["date"].dt.to_period("M"),
                big=lambda d: d["amount"] > 100))

print(df.isna().sum())                                          # missing values per
↪  column

# Split-apply-combine: several statistics per segment
summary = (df.groupby("segment")
             .agg(n_tx=("id", "count"),
                  revenue=("amount", "sum"),
                  avg_basket=("amount", "mean"),
                  share_big=("big", "mean"))
             .sort_values("revenue", ascending=False))

# Each customer's spend relative to their segment average (transform keeps row
↪  alignment)
df["rel_spend"] = df["amount"] / df.groupby("segment")["amount"].transform("mean")

# Wide table: revenue by region (rows) and month (columns)
pivot = df.pivot_table(index="region", columns="month", values="amount",
                       aggfunc="sum", fill_value=0)
print(summary.round(2)); print(pivot.iloc[:, -3:])
```

Practice Tip | Writing Fast and Readable Pandas

- Avoid `df.iterrows()` and `apply` with Python functions on large frames; prefer vectorised column arithmetic, `np.where`, `pd.cut` and `groupby` aggregations.
- Use method chaining with `assign` and `pipe` so each transformation is one readable step.
- Convert repetitive strings to `category` dtype and store intermediate data as Parquet.
- For data that does not fit in memory, consider Polars, DuckDB or Dask (see the Big Data section in Part A).

3.23 Data Visualization with Matplotlib and Other Tools

Visualisation serves two distinct purposes in machine learning. *Exploratory* graphics help the analyst understand distributions, spot outliers, detect leakage and form hypotheses; they are made quickly and discarded freely. *Explanatory* graphics communicate a finding to others — a learning curve in a paper or a feature-importance chart for stakeholders — and demand careful design. The same principles apply to both: choose the chart that matches the question, show the data honestly, label axes with units, and remove decoration that does not carry information.

3.23.1 Choosing the Right Chart

The first decision is the chart type, which depends on the number and kind of variables involved and the question being asked. Table 3.31 provides a practical selection guide.

Table 3.31 Choosing a chart type according to the analytical question.

Question	Variables	Recommended chart	Avoid
How is one variable distributed?	1 numeric	Histogram, KDE, ECDF	Pie chart
How do distributions compare across groups?	1 numeric + 1 categorical	Box plot, violin, strip plot	Bar of means only
How do two variables relate?	2 numeric	Scatter (hexbin if dense)	Line plot of unordered data
How does a quantity change over time?	time + numeric	Line chart	Bar chart with many bars
How do categories compare?	1 categorical + 1 numeric	Sorted bar chart, dot plot	3-D bars
What is the pattern in a matrix?	2 categorical/ordinal + numeric	Heatmap	Table of raw numbers
How do many variables relate?	many numeric	Pair plot, correlation heatmap, PCA scatter	Overloaded scatter
How good is a classifier?	scores + labels	ROC/PR curve, confusion matrix	Accuracy bar alone

3.23.2 Matplotlib Fundamentals

Matplotlib, created by John D. Hunter in 2003, is the foundation of most Python plotting. Its object-oriented interface centres on a **Figure** (the whole canvas) containing one or more **Axes** (individual plots with their own coordinate system). Explicitly creating these objects with `fig, ax = plt.subplots()` and calling methods on `ax` is more predictable than the state-based `plt.plot` style, especially for multi-panel figures. Listing 3.21 produces the histogram and box plot shown in Figure 3.27.

Listing 3.21 Histogram and grouped box plot with Matplotlib's object-oriented API.

```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(0)
income = rng.lognormal(mean=10.5, sigma=0.5, size=2000) / 1000      # in $1000s
groups = {"A": rng.normal(52, 8, 300), "B": rng.normal(60, 12, 300),
          "C": np.r_[rng.normal(48, 6, 290), rng.normal(95, 5, 10)]}  # C has outliers

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(9, 3.5), constrained_layout=True)
ax1.hist(income, bins=40, color="tab:blue", edgecolor="white")
ax1.axvline(np.median(income), color="tab:orange", ls="--", label="median")
ax1.set(xlabel="Annual income ($1000s)", ylabel="Count", title="Histogram")
ax1.legend(frameon=False)
```

```
ax2.boxplot(list(groups.values()), labels=list(groups.keys()))
ax2.set(xlabel="Customer group", ylabel="Session length (min)", title="Box plot")
for ax in (ax1, ax2):
    ax.spines[["top", "right"]].set_visible(False)
fig.savefig("hist_box.png", dpi=200)
```

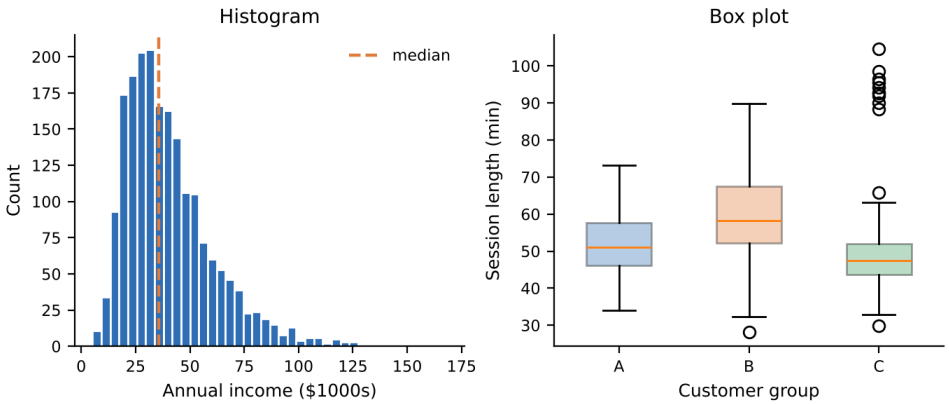


Figure 3.27 Left: a histogram reveals the right skew of a log-normally distributed income variable (median dashed). Right: box plots compare three groups; the points beyond the whiskers of group C are flagged outliers.

The box plot encodes a five-number summary: the box spans the first to third quartile (Q_1 to Q_3), the inner line marks the median, and the whiskers extend to the most extreme points within the Tukey fences of Equation (3.48); points beyond the fences are drawn individually as potential outliers.

$$\text{lower} = Q_1 - 1.5\,\text{IQR}, \quad \text{upper} = Q_3 + 1.5\,\text{IQR}, \quad \text{IQR} = Q_3 - Q_1 \tag{3.48}$$

where Q_1 and Q_3 are the 25th and 75th percentiles and IQR is the inter-quartile range.

3.23.3 Scatter Plots and Heatmaps

Scatter plots reveal relationships, clusters and heteroscedasticity between two numeric variables; colour can encode a third, categorical variable. Heatmaps display a matrix with colour, which makes them ideal for correlation matrices and confusion matrices. A diverging colour map centred on zero should be used for signed quantities such as correlations, and the numeric values should be annotated when the matrix is small. Figure 3.28 shows both chart types; the heatmap immediately reveals two blocks of mutually correlated features, a hint that dimensionality reduction or regularisation may help.

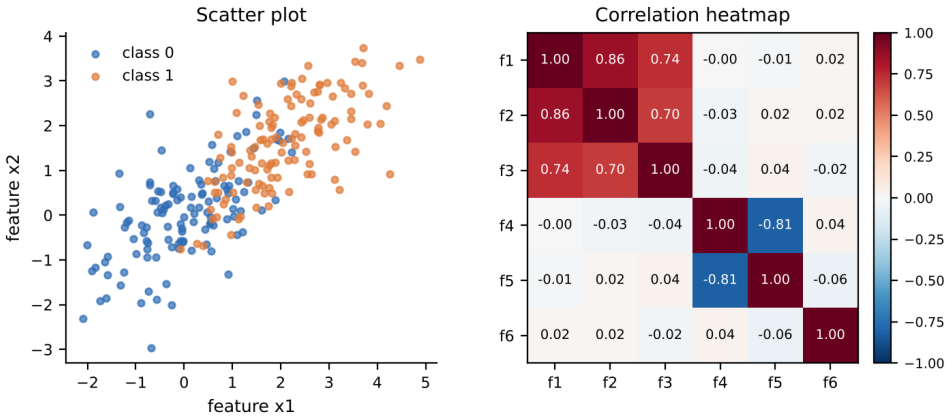


Figure 3.28 Left: a scatter plot coloured by class shows two partially overlapping groups. Right: an annotated correlation heatmap with a diverging colour map exposes two blocks of correlated features.

3.23.4 Seaborn and Plotly

Seaborn, built on Matplotlib by Michael Waskom, provides a high-level, dataset-oriented interface: one passes a DataFrame and names columns, and Seaborn handles grouping, colour mapping, statistical estimation (for example confidence bands) and legends. **Plotly** produces interactive HTML figures with zooming, panning and hover tooltips, which are valuable in dashboards and for exploring large scatter plots; its `plotly.express` module offers a similarly concise interface. Other notable tools include Altair (a declarative grammar-of-graphics library based on Vega-Lite), Bokeh (interactive web plots) and business-intelligence platforms such as Tableau and Power BI. Listing 3.22 shows equivalent exploratory plots in Seaborn and Plotly.

Listing 3.22 Exploratory plots with Seaborn (static) and Plotly Express (interactive).

```
import seaborn as sns
import plotly.express as px
import matplotlib.pyplot as plt

df = sns.load_dataset("penguins").dropna()

# Seaborn: pairwise relationships coloured by species, with KDEs on the diagonal
g = sns.pairplot(df, hue="species", vars=["bill_length_mm", "bill_depth_mm",
                                          "flipper_length_mm"], corner=True)
g.savefig("pairplot.png", dpi=150)

# Seaborn: annotated correlation heatmap
corr = df.select_dtypes("number").corr()
plt.figure(figsize=(5, 4))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="RdBu_r", vmin=-1, vmax=1, square=True)
plt.tight_layout(); plt.savefig("corr.png", dpi=150)

```

```
# Plotly: interactive scatter with hover information, saved as standalone HTML
fig = px.scatter(df, x="flipper_length_mm", y="body_mass_g", color="species",
                 symbol="sex", hover_data=["island"], trendline="ols")
fig.write_html("penguins_scatter.html")
```

Key Point | Principles of Honest Visualisation

- Start bar-chart axes at zero; line charts may use a narrower range if it is clearly labelled.
- Use perceptually uniform colour maps (viridis, cividis) for sequential data and diverging maps for signed data; avoid rainbow ('jet') colour maps.
- Show uncertainty (error bars, confidence bands) whenever comparing estimates.
- Do not rely on colour alone; combine with marker shape or direct labels for accessibility.

3.24 Building Machine Learning Models with Scikit-learn

Scikit-learn, first released in 2010 and described by Pedregosa et al. (2011) in the *Journal of Machine Learning Research*, is the standard library for classical machine learning in Python. Its success rests less on any single algorithm than on a consistent, minimal API that makes models interchangeable and composable. Once that API is understood, switching from logistic regression to a gradient-boosted ensemble is a one-line change, and preprocessing, model selection and evaluation can be assembled into a single object that is safe from data leakage.

3.24.1 The Estimator API

Definition 3.7 | Estimator, Transformer and Predictor

An **estimator** is any object that learns from data through a `fit(X, y)` method; its hyperparameters are set in the constructor and learned attributes end in an underscore (e.g. `coef_`).

A **transformer** is an estimator with a `transform(X)` method (e.g. `StandardScaler`).

A **predictor** is an estimator with `predict(X)` and usually `score(X, y)`; classifiers often also expose `predict_proba(X)`.

Table 3.32 summarises the core methods. Two design rules matter in practice. First, `fit` is called *only on training data*; `transform` and `predict` are then applied to validation or test data using parameters learned during fitting. Second, all estimators expose `get_`

`params` and `set_params`, which is what allows generic tools such as `GridSearchCV` to tune any model, including nested parameters addressed with the double-underscore syntax `step__param`.

Table 3.32 Core methods of the scikit-learn estimator interface.

Method	Available on	Purpose
`fit(X, y=None)`	All estimators	Learn parameters from training data; returns self
`transform(X)`	Transformers	Apply learned transformation (scaling, encoding, PCA)
`fit_transform(X, y)`	Transformers	Fit then transform, sometimes more efficiently
`predict(X)`	Predictors	Class labels or regression values
`predict_proba(X)`	Most classifiers	Class-membership probabilities
`decision_function(X)`	Margin-based classifiers	Signed confidence scores (for ROC curves)
`score(X, y)`	Predictors	Default metric: accuracy or R^2
`get_params` / `set_params`	All estimators	Inspect and modify hyperparameters

3.24.2 Pipelines and ColumnTransformer

Real tabular data mixes numeric and categorical columns, contains missing values and requires different preprocessing per column type. A `ColumnTransformer` applies a separate transformer to each group of columns and concatenates the results; a `Pipeline` chains transformers with a final estimator. The combined object behaves like a single estimator, as Figure 3.29 shows. The central benefit is **leakage prevention**: when the pipeline is cross-validated, imputation means, scaling statistics and category vocabularies are re-estimated on each training fold only, so no information from the held-out fold contaminates preprocessing.

Hyperparameters are tuned by cross-validation. For a candidate setting λ, k-fold cross-validation estimates generalisation performance as the average validation score in Equation (3.49), and grid search selects the λ that maximises it. Algorithm 3.12 formalises the procedure implemented by `GridSearchCV`. Model selection itself is discussed in depth in the Part A section on model selection and hyperparameter optimisation; here the focus is on the mechanics.

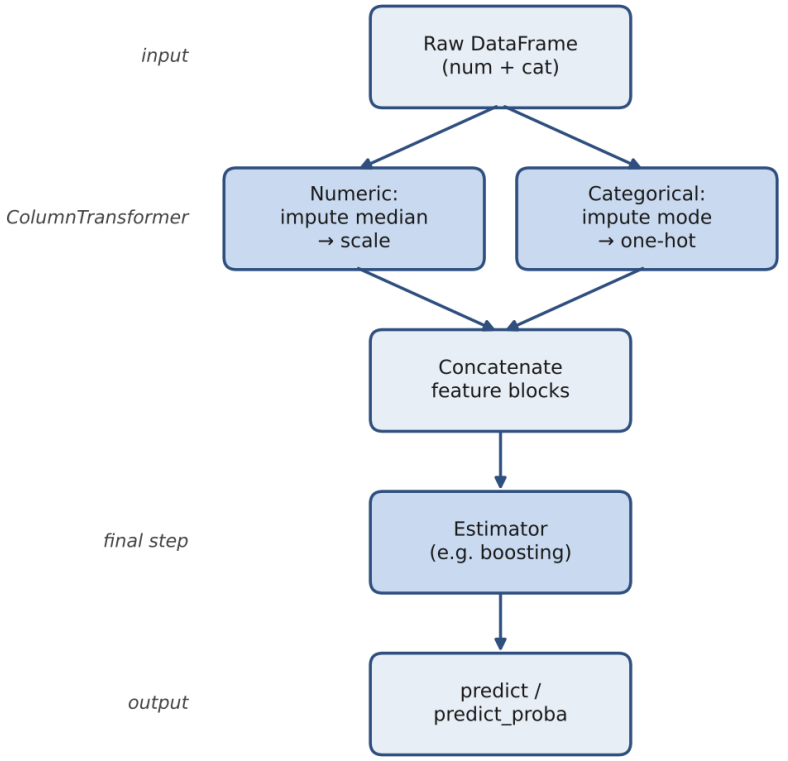


Figure 3.29 A scikit-learn Pipeline wrapping a ColumnTransformer. Numeric and categorical columns follow separate preprocessing branches, are concatenated, and feed a single estimator; the whole object is fitted and tuned as one unit.

$$\mathrm{CV}(\lambda) = \tfrac{1}{k}\sum_{j=1}^{k} S\left(f_{\lambda}^{(-j)},\ D_j\right)$$
$$\lambda^{*} = \arg\max_{\lambda \in \Lambda}\ \mathrm{CV}(\lambda) \tag{3.49}$$

where $D_1, \ldots, D_k$ are the folds, f_λ^(−j) is the pipeline fitted with hyperparameters λ on all folds except j, S is a scoring function (e.g. ROC-AUC) and Λ is the search grid.

Algorithm 3.12 Grid Search with k-Fold Cross-Validation

Input: Training data D; pipeline f with hyperparameter grid Λ; folds k; score S
Output: Best hyperparameters λ, *refitted model f*

1 split D into k stratified folds $D_1, \ldots, D_k$

```
    for each λ in Λ do
        for j = 1 to k do
            fit f_λ (all preprocessing + model) on D \ D_j
            s_j ← S(f_λ, D_j)
        CV(λ) ← mean(s_1, ..., s_k)
λ* ← argmax_λ CV(λ)
f* ← fit f_λ* on all of D
return λ*, f*
```

3.24.3 A Complete Scikit-learn Workflow

Listing 3.23 brings these ideas together for a binary classification problem on a mixed-type table: a stratified train–test split, a pipeline with a column transformer, a grid search over both preprocessing and model hyperparameters, and a final evaluation on the untouched test set.

Listing 3.23 A leak-free scikit-learn pipeline with ColumnTransformer and GridSearchCV.

```python
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV, StratifiedKFold
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, roc_auc_score

df = pd.read_csv("adult.csv")                                  # census income data
y = (df.pop("income") == ">50K").astype(int)
num = df.select_dtypes("number").columns.tolist()
cat = df.select_dtypes("object").columns.tolist()
X_tr, X_te, y_tr, y_te = train_test_split(df, y, test_size=0.2, stratify=y,
    random_state=0)

pre = ColumnTransformer([
    ("num", Pipeline([("imp", SimpleImputer(strategy="median")),
                      ("sc", StandardScaler())]), num),
    ("cat", Pipeline([("imp", SimpleImputer(strategy="most_frequent")),
                      ("ohe", OneHotEncoder(handle_unknown="ignore",
                       min_frequency=20))]), cat)])
pipe = Pipeline([("pre", pre), ("clf", LogisticRegression(max_iter=2000))])

grid = [{"clf": [LogisticRegression(max_iter=2000)], "clf__C": [0.01, 0.1, 1, 10]},
        {"clf": [HistGradientBoostingClassifier(random_state=0)],
         "pre__cat__ohe__min_frequency": [5, 20],
         "clf__learning_rate": [0.05, 0.1], "clf__max_leaf_nodes": [15, 31]}]
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)
search = GridSearchCV(pipe, grid, cv=cv, scoring="roc_auc", n_jobs=-1).fit(X_tr, y_tr)

```

```
print("best:", search.best_params_, "CV AUC = %.3f" % search.best_score_)
proba = search.predict_proba(X_te)[:, 1]
print("test AUC = %.3f" % roc_auc_score(y_te, proba))
print(classification_report(y_te, search.predict(X_te), digits=3))
```

Note that the grid in Listing 3.23 is a *list* of dictionaries, which allows the final step itself to be treated as a hyperparameter: logistic regression and gradient boosting are compared within the same search, each with its own sub-grid. When the grid becomes large, `RandomizedSearchCV`, `HalvingGridSearchCV` or Bayesian optimisation libraries such as Optuna are more efficient. After fitting, the best pipeline is persisted with `joblib`, as in Listing 3.24, so that exactly the same preprocessing is applied in production.

Listing 3.24 Inspecting, saving and reloading a fitted pipeline.

```
import joblib, sklearn
best = search.best_estimator_

# Feature names after the ColumnTransformer (useful for importance plots)
names = best.named_steps["pre"].get_feature_names_out()
print(len(names), names[:5])

# Cross-validation results as a sortable table
cv_table = (pd.DataFrame(search.cv_results_)
              .sort_values("rank_test_score")[["params", "mean_test_score",
              ↪  "std_test_score"]])
print(cv_table.head())

joblib.dump({"model": best, "sklearn_version": sklearn.__version__}, "model.joblib")
bundle = joblib.load("model.joblib")
assert bundle["sklearn_version"] == sklearn.__version__     # pickles are
↪  version-sensitive
print(bundle["model"].predict_proba(X_te.head(3)))
```

Note | Common Scikit-learn Mistakes

- Fitting a scaler or imputer on the full dataset before splitting (data leakage).
- Reporting the best cross-validation score as the final estimate; it is optimistically biased after tuning — use a held-out test set or nested cross-validation.
- Forgetting `stratify=y` for imbalanced classification, producing folds with very few positives.
- Using accuracy on imbalanced data; prefer ROC-AUC, PR-AUC, F1 or balanced accuracy.

3.25 Machine Learning Workflows in KNIME

Not every practitioner writes code. Analysts in business, healthcare and manufacturing often build models in visual, low-code environments, and even experienced programmers use such tools for rapid prototyping and for documenting a process in a form that non-programmers can audit. The **KNIME Analytics Platform** (the Konstanz Information Miner, developed at the University of Konstanz and first released in 2006) is a free, open-source example. Other tools in the same family include RapidMiner (now Altair AI Studio), Orange and Weka's KnowledgeFlow, as well as cloud services such as Azure Machine Learning designer. The concepts transfer directly: once you can read a KNIME workflow, you can read any of them.

3.25.1 Nodes, Ports and Workflows

> **D Definition 3.8 | KNIME Workflow**
>
> A **workflow** is a directed acyclic graph of **nodes**, each performing one operation (read, transform, learn, predict, evaluate, visualise). Nodes are connected through typed **ports**: data ports (triangles) carry tables, and model ports (squares) carry trained models. Each node has a traffic-light status: red (not configured), amber (configured, not executed) and green (executed).

Because each node caches its output table, the analyst can inspect the data after every step, re-execute only the parts of the graph that changed, and branch the flow to compare alternatives. Groups of nodes can be collapsed into *metanodes* or reusable *components*, and loops (for example *X-Partitioner*/*X-Aggregator* for cross-validation, or parameter-optimisation loops) wrap sub-graphs. KNIME also integrates Python and R scripting nodes, deep learning extensions and database connectors, so a visual workflow can call custom code when needed.

3.25.2 A Classification Workflow

Figure 3.30 shows the canonical supervised learning workflow in KNIME, and Table 3.33 describes each node and its key settings. The Partitioning node has two output ports: the first feeds the training rows to the Learner node, and the second sends the held-out rows directly to the Predictor, which also receives the trained model from the Learner's model port. The Scorer then compares predicted and actual classes.

3.25.3 Building the Workflow Step by Step

The following procedure builds the workflow of Figure 3.30 using a public dataset such as the UCI heart disease or bank marketing data.

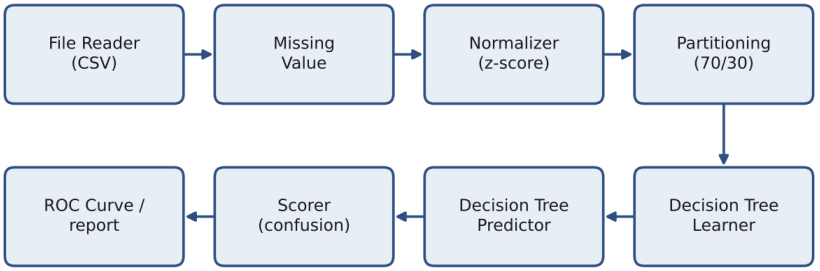


Figure 3.30 A node-based classification workflow as built in KNIME. Data flows left to right and wraps onto the second row; the Learner passes a trained model to the Predictor, which scores the held-out partition.

Table 3.33 Core KNIME nodes for a supervised classification workflow.

Node	Purpose	Key configuration	Code equivalent
File Reader / CSV Reader	Load a table from disk or URL	Delimiter, header row, column types	`pd.read_csv`
Missing Value	Impute or remove missing cells	Per-type strategy: mean, median, most frequent, remove row	`SimpleImputer`
Normalizer	Rescale numeric columns	Min–max, z-score or decimal scaling; outputs a reusable model	`StandardScaler`
Partitioning	Split into training and test sets	Relative %, stratified sampling, random seed	`train_test_split`
Learner (e.g. Decision Tree, Random Forest, Logistic Regression)	Train a model on the training partition	Target column, quality measure, pruning, depth	`model.fit`
Predictor	Apply the trained model to new rows	Append class probabilities	`model.predict`
Scorer	Compute confusion matrix and accuracy statistics	Actual vs predicted column	`confusion_matrix`
ROC Curve / Binary Classification Inspector	Threshold-free evaluation	Positive class, probability column	`roc_curve`

1. **Create a workflow.** In the KNIME Explorer choose *New KNIME Workflow* and give it a name.
2. **Read data.** Drag a *CSV Reader* node from the Node Repository onto the canvas, double-click to configure the file path, and execute it (F7). Right-click → *File Table* to inspect the output.
3. **Handle missing values.** Connect a *Missing Value* node; on the *Default* tab choose *Median* for numbers and *Most Frequent Value* for strings.
4. **Convert the target.** If the class column is numeric, add a *Number To String* node so that learners treat it as categorical.
5. **Normalise.** Add a *Normalizer* node with z-score normalisation on numeric columns. Its model output port can be connected to a *Normalizer* (*Apply*) node so that test data reuse training statistics — the visual equivalent of fitting the scaler on training data only.
6. **Partition.** Add *Partitioning*, choose 70 % relative size, *Stratified sampling* on the class column and a fixed random seed.
7. **Learn and predict.** Connect the top output to a *Decision Tree Learner* (or *Random Forest Learner*) and the bottom output plus the model port to the matching *Predictor*; tick *Append class probabilities*.
8. **Evaluate.** Connect the Predictor to a *Scorer* (choose actual and predicted columns) and to a *ROC Curve* node; execute and open the views.
9. **Iterate.** Wrap the learner–predictor pair in *X-Partitioner*/*X-Aggregator* nodes for 10-fold cross-validation, or use *Parameter Optimization Loop* nodes to tune hyperparameters.
10. **Deploy.** Export the model as PMML, or publish the workflow to KNIME Business Hub as a REST service.

> **N Note | Leakage in Visual Workflows**
>
> Visual tools make leakage easy to overlook: placing the Normalizer *before* Partitioning, as in the simplified Figure 3.30, computes scaling statistics on all rows. For tree models this is harmless because trees are invariant to monotone rescaling, but for distance- or gradient-based learners the Normalizer should be placed after Partitioning, with its model applied to the test branch through *Normalizer* (*Apply*).

> **T Practice Tip | When to Choose a Visual Tool**
>
> - Teaching, stakeholder workshops and audits, where the graph itself is documentation.
> - Data-blending tasks that combine databases, spreadsheets and web services.

- Teams with mixed programming skill; scripting nodes let coders extend the workflow.
- For large-scale custom deep learning, research code and fine-grained control, code-first frameworks remain preferable.

3.26 Building Neural Networks with TensorFlow and Keras

TensorFlow, released by Google in November 2015, is an end-to-end platform for numerical computing with automatic differentiation and hardware acceleration. **Keras**, created by François Chollet in 2015 as a user-friendly front end to several back ends, became TensorFlow's official high-level API (`tf.keras`) in TensorFlow 2.0 (2019). Keras 3, released in 2023, is again multi-back-end and can run on TensorFlow, JAX or PyTorch. Keras exposes three ways of defining models — the Sequential API, the Functional API and model subclassing — and a high-level training loop (`compile`, `fit`, `evaluate`, `predict`) that handles batching, metrics and callbacks. The architectures themselves (dense, convolutional and recurrent networks) are treated in Chapter 2; here we concentrate on how to build and train them.

3.26.1 The Sequential API

The **Sequential API** suits models that are a linear stack of layers with one input and one output. Listing 3.25 trains a small multilayer perceptron on Fashion-MNIST. The `compile` call fixes three things: the optimiser, the loss and the metrics. For integer class labels with a softmax output the appropriate loss is sparse categorical cross-entropy, given in Equation (3.50).

$$\mathcal{L} = -\frac{1}{B}\sum_{i=1}^{B}\log\frac{\exp(z_{i,y_i})}{\sum_{c=1}^{C}\exp(z_{i,c})} \tag{3.50}$$

where B is the batch size, C the number of classes, $z_{i,c}$ the logit for class c of example i, and y_i its integer label.

Listing 3.25 A Sequential multilayer perceptron for Fashion-MNIST in Keras.

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

(x_train, y_train), (x_test, y_test) = keras.datasets.fashion_mnist.load_data()
x_train, x_test = x_train / 255.0, x_test / 255.0            # scale pixels to [0, 1]
```

```

model = keras.Sequential([
    keras.Input(shape=(28, 28)),
    layers.Flatten(),
    layers.Dense(256, activation="relu"),
    layers.Dropout(0.3),
    layers.Dense(128, activation="relu"),
    layers.Dense(10)                                    # logits; softmax inside
    ↪ the loss
])
model.compile(optimizer=keras.optimizers.Adam(1e-3),
              loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=["accuracy"])
model.summary()

history = model.fit(x_train, y_train, validation_split=0.1,
                    epochs=20, batch_size=128, verbose=2)
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
print(f"test accuracy: {test_acc:.4f}")
```

3.26.2 The Functional API

The **Functional API** treats layers as functions applied to symbolic tensors, so any directed acyclic graph of layers can be expressed: multiple inputs (for example an image and tabular metadata), multiple outputs (a classification head and a regression head), shared layers and residual connections. Figure 3.31 depicts a two-input, two-output model and Listing 3.26 implements it.

Listing 3.26 A two-input, two-output network with the Functional API.

```
from tensorflow import keras
from tensorflow.keras import layers

img_in = keras.Input(shape=(64, 64, 3), name="image")
tab_in = keras.Input(shape=(12,), name="tabular")

x = layers.Conv2D(32, 3, activation="relu")(img_in)
x = layers.MaxPooling2D()(x)
x = layers.Conv2D(64, 3, activation="relu")(x)
x = layers.GlobalAveragePooling2D()(x)

t = layers.Dense(32, activation="relu")(tab_in)
h = layers.Concatenate()([x, t])
h = layers.Dropout(0.3)(layers.Dense(64, activation="relu")(h))

cls_out = layers.Dense(5, activation="softmax", name="label")(h)
reg_out = layers.Dense(1, name="score")(h)

model = keras.Model(inputs=[img_in, tab_in], outputs=[cls_out, reg_out])
model.compile(optimizer="adam",
```

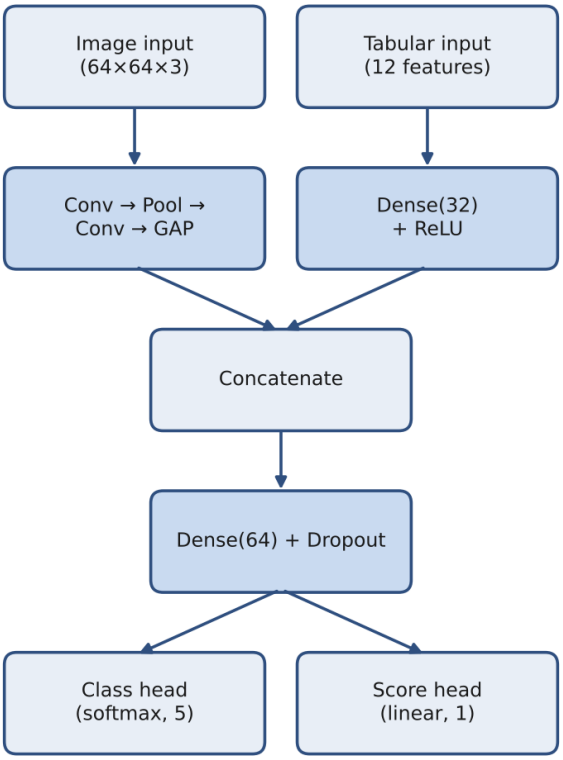


Figure 3.31 A multi-input, multi-output model expressible with the Keras Functional API: an image branch and a tabular branch are concatenated, then feed a classification head and a regression head.

```
                loss={"label": "sparse_categorical_crossentropy", "score": "mse"},
                loss_weights={"label": 1.0, "score": 0.2},
                metrics={"label": ["accuracy"], "score": ["mae"]})
# model.fit({"image": X_img, "tabular": X_tab}, {"label": y_cls, "score": y_reg},
    epochs=10)
```

For architectures with dynamic control flow, **model subclassing** (inheriting from `keras.Model` and writing a `call` method) offers full flexibility at the cost of losing some static analysis such as `model.summary()` before the model is built. As a rule of thumb, use Sequential when possible, Functional when the graph is non-linear, and subclassing only when necessary.

3.26.3 Callbacks and Training Control

Callbacks are objects whose methods are invoked at defined points of training (start and end of each epoch or batch). They implement early stopping, checkpointing, learning-rate schedules and logging without modifying the training loop. Table 3.34 lists the most useful built-in callbacks and Listing 3.27 combines them with an efficient `tf.data` input pipeline.

Table 3.34 Frequently used Keras callbacks.

Callback	What it does	Typical setting
`EarlyStopping`	Stops when a monitored metric stops improving	`monitor='val_loss', patience=5, restore_best_weights=True`
`ModelCheckpoint`	Saves the model or weights during training	`save_best_only=True`, `.keras` format
`ReduceLROnPlateau`	Divides the learning rate when progress stalls	`factor=0.5, patience=3`
`LearningRateSchedul`	Sets the learning rate by epoch from a function	Cosine or step decay
`TensorBoard`	Writes logs for interactive dashboards	`log_dir='runs/exp1'`
`CSVLogger`	Appends per-epoch metrics to a CSV file	Lightweight experiment record

Listing 3.27 Training with a tf.data pipeline, callbacks and saving the best model.

```
import tensorflow as tf
from tensorflow import keras

AUTOTUNE = tf.data.AUTOTUNE
train_ds = (tf.data.Dataset.from_tensor_slices((x_train, y_train))
            .shuffle(10_000, seed=0).batch(128).prefetch(AUTOTUNE))
val_ds = tf.data.Dataset.from_tensor_slices((x_val,
    y_val)).batch(256).prefetch(AUTOTUNE)

callbacks = [
    keras.callbacks.EarlyStopping(monitor="val_loss", patience=5,
        restore_best_weights=True),
    keras.callbacks.ModelCheckpoint("best.keras", monitor="val_accuracy",
        save_best_only=True),
    keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=2,
        min_lr=1e-5),
    keras.callbacks.TensorBoard(log_dir="runs/mlp_baseline"),
    keras.callbacks.CSVLogger("history.csv"),
]
history = model.fit(train_ds, validation_data=val_ds, epochs=100, callbacks=callbacks)

```

```
best = keras.models.load_model("best.keras")
best.export("saved_model/fashion_mlp")   # TF SavedModel for TF Serving / TFLite
    ↪ conversion
```

Key Point | Keras in Three Sentences

Define the graph (Sequential, Functional or subclass); `compile` it with an optimiser, loss and metrics; then `fit` it on a `tf.data` pipeline with callbacks for early stopping and checkpointing.

Use `from_logits=True` and omit the final softmax for numerical stability.

Save in the native `.keras` format for training and export a SavedModel for serving.

3.27 Building Neural Networks with PyTorch

PyTorch, released by Facebook AI Research (now Meta AI) in 2016 and described by Paszke et al. at NeurIPS 2019, has become the dominant framework in machine learning research and is widely used in industry. Its defining feature is **define-by-run** (eager) execution: the computational graph is built on the fly as ordinary Python code runs, so models can be debugged with standard tools and can use arbitrary Python control flow. PyTorch 2.0 (2023) added `torch.compile`, which captures and optimises graphs for speed while retaining the eager programming model. Compared with Keras, PyTorch exposes the training loop explicitly, which costs a few more lines but gives complete control.

3.27.1 Tensors and Automatic Differentiation

A PyTorch **tensor** is an n-dimensional array, similar to a NumPy array, that can live on a CPU or GPU and can record the operations applied to it. If a tensor is created with `requires_grad=True`, every operation involving it adds a node to a dynamic graph; calling `.backward()` on a scalar result traverses the graph in reverse and accumulates gradients into the `.grad` attribute of each leaf tensor. This is reverse-mode automatic differentiation, which computes the gradient of a scalar loss with respect to millions of parameters at a cost of roughly one additional forward pass. Equation (3.51) shows the chain rule that autograd applies mechanically, and Listing 3.28 verifies it on a tiny example.

$$L = \ell(f(g(\mathbf{w}))) \qquad \frac{\partial L}{\partial \mathbf{w}} = \frac{\partial \ell}{\partial f} \cdot \frac{\partial f}{\partial g} \cdot \frac{\partial g}{\partial \mathbf{w}} \tag{3.51}$$

where g, f and ℓ are successive differentiable operations recorded in the computational graph; autograd multiplies the local derivatives from the output back to the parameters w.

Listing 3.28 Tensors, devices and automatic differentiation in PyTorch.

```python
import torch

device = "cuda" if torch.cuda.is_available() else "cpu"
x = torch.tensor([[1.0, 2.0], [3.0, 4.0]], device=device)
w = torch.randn(2, 1, device=device, requires_grad=True)
b = torch.zeros(1, device=device, requires_grad=True)
y = torch.tensor([[1.0], [0.0]], device=device)

logits = x @ w + b                          # graph is recorded as code executes
loss = torch.nn.functional.binary_cross_entropy_with_logits(logits, y)
loss.backward()                             # reverse-mode autodiff

# Analytic check: dL/dw = X^T (sigmoid(z) - y) / n
manual = x.T @ (torch.sigmoid(logits) - y) / len(x)
print(torch.allclose(w.grad, manual.detach()))   # True

with torch.no_grad():                       # disable tracking for inference / updates
    w -= 0.1 * w.grad
w.grad.zero_()                              # gradients accumulate unless cleared
```

3.27.2 Defining Models with nn.Module

Models are classes that inherit from `torch.nn.Module`. Layers assigned as attributes in `__init__` are registered automatically, so `model.parameters()` returns all trainable weights and `model.to(device)` moves them together. The `forward` method defines the computation. Listing 3.29 defines a small convolutional network; Chapter 2 explains the convolution and pooling operations it uses.

Listing 3.29 A convolutional classifier defined as an nn.Module.

```python
import torch
from torch import nn

class SmallCNN(nn.Module):
    def __init__(self, n_classes: int = 10):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1), nn.BatchNorm2d(32), nn.ReLU(),
            nn.MaxPool2d(2),                                    # 32x32 -> 16x16
            nn.Conv2d(32, 64, 3, padding=1), nn.BatchNorm2d(64), nn.ReLU(),
            nn.MaxPool2d(2),                                    # 16x16 -> 8x8
```

```
            nn.Conv2d(64, 128, 3, padding=1), nn.ReLU(),
            nn.AdaptiveAvgPool2d(1))                          # global average pooling
        self.head = nn.Sequential(nn.Flatten(), nn.Dropout(0.3), nn.Linear(128,
        ↪  n_classes))

    def forward(self, x):
        return self.head(self.features(x))                   # returns logits

model = SmallCNN()
print(sum(p.numel() for p in model.parameters() if p.requires_grad), "trainable
↪  parameters")
print(model(torch.randn(4, 3, 32, 32)).shape)                 # torch.Size([4, 10])
```

3.27.3 Datasets and DataLoaders

PyTorch separates *what* the data are from *how* they are batched. A `Dataset` implements `__len__` and `__getitem__`, returning one example (often after a transform); a `DataLoader` wraps a dataset to provide shuffling, batching, parallel loading with worker processes and pinned memory for fast host-to-GPU transfer. Listing 3.30 shows a custom dataset for images listed in a CSV file.

Listing 3.30 A custom Dataset and DataLoader for labelled image files.

```
import pandas as pd
from PIL import Image
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms

class CsvImageDataset(Dataset):
    """Rows of a CSV: filepath,label (integer)."""
    def __init__(self, csv_path, transform=None):
        self.df = pd.read_csv(csv_path)
        self.transform = transform

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        row = self.df.iloc[idx]
        img = Image.open(row["filepath"]).convert("RGB")
        if self.transform:
            img = self.transform(img)
        return img, int(row["label"])

tfm = transforms.Compose([transforms.Resize((32, 32)), transforms.ToTensor(),
                          transforms.Normalize([0.5] * 3, [0.5] * 3)])
train_dl = DataLoader(CsvImageDataset("train.csv", tfm), batch_size=64, shuffle=True,
                      num_workers=4, pin_memory=True, drop_last=True)
val_dl = DataLoader(CsvImageDataset("val.csv", tfm), batch_size=256, num_workers=4)
```

3.27.4 The Training Loop

Every PyTorch training loop follows the same five-step rhythm per mini-batch, formalised in Algorithm 3.13: move the batch to the device, run the forward pass, compute the loss, back-propagate, and take an optimiser step after clearing old gradients. The model must be switched between `train()` and `eval()` modes because dropout and batch normalisation behave differently in each. Listing 3.31 implements the loop with mixed-precision training, which uses 16-bit floating point where safe to roughly halve memory use and speed up modern GPUs.

Algorithm 3.13 Mini-batch Training Loop in PyTorch

Input: Model f_θ; DataLoaders for training and validation; loss ℓ; optimiser; epochs E
Output: Trained parameters θ (best checkpoint by validation loss)

```
best ← ∞
for epoch = 1 to E do
    model.train()
    for each batch (x, y) in training loader do
        move x, y to device
        optimiser.zero_grad()
        ŷ ← f_θ(x); L ← ℓ(ŷ, y)
        L.backward()                          ▷ compute ∇θ L
        optimiser.step()                      ▷ update θ
    model.eval(); with gradients disabled compute validation loss v
    if v < best then best ← v; save θ
    update learning-rate scheduler
return θ from best checkpoint
```

Listing 3.31 A complete training and validation loop with mixed precision and checkpointing.

```python
import torch
from torch import nn

device = "cuda" if torch.cuda.is_available() else "cpu"
model = SmallCNN(n_classes=10).to(device)
criterion = nn.CrossEntropyLoss(label_smoothing=0.1)
optimizer = torch.optim.AdamW(model.parameters(), lr=3e-3, weight_decay=1e-4)
scheduler = torch.optim.lr_scheduler.OneCycleLR(optimizer, max_lr=3e-3,
                                                total_steps=30 * len(train_dl))
scaler = torch.cuda.amp.GradScaler(enabled=(device == "cuda"))
best_val = float("inf")

for epoch in range(30):
    model.train()
    for x, y in train_dl:
        x, y = x.to(device, non_blocking=True), y.to(device, non_blocking=True)
        optimizer.zero_grad(set_to_none=True)
        with torch.autocast(device_type=device, enabled=(device == "cuda")):
```

```
            loss = criterion(model(x), y)
        scaler.scale(loss).backward()
        scaler.step(optimizer); scaler.update(); scheduler.step()

    model.eval(); val_loss, correct, n = 0.0, 0, 0
    with torch.no_grad():
        for x, y in val_dl:
            x, y = x.to(device), y.to(device)
            out = model(x)
            val_loss += criterion(out, y).item() * len(y)
            correct += (out.argmax(1) == y).sum().item(); n += len(y)
    val_loss /= n
    print(f"epoch {epoch:2d}  val_loss {val_loss:.4f}  val_acc {correct / n:.4f}")
    if val_loss < best_val:
        best_val = val_loss
        torch.save(model.state_dict(), "best_model.pt")
```

Table 3.35 Comparison of Keras/TensorFlow and PyTorch for everyday work.

Aspect	Keras / TensorFlow	PyTorch
Execution model	Eager by default; `tf.function` traces graphs	Eager; `torch.compile` for graph optimisation
Training loop	Built-in `fit` with callbacks	Explicit loop (or Lightning, Accelerate)
Data pipeline	`tf.data`	`Dataset` + `DataLoader`
Model definition	Sequential, Functional, subclassing	`nn.Module` subclassing, `nn.Sequential`
Deployment	SavedModel, TF Serving, TFLite, TF.js	TorchScript/`torch.export`, ONNX, TorchServe, ExecuTorch
Ecosystem strength	Production and mobile tooling	Research code, Hugging Face models

Table 3.35 summarises the practical differences. In practice the frameworks have converged: both offer eager execution with optional compilation, and Keras 3 can even run on a PyTorch back end. The choice is usually dictated by the ecosystem around a project — the pretrained models, deployment targets and team experience — rather than by capability.

Practice Tip | PyTorch Debugging Checklist

- Overfit a single batch first: training loss should approach zero within a few hundred steps.
- Print tensor shapes at each stage of `forward` when a size mismatch occurs.
- Check that `model.eval()` and `torch.no_grad()` are used for validation.

- Confirm that the loss expects logits (`CrossEntropyLoss`) rather than probabilities.
- If the loss becomes NaN, lower the learning rate, check input normalisation and clip gradients with `nn.utils.clip_grad_norm_`.

3.28 Dataset Selection and Data Annotation

The quality of a model is bounded by the quality of its data. Andrew Ng's advocacy of *data-centric AI* (from around 2021) crystallised a lesson practitioners had long known: for many applied problems, improving labels and coverage yields larger gains than changing the architecture. This section addresses two questions that precede any modelling: which dataset should be used, and how should examples be labelled reliably and economically?

3.28.1 Criteria for Selecting a Dataset

A dataset is fit for purpose when it represents the population and conditions under which the model will operate, carries labels that measure the intended concept, and can be used legally and ethically. Table 3.36 lists criteria to assess before committing to a dataset, whether it is a public benchmark, an internal database or one to be collected. The *datasheets for datasets* proposal of Gebru et al. (2021, *Communications of the ACM*) provides a structured questionnaire covering the same ground and is good practice for documenting any new dataset.

3.28.2 Annotation Process and Tools

When labels must be created, annotation should be treated as an engineering process with its own specification, quality control and iteration. The core artefact is the **annotation guideline**: a document defining each label, with positive and negative examples and rules for ambiguous cases. Figure 3.32 shows the iterative workflow, in which disagreements discovered during quality control feed back into clearer guidelines.

Popular open-source annotation tools include **Label Studio** (images, text, audio, time series), **CVAT** (the Computer Vision Annotation Tool, for bounding boxes, polygons and video tracking), **doccano** (text classification, sequence labelling) and **Prodigy** (a commercial, scriptable tool with built-in active learning). Commercial platforms such as Labelbox, Scale AI and Amazon SageMaker Ground Truth add workforce management. Crowdsourcing through marketplaces such as Amazon Mechanical Turk is cheap but requires redundancy (several annotators per item), gold-standard check

Table 3.36 Criteria for selecting a dataset for a machine learning project.

Criterion	Key question	Warning signs
Relevance	Does it measure the target concept in the deployment context?	Proxy labels; different population
Representativeness	Does it cover the subgroups, devices, seasons and edge cases seen in use?	Single site; demographic skew
Size	Is there enough data per class for the model complexity?	Rare classes with < 50 examples
Label quality	How were labels produced, by whom, with what agreement?	No annotation guidelines; low κ
Class balance	What is the prevalence of each class?	Extreme imbalance not reflected in metrics
Temporal validity	Is the data recent enough; is there drift?	Old data in a fast-changing domain
Leakage risk	Are there duplicates, patient overlap or future information?	Near-duplicate images across splits
Licensing and consent	May it be used for this purpose and redistributed?	Scraped data; unclear licence
Privacy	Does it contain personal or sensitive attributes?	Identifiable records without safeguards
Documentation	Is there a datasheet, data dictionary and provenance record?	Undocumented preprocessing

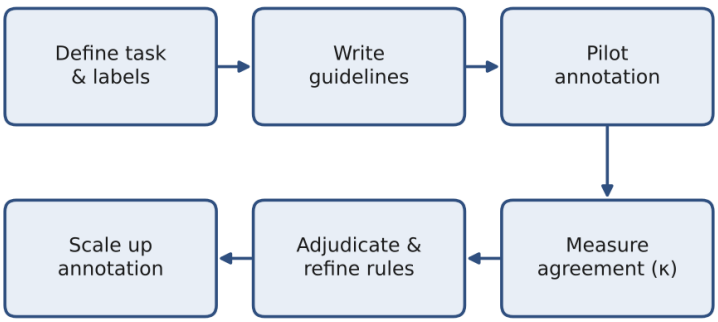


Figure 3.32 The iterative annotation workflow. Disagreement analysis and adjudication feed back into the guidelines until inter-annotator agreement is acceptable.

questions and aggregation methods; expert domains such as radiology require qualified annotators.

3.28.3 Inter-Annotator Agreement and Cohen's Kappa

Agreement between annotators is the main quantitative indicator of label quality and task clarity. Raw percentage agreement is misleading because two annotators would agree sometimes purely by chance, especially when one class dominates. **Cohen's kappa** (Cohen, 1960) corrects for chance agreement, as defined in Equation (3.52).

$$\kappa = \frac{p_o - p_e}{1 - p_e}$$
$$p_o = \frac{1}{N}\sum_{k} n_{kk}, \qquad p_e = \sum_{k} \frac{n_{k\cdot}}{N} \cdot \frac{n_{\cdot k}}{N} \tag{3.52}$$

where N is the number of items, n_{kk} the count of items both annotators labelled k, $n_{k\cdot}$ and $n_{\cdot k}$ the row and column totals of the agreement matrix for annotators A and B, p_o the observed agreement and p_e the agreement expected by chance.

E Worked Example 3.13 | Computing Cohen's Kappa

Two annotators label $N = 100$ product reviews as Positive or Negative. Annotator A says Positive for 60 reviews, B for 55. They agree on 45 Positive and 30 Negative items.

- Observed agreement: $p_o = (45 + 30) / 100 = 0.75$.
- Chance agreement: Positive: $0.60 \times 0.55 = 0.33$; Negative: $0.40 \times 0.45 = 0.18$; $p_e = 0.51$.
- Kappa: $\kappa = (0.75 - 0.51) / (1 - 0.51) = 0.24 / 0.49 \approx 0.49$.

Although the annotators agree on 75 % of items, $\kappa \approx 0.49$ indicates only *moderate* agreement on the Landis–Koch (1977) scale (0.41–0.60), suggesting the guidelines need clarification.

Listing 3.32 computes the same value with scikit-learn. For more than two annotators, **Fleiss' kappa** generalises the idea; **Krippendorff's alpha** additionally handles missing ratings and ordinal or interval labels. For ordinal scales, a *weighted* kappa penalises large disagreements (e.g. 1 versus 5 stars) more than adjacent ones.

Listing 3.32 Cohen's kappa and a confusion matrix between two annotators.

```
import numpy as np
from sklearn.metrics import cohen_kappa_score, confusion_matrix

# Reconstruct the worked example: 45 PP, 15 PN, 10 NP, 30 NN
a = np.array(["P"] * 45 + ["P"] * 15 + ["N"] * 10 + ["N"] * 30)
b = np.array(["P"] * 45 + ["N"] * 15 + ["P"] * 10 + ["N"] * 30)

print(confusion_matrix(a, b, labels=["P", "N"]))
print("kappa = %.3f" % cohen_kappa_score(a, b))                    # 0.490

```

```
# Ordinal labels (1-5 stars): quadratic weights penalise distant disagreements
r1 = [5, 4, 4, 2, 1, 3, 5, 2]; r2 = [4, 4, 5, 2, 2, 3, 5, 1]
print("weighted kappa = %.3f" % cohen_kappa_score(r1, r2, weights="quadratic"))
```

3.28.4 Active Learning

Labelling is expensive, but not all examples are equally informative. **Active learning** lets the model choose which unlabelled examples a human should annotate next, typically those about which it is most uncertain (Settles, 2009, provides a classic survey). In *pool-based uncertainty sampling* (Algorithm 3.14), the uncertainty of an example can be measured by least confidence, the margin between the top two class probabilities, or the predictive entropy of Equation (3.53).

$$x^{*} = \arg\max_{x \in \mathcal{U}} \; -\sum_{c=1}^{C} p_{\theta}(c \mid x) \log p_{\theta}(c \mid x) \tag{3.53}$$

where $\mathcal{U}$ is the pool of unlabelled examples and p_θ(c | x) is the current model's predicted probability of class c.

Algorithm 3.14 Pool-based Active Learning with Uncertainty Sampling

Input: Small labelled set L; unlabelled pool U; model f; batch size b; labelling budget B
Output: Trained model f and enlarged labelled set L

```
train f on L
while |L| < B and U is not empty do
    for each x in U compute uncertainty u(x) (e.g. entropy of f(x))
    S ← the b examples in U with the largest u(x)
    obtain labels for S from human annotators (oracle)
    L ← L ∪ S; U ← U \ S
    retrain f on L
return f, L
```

Pure uncertainty sampling can select redundant or outlying examples; practical systems combine uncertainty with *diversity* (e.g. clustering the candidates, or core-set selection) and use batch selection for efficiency. Related label-efficient strategies include **weak supervision**, in which programmatic labelling functions produce noisy labels that are then denoised (the Snorkel system of Ratner et al., 2017), **semi-supervised learning**, and using large language models as pre-annotators whose output humans verify — a growing practice that still requires agreement checks against human labels.

C Case Study 3.7 | Labelling Medical Images

A hospital team building a pneumonia detector from chest X-rays first wrote a 6-page guideline with example images, then had three radiologists label a pilot set of 300 images. Fleiss' κ was 0.52; disagreement analysis showed confusion between pneumonia and atelectasis. After adding explicit rules and examples for that boundary, κ rose to 0.71.

The team then used uncertainty sampling to choose the next 2,000 images for labelling from a pool of 40,000, reaching the accuracy of a randomly sampled 5,000-image set with less than half the radiologist time.

3.29 Benchmark Datasets and Experimental Design

Research in machine learning advances largely through comparison: a new method is claimed to be better than existing ones on some task. Such claims are only as credible as the experiments behind them. Shared **benchmark datasets** make comparisons possible across laboratories, and sound **experimental design** — appropriate baselines, controlled variables, ablations and repeated runs — makes them meaningful. This section surveys the most influential benchmarks and the design principles that distinguish a convincing experiment from an anecdote.

3.29.1 Influential Benchmark Datasets

Benchmarks have repeatedly shaped the field. MNIST made handwritten digit recognition a standard test bed in the 1990s; ImageNet and its Large Scale Visual Recognition Challenge (ILSVRC, 2010–2017) triggered the deep learning revolution when AlexNet won the 2012 competition by a wide margin; GLUE and SuperGLUE tracked the rise of pretrained language models. Table 3.37 lists widely used datasets across modalities.

N Note | Benchmark Overfitting and Saturation

When an entire community tunes methods on the same test set for years, the reported gains partly reflect adaptation to that test set. Recht et al. (2019) rebuilt fresh test sets for CIFAR-10 and ImageNet and found accuracy drops of roughly 3–15 percentage points, although model *rankings* were largely preserved.

Benchmarks also *saturate*: when models exceed estimated human performance (as on GLUE within about a year of its release), the benchmark stops discriminating and is replaced. Contamination — test items appearing in web-scale pretraining data — is a growing concern for language-model benchmarks.

Table 3.37 Widely used benchmark datasets in machine learning research.

Dataset	Domain / task	Size	Typical metric	Reference
MNIST	Handwritten digits, 10 classes	60k train / 10k test, 28×28 grey	Accuracy	LeCun et al., 1998
Fashion-MNIST	Clothing images, 10 classes	60k / 10k, 28×28 grey	Accuracy	Xiao et al., 2017
CIFAR-10 / CIFAR-100	Natural images, 10 / 100 classes	50k / 10k, 32×32 colour	Accuracy	Krizhevsky, 2009
ImageNet (ILSVRC-2012)	Object classification, 1,000 classes	≈1.28M train / 50k val	Top-1 / top-5 accuracy	Deng et al., 2009; Russakovsky et al., 2015
MS COCO	Detection, segmentation, captioning	≈330k images, 80 object categories	mAP@[.5:.95]	Lin et al., 2014
Pascal VOC	Detection and segmentation, 20 classes	≈11k images (2012)	mAP, mIoU	Everingham et al., 2010
UCI ML Repository	Hundreds of tabular datasets (Iris, Adult, Wine ...)	Varies	Accuracy, AUC, RMSE	Dua & Graff (maintainers)
OpenML-CC18	Curated tabular classification suite	72 datasets	Accuracy, AUC	Bischl et al., 2021
GLUE / SuperGLUE	Natural language understanding suites	9 / 8 tasks	Average task score	Wang et al., 2018; 2019
SQuAD 1.1 / 2.0	Extractive question answering	≈100k / +50k unanswerable	Exact match, F1	Rajpurkar et al., 2016; 2018
IMDb reviews	Binary sentiment classification	25k / 25k	Accuracy	Maas et al., 2011
LibriSpeech	Speech recognition	≈1,000 hours	Word error rate	Panayotov et al., 2015
MMLU	Multitask knowledge for language models	57 subjects, ≈16k questions	Accuracy	Hendrycks et al., 2021

3.29.2 Baselines

A **baseline** is a reference method against which a proposed method is judged. Good experimental practice includes baselines at several levels: a *trivial* baseline (majority class, mean prediction, or persistence for time series) that establishes the floor; a *simple, strong* baseline (logistic regression, gradient-boosted trees for tabular data, a fine-tuned ResNet for images) that a new method must beat to be useful; and the *current state of the art*. Baselines must be given the same care as the proposed method — the same tuning budget, the same data and the same preprocessing. Many published improvements disappear when baselines are tuned properly; Melis, Dyer and Blunsom (2018), for instance, showed that well-tuned LSTM language models outperformed several more recent architectures.

3.29.3 Controls, Ablations and Experimental Protocol

An experiment is a controlled comparison: only the factor under study should change between conditions, while data splits, preprocessing, compute budget and evaluation protocol are held fixed. An **ablation study** removes or replaces one component of a proposed method at a time to measure its contribution. Table 3.38 shows a typical ablation table; it reveals that the new attention module and the augmentation each contribute, while the auxiliary loss adds little and could be dropped for simplicity.

Table 3.38 Example ablation study for an image classifier (mean $\pm$ standard deviation of top-1 accuracy over 5 seeds on a validation set).

Variant	Attention module	Strong augmentation	Auxiliary loss	Accuracy (%)
Full model	✓	✓	✓	84.6 ± 0.3
− attention module		✓	✓	82.1 ± 0.4
− strong augmentation	✓		✓	83.0 ± 0.3
− auxiliary loss	✓	✓		84.4 ± 0.4
Baseline (none)				80.2 ± 0.5

Figure 3.33 summarises a rigorous experimental protocol. Two points deserve emphasis. First, the test set is used *once*, at the end; all design decisions, including early stopping and model selection, use validation data. Second, results should be reported over several random seeds or data splits, since run-to-run variation from initialisation, data order and non-deterministic GPU kernels is often comparable to the improvement being claimed.

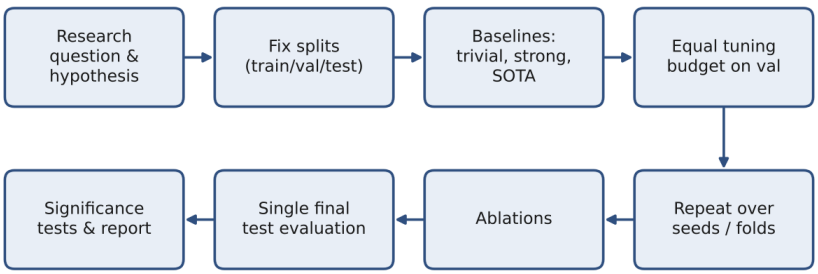


Figure 3.33 A rigorous experimental protocol. The test split is locked away until all design decisions are final, and every method receives the same tuning budget and repeated runs.

Key Point | Checklist for a Convincing Comparison

- State the hypothesis before running experiments.
- Include trivial, strong and state-of-the-art baselines, tuned with equal effort.
- Hold everything constant except the factor under study; run ablations for each component.
- Report mean and variability over at least 3–5 seeds or folds, and test significance (see the section on statistical significance testing).
- Touch the test set only once; report compute cost alongside accuracy.

3.30 Reproducibility and Experiment Tracking

A result is **reproducible** if another researcher, using the same code and data, obtains the same numbers; it is **replicable** if an independent implementation or new data leads to the same conclusion. Both properties are surprisingly hard to achieve in machine learning because results depend on random seeds, library versions, hardware, data versions and hundreds of hyperparameters. Surveys of the literature (for example the NeurIPS 2019 reproducibility programme described by Pineau et al., 2021) prompted major venues to adopt reproducibility checklists. This section covers the three pillars of reproducible practice: controlling randomness, pinning the environment and tracking experiments.

3.30.1 Controlling Randomness

Randomness enters through weight initialisation, data shuffling, dropout, data augmentation, train–test splitting and some GPU kernels. Setting seeds for every random number generator makes runs repeatable on the same hardware and software; Listing 3.33 shows a helper function covering Python, NumPy, PyTorch and TensorFlow. Note that bit-for-bit reproducibility across *different* GPUs or library versions is generally not guaranteed, and deterministic algorithms can be slower. Seeding is not a substitute for reporting variability: one should fix seeds to make each run repeatable, then run *several* seeds to measure variance.

Listing 3.33 A helper that seeds all common random number generators and requests deterministic GPU kernels.

```
import os, random
import numpy as np

def set_seed(seed: int = 42, deterministic: bool = True) -> None:
    os.environ["PYTHONHASHSEED"] = str(seed)
    random.seed(seed)
```

```
    np.random.seed(seed)
    try:
        import torch
        torch.manual_seed(seed); torch.cuda.manual_seed_all(seed)
        if deterministic:
            os.environ["CUBLAS_WORKSPACE_CONFIG"] = ":4096:8"
            torch.backends.cudnn.benchmark = False
            torch.use_deterministic_algorithms(True, warn_only=True)
    except ImportError:
        pass
    try:
        import tensorflow as tf
        tf.keras.utils.set_random_seed(seed)          # seeds Python, NumPy and TF
        if deterministic:
            tf.config.experimental.enable_op_determinism()
    except ImportError:
        pass

# DataLoader workers need their own seeds for reproducible augmentation
def seed_worker(worker_id):
    s = (np.random.get_state()[1][0] + worker_id) % 2**32
    np.random.seed(s); random.seed(s)

set_seed(2026)
```

3.30.2 Pinning the Environment and Versioning Data

Library upgrades can silently change results — a new default hyperparameter, a different numerical routine, a changed random stream. The environment should therefore be recorded precisely: a lock file (`requirements.txt` with exact versions, `conda-lock`, `poetry.lock` or `uv.lock`) and, for full isolation, a Docker image whose tag is recorded with each result. Code is versioned with Git and every experiment should log the commit hash. Data change too, so datasets need versions: tools such as **DVC** (Data Version Control) store large files outside Git while tracking content hashes inside it, and the table formats discussed in Part A (Delta Lake, Apache Iceberg) provide time travel over data in a lakehouse. Listing 3.34 shows the minimal commands.

Listing 3.34 Pinning dependencies, versioning data with DVC and tagging the code state (shell).

```
# Exact package versions of the current environment
pip freeze > requirements.lock.txt
# Rebuild later with: pip install -r requirements.lock.txt

# Track a large dataset with DVC (content-addressed, stored in remote storage)
dvc init
dvc add data/raw/churn.csv              # creates data/raw/churn.csv.dvc (hash) for
↪ Git
dvc remote add -d store s3://my-bucket/dvc
git add data/raw/churn.csv.dvc .gitignore requirements.lock.txt
```

```
git commit -m "Data v1 and pinned environment"
dvc push

# Tag the exact state used for a paper or release
git tag -a paper-v1 -m "Results in Table 3"
docker build -t churn-model:paper-v1 .
```

3.30.3 Experiment Tracking with MLflow and Weights & Biases

Definition 3.9 | **Experiment Tracking**

Experiment tracking is the systematic, automatic recording of every training run's configuration (hyperparameters, code version, data version, environment), metrics over time, artefacts (models, plots, predictions) and metadata, in a searchable store that supports comparison across runs.

Spreadsheets and file names such as `model_final_v3_really_final.pt` do not scale beyond a handful of runs. **MLflow** (open source, released by Databricks in 2018) provides tracking, a model registry and model packaging; it can run locally with a file store or as a shared server. **Weights & Biases** (W&B) is a hosted service with rich dashboards, hyperparameter sweeps and collaborative reports; alternatives include TensorBoard, Neptune, Comet, ClearML and Aim. Listing 3.35 logs a scikit-learn run with MLflow and Listing 3.36 logs a PyTorch training loop with W&B.

Listing 3.35 Tracking a scikit-learn experiment with MLflow.

```
import mlflow, mlflow.sklearn, subprocess
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import roc_auc_score, f1_score

mlflow.set_tracking_uri("file:./mlruns")          # or http://mlflow-server:5000
mlflow.set_experiment("churn-baselines")
params = {"n_estimators": 400, "max_depth": 12, "min_samples_leaf": 5, "random_state": 0}

with mlflow.start_run(run_name="rf-depth12"):
    commit = subprocess.check_output(["git", "rev-parse", "HEAD"], text=True).strip()
    mlflow.set_tags({"git_commit": commit, "data_version": "churn-v1"})
    mlflow.log_params(params)

    model = RandomForestClassifier(**params, n_jobs=-1).fit(X_train, y_train)
    proba = model.predict_proba(X_val)[:, 1]
    mlflow.log_metrics({"val_auc": roc_auc_score(y_val, proba),
                        "val_f1": f1_score(y_val, proba > 0.5)})
    mlflow.log_artifact("reports/feature_importance.png")
    mlflow.sklearn.log_model(model, artifact_path="model",
                             registered_model_name="churn-rf")
# Compare runs in the browser:  mlflow ui  ->  http://127.0.0.1:5000
```

Listing 3.36 Logging a PyTorch training run and its configuration with Weights & Biases.

```python
import wandb

config = dict(arch="SmallCNN", lr=3e-3, batch_size=64, epochs=30, weight_decay=1e-4, seed=2026)
run = wandb.init(project="cifar10-study", config=config, tags=["baseline"])
cfg = wandb.config                                  # single source of truth for hyperparameters

wandb.watch(model, log="gradients", log_freq=200)      # gradient histograms
for epoch in range(cfg.epochs):
    train_loss = train_one_epoch(model, train_dl, optimizer)    # user-defined helpers
    val_loss, val_acc = evaluate(model, val_dl)
    wandb.log({"epoch": epoch, "train/loss": train_loss,
               "val/loss": val_loss, "val/acc": val_acc,
               "lr": optimizer.param_groups[0]["lr"]})

artifact = wandb.Artifact("smallcnn-weights", type="model")
artifact.add_file("best_model.pt")
run.log_artifact(artifact)
run.finish()
```

3.30.4 A Reproducibility Checklist

Table 3.39 condenses the recommendations of venue checklists (such as those of NeurIPS and ICML) into items that can be verified before submitting a paper or handing over a model.

Table 3.39 Reproducibility checklist for machine learning experiments.

Area	Item to record or provide	Done?
Code	Public repository, commit hash, entry-point script, licence	☐
Environment	Lock file or container image; hardware (GPU type, count)	☐
Data	Source, version/hash, preprocessing script, exact splits	☐
Randomness	Seeds used; number of runs; determinism settings	☐
Hyperparameters	Final values, search space, search method and budget	☐
Training	Epochs/steps, optimiser, schedule, early-stopping rule	☐
Evaluation	Metric definitions, test protocol, statistical tests	☐
Results	Mean $\pm$ std or CI over runs; all baselines under same protocol	☐
Compute	Wall-clock time, GPU-hours, estimated energy/CO_2 (see Green AI, Part A)	☐
Artefacts	Trained weights, predictions, logs or tracking-server export	☐

T Practice Tip | Make Reproducibility the Default

- Drive every run from a configuration file (YAML or Hydra) rather than editing code.
- Log automatically: a run that is not tracked did not happen.
- Re-run the full pipeline from raw data in a clean environment before publication.
- Archive the exact test-set predictions; they allow later significance testing without retraining.

3.31 Model Comparison and Statistical Significance Testing

Suppose model A achieves 85.6 % cross-validated accuracy and model B 85.0 %. Is A genuinely better, or is the difference an artefact of the particular data split and random seed? Statistical hypothesis testing provides a principled answer by estimating how likely such a difference would be if the two models were in fact equally good. The statistical foundations — sampling distributions, p-values and hypothesis tests in general — were introduced in Part A; this section applies them to the specific designs that arise when comparing learning algorithms, following the influential analyses of Dietterich (1998), Nadeau and Bengio (2003) and Demšar (2006).

3.31.1 Choosing a Test

The appropriate test depends on what is being compared and how the scores were obtained. Table 3.40 summarises the main options. A central distinction is between comparing two models on *one* dataset, where the samples are folds, seeds or individual test examples, and comparing models across *many* datasets, where each dataset is one sample.

3.31.2 Paired t-Test and the Corrected Resampled t-Test

When both models are evaluated on the same k folds, the scores are *paired*. Let d_j be the difference in score on fold j. The paired t-statistic in Equation (3.54) tests whether the mean difference is zero and is compared with a Student t distribution with $k - 1$ degrees of freedom.

Table 3.40 Statistical tests for comparing machine learning models.

Scenario	Test	Assumptions / notes
Two models, one dataset, k-fold CV scores	Paired t-test	Approx. normal differences; inflated Type I error because folds overlap
Two models, one dataset, (repeated) CV	Corrected resampled t-test (Nadeau–Bengio)	Corrects variance for training-set overlap
Two models, one dataset, 5×2 CV	5×2cv paired t-test (Dietterich)	Low Type I error; uses 5 repetitions of 2-fold CV
Two classifiers, one fixed test set	McNemar's test	Uses only the discordant predictions; one training run each
Two models, many datasets (or folds)	Wilcoxon signed-rank test	Non-parametric; uses ranks of paired differences
$k \geq 3$ models, many datasets	Friedman test + Nemenyi post-hoc	Ranks per dataset; critical-difference diagram
$k \geq 3$ models vs a control	Friedman + Holm/Bonferroni–Dunn	More powerful than Nemenyi when only comparing to one model

$$\bar{d} = \frac{1}{k}\sum_{j=1}^{k} d_j, \qquad s_d^2 = \frac{1}{k-1}\sum_{j=1}^{k}\left(d_j - \bar{d}\right)^2$$
$$t = \frac{\bar{d}}{s_d/\sqrt{k}} \sim t_{k-1} \tag{3.54}$$

where $d_j = a_j - b_j$ is the score difference of models A and B on fold j and s_d is the sample standard deviation of the differences.

The paired t-test assumes the k differences are independent. In cross-validation they are not: any two training sets share a fraction $(k-2)/(k-1)$ of their examples, so the differences are positively correlated and s_d underestimates the true variability. The result is an inflated Type I error — the test declares differences significant too often. Nadeau and Bengio (2003) proposed the **corrected resampled t-test** in Equation (3.55), which inflates the variance by a term that depends on the ratio of test to training set size.

$$t_{\text{corr}} = \frac{\bar{d}}{\sqrt{\left(\frac{1}{J} + \frac{n_{\text{test}}}{n_{\text{train}}}\right)s_d^2}} \sim t_{J-1} \tag{3.55}$$

where J is the total number of train–test resamples (e.g. $J = k$ for one round of k-fold CV, or $J = r \cdot k$ for r repetitions), and n_{test}/n_train is the ratio of test to training sizes ($1/(k-1)$ for k-fold CV).

E Worked Example 3.14 | Paired and Corrected t-Tests

Ten-fold CV accuracies of models A and B produce fold differences $d = (0.007, 0.005, 0.002, 0.009, 0.005, 0.007, 0.003, 0.007, 0.003, 0.008)$.

- Mean difference $\bar{d} = 0.0056$; standard deviation $s_d = 0.00237$.
- Paired t: $t = 0.0056 / (0.00237/\sqrt{10}) = 0.0056 / 0.000748 \approx 7.48$ with 9 d.f.; $p \approx 0.00004$.
- Corrected t: variance factor $1/10 + 1/9 = 0.2111$; denominator $\sqrt{(0.2111 \times 0.00237^2)} = 0.00109$; $t_{corr} \approx 5.15$; $p \approx 0.0006$.

Both tests reject the null at $\alpha = 0.05$, but the corrected statistic is markedly smaller. With a less consistent improvement the two tests often disagree, and the corrected test is the one to trust.

Listing 3.37 Paired t-test, corrected resampled t-test and Wilcoxon test with SciPy.

```python
import numpy as np
from scipy import stats

a = np.array([.842, .851, .838, .860, .847, .855, .843, .852, .849, .858])   # model A, 10 folds
b = np.array([.835, .846, .836, .851, .842, .848, .840, .845, .846, .850])   # model B, same folds

# Standard paired t-test (optimistic for CV)
t, p = stats.ttest_rel(a, b)
print(f"paired t = {t:.2f}, p = {p:.2g}")

# Nadeau-Bengio corrected resampled t-test
def corrected_ttest(a, b, n_train, n_test):
    d = a - b; J = len(d)
    var = (1 / J + n_test / n_train) * d.var(ddof=1)
    t = d.mean() / np.sqrt(var)
    return t, 2 * stats.t.sf(abs(t), df=J - 1)

t_c, p_c = corrected_ttest(a, b, n_train=9, n_test=1)        # ratio 1/9 for 10-fold CV
print(f"corrected t = {t_c:.2f}, p = {p_c:.2g}")

# Non-parametric alternative: Wilcoxon signed-rank test
w, p_w = stats.wilcoxon(a, b)
print(f"Wilcoxon W = {w:.0f}, p = {p_w:.3g}")
```

3.31.3 McNemar's Test

When training is expensive (as for large neural networks) one often has a single train–test split and one trained instance of each model. **McNemar's test** compares the two classifiers on the same test examples using a 2×2 contingency table of correct and incorrect predictions. Only the *discordant* cells matter: b, the number of examples that A misclassifies but B gets right, and c, the number that A gets right but B misclassifies. Under the null hypothesis of equal error rates, b and c should be similar; the statistic

with Edwards' continuity correction is given in Equation (3.56). For small b + c (below about 25), an exact binomial test is used instead.

$$\chi^2 = \frac{(|b-c|-1)^2}{b+c} \sim \chi^2_1 \tag{3.56}$$

where b and c are the off-diagonal counts of the contingency table of the two classifiers' correctness on the same test set.

E Worked Example 3.15 | McNemar's Test

On 2,000 test images, a baseline CNN (A) and a new model (B) give: both correct 1,790; both wrong 153; A wrong and B right b = 38; A right and B wrong c = 19.

- $\chi^2 = (|38 - 19| - 1)^2 \,/\, (38 + 19) = 18^2 \,/\, 57 = 324 \,/\, 57 \approx 5.68$.
- With 1 degree of freedom, $p \approx 0.017 < 0.05$, so the null hypothesis of equal error rates is rejected.
- Accuracies: $A = (1790 + 19)/2000 = 90.45\ \%$; $B = (1790 + 38)/2000 = 91.40\ \%$. The exact binomial version gives $p \approx 0.016$, consistent with the χ^2 approximation.

Listing 3.38 McNemar's test from the predictions of two classifiers.

```python
import numpy as np
from statsmodels.stats.contingency_tables import mcnemar

def mcnemar_from_preds(y_true, pred_a, pred_b, exact=None):
    ca, cb = pred_a == y_true, pred_b == y_true
    table = np.array([[np.sum(ca & cb),  np.sum(ca & ~cb)],
                      [np.sum(~ca & cb), np.sum(~ca & ~cb)]])
    b_plus_c = table[0, 1] + table[1, 0]
    exact = (b_plus_c < 25) if exact is None else exact
    res = mcnemar(table, exact=exact, correction=True)
    return table, res.statistic, res.pvalue

table = np.array([[1790, 19], [38, 153]])          # [[both right, A right/B wrong], [A wrong/B right, both wrong]]
res = mcnemar(table, exact=False, correction=True)
print(f"chi2 = {res.statistic:.2f}, p = {res.pvalue:.3f}")   # chi2 = 5.68, p = 0.017
```

3.31.4 Wilcoxon Signed-Rank Test

The **Wilcoxon signed-rank test** is a non-parametric alternative to the paired t-test that does not assume normally distributed differences and is robust to outliers. The absolute differences $|d_i|$ are ranked (zeros discarded, ties given average ranks), and the ranks of positive and negative differences are summed separately, as in Equation (3.57). Demšar (2006) recommends it as the default for comparing two classifiers across multiple datasets, where differences in scale between datasets make averaging

raw scores questionable. In the worked t-test example, all ten differences are positive, so W = 0 and the exact two-sided p-value is $2/2^{10} \approx 0.002$.

$$W^{+} = \sum_{d_i > 0} \operatorname{rank}(|d_i|), \quad W^{-} = \sum_{d_i < 0} \operatorname{rank}(|d_i|), \quad W = \min\left(W^{+}, W^{-}\right) \tag{3.57}$$

where small values of W indicate that one model is consistently better; the p-value comes from the exact distribution for small N or a normal approximation for large N.

3.31.5 Friedman Test, Nemenyi Post-hoc and Critical-Difference Diagrams

To compare $k \geq 3$ algorithms over N datasets, Demšar (2006) recommends the **Friedman test**. On each dataset the algorithms are ranked (1 = best, average ranks for ties), and R_j denotes the average rank of algorithm j. Under the null hypothesis that all algorithms are equivalent, all average ranks should be close to $(k + 1)/2$. The Friedman statistic and the less conservative Iman–Davenport F-statistic are given in Equation (3.58).

$$\chi_F^2 = \frac{12N}{k(k+1)}\left[\sum_{j=1}^{k} R_j^2 - \frac{k(k+1)^2}{4}\right]$$
$$F_F = \frac{(N-1)\,\chi_F^2}{N(k-1) - \chi_F^2} \sim F_{k-1,\;(k-1)(N-1)} \tag{3.58}$$

where N is the number of datasets, k the number of algorithms and R_j the average rank of algorithm j.

If the null hypothesis is rejected, a post-hoc test identifies which pairs differ. The **Nemenyi test** declares two algorithms significantly different if their average ranks differ by at least the critical difference in Equation (3.59). The results are displayed compactly in a **critical-difference (CD) diagram** such as Figure 3.34, in which algorithms are placed on a rank axis and groups that are *not* significantly different are joined by horizontal bars. Algorithm 3.15 summarises the full procedure.

$$\mathrm{CD} = q_\alpha \sqrt{\frac{k(k+1)}{6N}} \tag{3.59}$$

where q_α is the critical value of the Studentised range statistic divided by $\sqrt{2}$ (for $\alpha = 0.05$: 2.343 for $k = 3$, 2.569 for $k = 4$, 2.728 for $k = 5$).

Algorithm 3.15 Comparing k Algorithms over N Datasets (Friedman + Nemenyi)

Input: Score matrix $S \in \mathbb{R}^{N \times k}$ (rows: datasets, columns: algorithms); significance level α
Output: Average ranks, significance decision and groups of equivalent algorithms

```
for i = 1 to N do
    rank the k scores in row i (1 = best; average ranks for ties)
R_j ← mean rank of algorithm j over the N datasets, for j = 1 ... k
compute χ²_F and F_F; obtain the p-value from F(k−1, (k−1)(N−1))
if p ≥ α then
    return 'no significant differences'
CD ← q_α · √(k(k+1)/(6N))
for each pair (j, l) do
    if |R_j − R_l| ≥ CD then mark j and l as significantly different
draw CD diagram: place R_j on rank axis; join cliques whose rank span < CD
return R, p, significant pairs
```

E Worked Example 3.16 | **Friedman and Nemenyi on 10 Datasets**

Four algorithms — logistic regression (LR), random forest (RF), gradient boosting (GB) and an MLP — are evaluated on $N = 10$ datasets. Their average ranks are $R = (3.85, 2.20, 1.10, 2.85)$.

- Friedman: $\chi^2_F = (12 \cdot 10)/(4 \cdot 5) \times [3.85^2 + 2.20^2 + 1.10^2 + 2.85^2 - 4 \cdot 25/4] = 6 \times (28.995 - 25) \approx 23.97$.
- Iman–Davenport: $F_F = 9 \times 23.97 / (10 \cdot 3 - 23.97) \approx 35.8$ with $(3, 27)$ d.f.; $p < 10^{-8}$. (SciPy's `friedmanchisquare` applies a tie correction and reports 24.2.)
- Nemenyi: $CD = 2.569 \times \sqrt{(4 \cdot 5/60)} = 2.569 \times 0.577 \approx 1.48$.
- GB vs LR: $|1.10 - 3.85| = 2.75 > 1.48 \rightarrow$ significant. GB vs MLP: $1.75 > 1.48 \rightarrow$ significant. GB vs RF: $1.10 < 1.48 \rightarrow$ not significant. RF vs LR: $1.65 > 1.48 \rightarrow$ significant. RF vs MLP: $0.65 \rightarrow$ not significant.

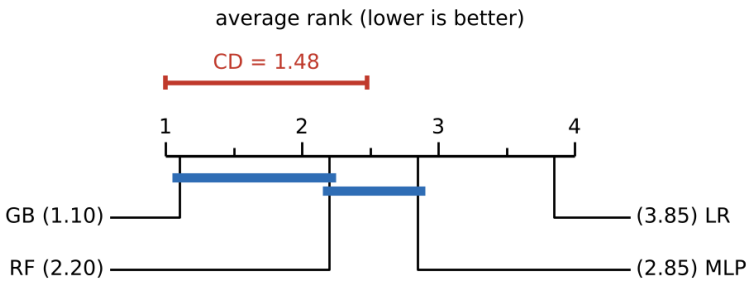


Figure 3.34 Critical-difference diagram for the worked example ($k = 4$, $N = 10$, $CD = 1.48$ at $\alpha = 0.05$). Lower rank is better; algorithms joined by a thick bar are not significantly different.

The CD diagram makes the conclusions visible at a glance: gradient boosting is significantly better than both the MLP and logistic regression, but cannot be distinguished from random forest on this evidence; random forest, in turn, cannot be distinguished from the MLP. Listing 3.39 reproduces the analysis in Python.

Listing 3.39 Friedman test, average ranks and Nemenyi critical difference.

```python
import numpy as np
from scipy import stats

names = ["LR", "RF", "GB", "MLP"]
S = np.array([[.81, .84, .86, .83], [.72, .75, .78, .74], [.90, .91, .93, .92],
              [.65, .70, .69, .66], [.88, .87, .91, .89], [.77, .80, .82, .79],
              [.93, .94, .95, .93], [.69, .73, .74, .70], [.84, .86, .88, .85],
              [.79, .81, .83, .80]])                      # 10 datasets x 4 algorithms (accuracy)
N, k = S.shape

stat, p = stats.friedmanchisquare(*S.T)
ranks = np.array([stats.rankdata(-row) for row in S])    # rank 1 = highest accuracy
R = ranks.mean(axis=0)
print(dict(zip(names, R.round(2))), f"Friedman chi2 = {stat:.2f}, p = {p:.1e}")

q_alpha = {3: 2.343, 4: 2.569, 5: 2.728, 6: 2.850}[k]   # Nemenyi, alpha = 0.05
cd = q_alpha * np.sqrt(k * (k + 1) / (6 * N))
print(f"CD = {cd:.2f}")
for i in range(k):
    for j in range(i + 1, k):
        diff = abs(R[i] - R[j])
        print(f"{names[i]:>3} vs {names[j]:<3} |dR| = {diff:.2f}", "*" if diff >= cd else "")
# Packages such as scikit-posthocs (posthoc_nemenyi_friedman) and autorank draw CD diagrams.
```

Note | Significance Is Not the Whole Story

- With enough test examples, trivially small differences become statistically significant; always report effect sizes and confidence intervals (next section).
- When many comparisons are made, control the family-wise error rate (Holm, Bonferroni) or the false discovery rate (Benjamini–Hochberg).
- Bayesian alternatives (Benavoli et al., 2017) estimate the probability that one model is practically better, using a region of practical equivalence.

3.32 Interpreting Experimental Results

A p-value answers a narrow question: how surprising would the observed difference be if there were no true difference? It does not say how large the difference is, how pre-

cisely it has been estimated, why the model fails where it does, or whether more data would help. Interpreting results well requires four further tools: confidence intervals, effect sizes, error analysis and learning curves, together with a disciplined reporting format.

3.32.1 Confidence Intervals

A **confidence interval** (CI) gives a range of plausible values for a performance measure. For accuracy $\hat{p}$ measured on n independent test examples, the normal-approximation (Wald) interval is given in Equation (3.60); the Wilson score interval is preferable when $\hat{p}$ is near 0 or 1 or n is small. For metrics without a simple formula — F1, ROC-AUC, mean average precision — the **bootstrap** is the standard tool: resample the test set with replacement many times, recompute the metric, and take the 2.5th and 97.5th percentiles (Listing 3.40).

$$\hat{p} \pm z_{1-\alpha/2}\sqrt{\frac{\hat{p}(1-\hat{p})}{n}} \tag{3.60}$$

where $\hat{p}$ is the observed accuracy, n the number of test examples and $z_{1-\alpha/2} = 1.96$ for a 95 % interval.

E Worked Example 3.17 | How Big Must a Test Set Be?

A classifier achieves $\hat{p} = 0.90$ on $n = 500$ test examples. The 95 % CI half-width is $1.96 \times \sqrt{(0.9 \times 0.1 / 500)} = 1.96 \times 0.0134 \approx 0.026$, so the interval is 87.4 %–92.6 %.

- To halve the half-width to 1.3 points, n must quadruple to 2,000.
- Consequently a claimed 1-point improvement measured on 500 examples is well within noise; such comparisons need paired tests (McNemar) or much larger test sets.

Listing 3.40 Percentile bootstrap confidence intervals for any metric, including a paired difference.

```python
import numpy as np
from sklearn.metrics import roc_auc_score

def bootstrap_ci(y, s, metric, n_boot=2000, alpha=0.05, seed=0):
    rng = np.random.default_rng(seed); n = len(y); vals = []
    for _ in range(n_boot):
        idx = rng.integers(0, n, n)
        if len(np.unique(y[idx])) < 2:          # AUC needs both classes
            continue
        vals.append(metric(y[idx], s[idx]))
    lo, hi = np.percentile(vals, [100 * alpha / 2, 100 * (1 - alpha / 2)])
    return metric(y, s), lo, hi

auc, lo, hi = bootstrap_ci(y_test, proba_b, roc_auc_score)
print(f"AUC = {auc:.3f}  (95% CI {lo:.3f}-{hi:.3f})")
```

```

# Paired bootstrap for the difference between two models on the same test set
diff = lambda y, s: roc_auc_score(y, s[:, 1]) - roc_auc_score(y, s[:, 0])
d, lo, hi = bootstrap_ci(y_test, np.c_[proba_a, proba_b], diff)
print(f"Delta AUC = {d:+.3f}  (95% CI {lo:+.3f} to {hi:+.3f})")   # excludes 0 ->
↪  evidence of a difference
```

3.32.2 Effect Sizes

An **effect size** quantifies the magnitude of a difference independently of sample size. In model comparison the most direct effect size is simply the difference in the metric with its CI (e.g. +1.2 AUC points, 95 % CI +0.4 to +2.0). Standardised measures are useful when comparing across studies: Cohen's d for paired differences is defined in Equation (3.61), with conventional thresholds of 0.2 (small), 0.5 (medium) and 0.8 (large). Practical significance must ultimately be judged in domain terms: a 0.5 % reduction in fraud losses may be worth millions, while a 2 % accuracy gain may be irrelevant if it doubles inference cost.

$$d = \frac{\bar{d}}{s_d} \tag{3.61}$$

where $\bar{d}$ and s_d are the mean and standard deviation of the paired score differences.

3.32.3 Error Analysis

Aggregate metrics conceal *where* a model fails. **Error analysis** examines misclassified examples systematically: inspecting the confusion matrix to find confused class pairs; slicing performance by subgroup (age band, device type, image source, sentence length) to reveal hidden weaknesses and fairness problems (see the Responsible AI section in Part A); reading or viewing a random sample of errors and tagging their causes (label noise, ambiguous input, out-of-distribution data, genuine model mistakes); and ranking error categories by frequency to decide what to fix first. Very often this reveals that a sizeable fraction of 'errors' are actually label mistakes, which argues for cleaning the test set before chasing further model improvements.

3.32.4 Learning Curves

A **learning curve** plots training and validation performance against the number of training examples. It diagnoses whether collecting more data is likely to help. Figure 3.35 shows the two characteristic patterns. In the high-bias case, training and validation errors converge quickly to a high plateau: more data will not help, but a more

expressive model or better features will. In the high-variance case, a large gap persists between low training error and higher validation error, and the validation curve is still descending: more data, regularisation or augmentation should help.

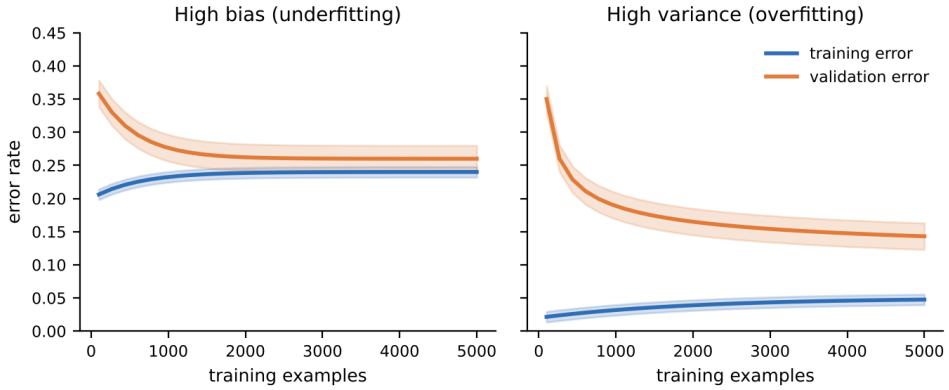


Figure 3.35 Learning curves for a high-bias model (left), whose curves converge to a high error, and a high-variance model (right), whose large gap and still-falling validation error suggest that more data would help. Shaded bands show ± 1 standard deviation over folds.

Listing 3.41 Computing a learning curve with scikit-learn.

```
import numpy as np
from sklearn.model_selection import learning_curve, StratifiedKFold

sizes, train_sc, val_sc = learning_curve(
    pipe, X, y, train_sizes=np.linspace(0.1, 1.0, 8),
    cv=StratifiedKFold(5, shuffle=True, random_state=0),
    scoring="roc_auc", n_jobs=-1, shuffle=True, random_state=0)

for n, tr, va in zip(sizes, train_sc.mean(1), val_sc.mean(1)):
    print(f"n={n:6d}  train AUC={tr:.3f}  val AUC={va:.3f}  gap={tr - va:.3f}")
# Large, persistent gap -> variance (more data / regularisation);
# both low and flat     -> bias (richer model / better features).
```

3.32.5 Reporting Results

Clear reporting lets readers judge a claim quickly. Table 3.41 gives templates for the most common reporting situations, and the box that follows offers a model results paragraph.

T Practice Tip | A Model Results Paragraph

"Gradient boosting achieved the highest test ROC-AUC (0.871, 95 % bootstrap CI 0.852–0.889), an improvement of 0.019 (CI 0.006–0.032) over logistic regression. Across ten repeated

Table 3.41 Templates for reporting experimental results.

Situation	What to report	Example phrasing
Single model, held-out test	Metric, 95 % CI, test-set size	AUC 0.871 (95 % CI 0.852–0.889; n = 2,410)
Two models, one dataset	Both scores, difference with CI, test and p-value	+1.9 points (CI 0.6–3.2); McNemar $\chi^2 = 5.7$, $p = 0.017$
Repeated runs	Mean ± std over seeds, number of seeds	84.6 ± 0.3 % (5 seeds)
Many models, many datasets	Average ranks, Friedman p, CD diagram	Friedman $p < 0.001$; GB best (rank 1.1)
Ablation	Change relative to full model per component	Removing attention: −2.5 points
Efficiency	Parameters, training GPU-hours, latency	25M params, 14 GPU-h, 8 ms/image
Negative result	Effect size and CI showing no meaningful gain	$\Delta = +0.1$ (CI −0.4 to +0.6): no evidence of improvement

10-fold CV runs the corrected resampled t-test confirmed the difference ($t = 3.4$, $p = 0.002$; Cohen's $d = 0.9$). Error analysis showed that 41 % of remaining false negatives were customers with under three months' tenure, suggesting that additional early-engagement features may help."

3.33 Model Deployment Using APIs and Web Applications

A model that lives only in a notebook creates no value. **Deployment** makes predictions available to the people and systems that need them, reliably, at the required speed and with the ability to monitor and update the model over time. This section covers the most common pattern — wrapping a model in a web API — together with quick interactive front ends for demonstrations and containers for portable packaging. The broader MLOps lifecycle (monitoring, drift detection, retraining and feature stores) was discussed in Part A; here we focus on the hands-on mechanics.

3.33.1 Serving Patterns

Three serving patterns cover most applications. In **batch (offline) inference**, a scheduled job scores a whole table — for example every customer each night — and writes predictions to a database; it is simple, cheap and sufficient when decisions are not time-critical. In **online (real-time) inference**, a service answers individual requests within

milliseconds, typically through a REST or gRPC API. In **edge inference**, the model runs on the device (phone, camera, microcontroller) after conversion to a compact format such as TensorFlow Lite, Core ML or ONNX, which removes network latency and keeps data local. Table 3.42 compares tools for each layer of a deployment.

Table 3.42 Tools for deploying machine learning models.

Purpose	Tools	Best for
REST API framework	FastAPI, Flask	Custom Python services; full control over logic
Interactive demo / internal app	Streamlit, Gradio, Dash	Prototypes, stakeholder demos, Hugging Face Spaces
Dedicated model servers	TorchServe, TensorFlow Serving, NVIDIA Triton, BentoML, Ray Serve	High throughput, batching, GPU sharing, model versioning
Portable model formats	ONNX, TorchScript / torch.export, SavedModel, TFLite	Cross-framework and edge deployment
Containers and orchestration	Docker, Kubernetes, KServe	Reproducible, scalable deployment
Managed cloud platforms	AWS SageMaker, Google Vertex AI, Azure ML	Hosted endpoints with autoscaling and monitoring

3.33.2 A REST API with FastAPI

FastAPI (first released in 2018) is a modern Python web framework that uses type hints and the Pydantic library to validate request bodies automatically and to generate interactive OpenAPI documentation at `/docs`. **Flask** is an older, minimal framework that remains popular; the code structure is similar. Listing 3.42 serves the churn pipeline developed in the next section. The model is loaded once at start-up, not per request; inputs are validated against a schema; and the response includes the model version so that every prediction can be traced.

Listing 3.42 Serving a scikit-learn pipeline as a validated REST API with FastAPI (file `app.py`).

```python
from typing import Literal
import joblib, pandas as pd
from fastapi import FastAPI
from pydantic import BaseModel, Field

bundle = joblib.load("model.joblib")                # loaded once at start-up
model, MODEL_VERSION = bundle["model"], bundle.get("version", "1.0.0")
app = FastAPI(title="Churn prediction API", version=MODEL_VERSION)

class Customer(BaseModel):
    tenure: int = Field(ge=0, le=100)
    contract: Literal["Month-to-month", "One year", "Two year"]
```

```
    internet: Literal["DSL", "Fiber optic", "No"]
    payment: str
    tech_support: Literal["Yes", "No"]
    senior: int = Field(ge=0, le=1)
    monthly_charges: float = Field(gt=0)
    total_charges: float = Field(ge=0)
    support_calls: int = Field(ge=0)

class Prediction(BaseModel):
    churn_probability: float
    churn: bool
    model_version: str

@app.get("/health")
def health():
    return {"status": "ok", "model_version": MODEL_VERSION}

@app.post("/predict", response_model=list[Prediction])
def predict(customers: list[Customer], threshold: float = 0.35):
    X = pd.DataFrame([c.model_dump() for c in customers])
    p = model.predict_proba(X)[:, 1]
    return [Prediction(churn_probability=round(float(pi), 4), churn=bool(pi >=
    ↪  threshold),
                       model_version=MODEL_VERSION) for pi in p]
# Run: uvicorn app:app --host 0.0.0.0 --port 8000   (docs at
↪  http://localhost:8000/docs)
```

A client calls the service with an HTTP POST request. Listing 3.43 shows a request from Python and the equivalent command-line call; the same endpoint can be consumed by a web front end, a mobile app or another microservice.

Listing 3.43 Calling the prediction API from Python (the curl equivalent is shown as a comment).

```python
import requests

customer = {"tenure": 3, "contract": "Month-to-month", "internet": "Fiber optic",
            "payment": "Electronic check", "tech_support": "No", "senior": 0,
            "monthly_charges": 89.5, "total_charges": 268.4, "support_calls": 4}

r = requests.post("http://localhost:8000/predict", json=[customer], timeout=5)
r.raise_for_status()
print(r.json())   # [{'churn_probability': 0.83, 'churn': True, 'model_version':
↪  '1.0.0'}]

# curl -X POST localhost:8000/predict -H "Content-Type: application/json" \
#      -d '[{"tenure": 3, "contract": "Month-to-month", ...}]'
```

3.33.3 Interactive Web Apps with Streamlit and Gradio

For demonstrations, internal tools and user testing, a full web front end is unnecessary. **Streamlit** turns a Python script into a web app: widgets such as sliders and select boxes return values directly, and the script reruns top to bottom on each interaction. **Gradio** specialises in wrapping a prediction function with typed input and output components (image, text, audio, label) and is widely used for machine learning demos, including on Hugging Face Spaces. Listing 3.44 shows a Streamlit app for the churn model, and Listing 3.45 a Gradio image-classification demo for the deep learning model of the next-but-one section.

Listing 3.44 A Streamlit front end for the churn model (run with `streamlit run churn_app.py`).

```python
import joblib, pandas as pd
import streamlit as st

@st.cache_resource                       # load the model once per server process
def load_model():
    return joblib.load("model.joblib")["model"]

model = load_model()
st.title("Customer churn risk")
col1, col2 = st.columns(2)
with col1:
    tenure = st.slider("Tenure (months)", 0, 72, 12)
    contract = st.selectbox("Contract", ["Month-to-month", "One year", "Two year"])
    internet = st.selectbox("Internet service", ["DSL", "Fiber optic", "No"])
    tech = st.radio("Tech support", ["Yes", "No"], horizontal=True)
with col2:
    monthly = st.number_input("Monthly charges ($)", 18.0, 120.0, 70.0)
    calls = st.number_input("Support calls (last 90 days)", 0, 20, 1)
```

```
    payment = st.selectbox("Payment", ["Electronic check", "Mailed check", "Bank
    ↪ transfer", "Credit card"])
    senior = int(st.checkbox("Senior citizen"))

row = pd.DataFrame([dict(tenure=tenure, contract=contract, internet=internet,
↪ payment=payment,
                         tech_support=tech, senior=senior, monthly_charges=monthly,
                         total_charges=monthly * tenure, support_calls=calls)])
p = model.predict_proba(row)[0, 1]
st.metric("Churn probability", f"{p:.1%}")
st.progress(float(p))
if p >= 0.35:
    st.warning("High risk: consider a retention offer.")
```

Listing 3.45 A Gradio demo wrapping an image classifier.

```
import gradio as gr
import torch
from PIL import Image

model = torch.jit.load("flower_classifier.pt").eval()       # exported TorchScript
↪ model
classes = ["daisy", "dandelion", "roses", "sunflowers", "tulips"]

def classify(img: Image.Image) -> dict:
    x = eval_transform(img).unsqueeze(0)                    # same preprocessing as
    ↪ validation
    with torch.no_grad():
        probs = torch.softmax(model(x), dim=1)[0]
    return {c: float(p) for c, p in zip(classes, probs)}

demo = gr.Interface(fn=classify, inputs=gr.Image(type="pil"),
                    outputs=gr.Label(num_top_classes=3),
                    title="Flower classifier", examples=["examples/rose.jpg"])
demo.launch()                                               # share=True gives a
↪ temporary public URL
```

3.33.4 Containerising with Docker

> **Definition 3.10** | **Container**
>
> A **container** is a lightweight, isolated process that packages an application together with its runtime, libraries and configuration, built from a declarative recipe (a *Dockerfile*) into an immutable *image*. The same image runs identically on a laptop, a server or a cloud cluster.

Docker eliminates the "works on my machine" problem by freezing the environment discussed in the section on reproducibility. Listing 3.46 packages the FastAPI service. Copying the requirements file before the application code allows Docker to cache the

dependency layer, so rebuilding after a code change takes seconds.

Listing 3.46 Dockerfile for the FastAPI churn service, with build and run commands.

```
# ---- Dockerfile ----
FROM python:3.11-slim
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
WORKDIR /app

COPY requirements.lock.txt .
RUN pip install --no-cache-dir -r requirements.lock.txt   # cached unless requirements
↪ change

COPY app.py model.joblib ./
RUN useradd --create-home appuser
USER appuser                                              # do not run as root
EXPOSE 8000
HEALTHCHECK CMD python -c "import urllib.request;
↪ urllib.request.urlopen('http://localhost:8000/health')"
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000", "--workers", "2"]

# ---- shell ----
# docker build -t churn-api:1.0.0 .
# docker run -d -p 8000:8000 --name churn churn-api:1.0.0
# docker push registry.example.com/churn-api:1.0.0
```

3.33.5 Deployment Architecture

Figure 3.36 shows how these pieces fit together in a typical production deployment. Clients reach the service through an API gateway or load balancer that handles authentication and rate limiting; several identical container replicas run the model; a model registry (such as the MLflow registry from the section on experiment tracking) supplies versioned artefacts; and every request and prediction is logged for monitoring, which feeds drift detection and the retraining pipeline.

Practice Tip | Deployment Checklist

- Package preprocessing *with* the model (a single pipeline) to avoid training–serving skew.
- Validate inputs and return clear errors; never let malformed data reach the model silently.
- Version everything: model, API, container image; return the model version with each prediction.
- Add health checks, latency and error-rate metrics, and log inputs/outputs (respecting privacy).
- Roll out gradually (shadow mode, canary or A/B test) and keep the previous version ready for rollback.

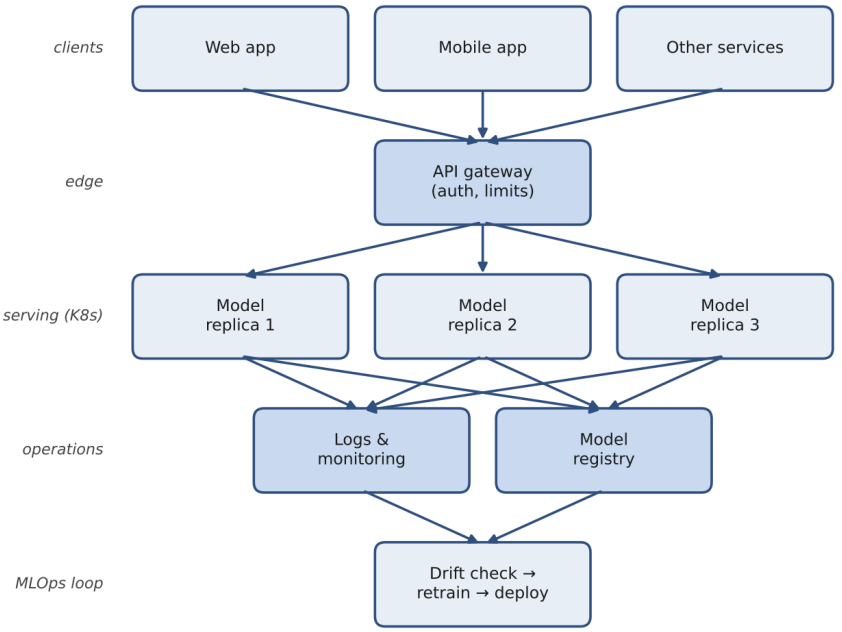


Figure 3.36 A typical online deployment architecture. Containerised model replicas sit behind a gateway, load versioned models from a registry, and log predictions for monitoring and retraining.

3.34 End-to-End Machine Learning Project

This section walks through a complete tabular machine learning project, from business question to deployed service, using customer churn prediction for a telecommunications provider as the running example. The data are a synthetic dataset of 7,000 customers modelled on the widely used IBM Telco Customer Churn sample, so that every number reported here can be regenerated with the listings. The emphasis is on the *decisions* made at each step and how they connect to the tools and principles of the preceding sections. Figure 3.37 gives the roadmap.

3.34.1 Problem Framing

The business goal is to reduce the number of customers who cancel their subscription. The retention team can contact a limited number of customers each month with an offer. The machine learning task is therefore framed as **binary classification**: predict, for each active customer, the probability of churning within the next month, so the team

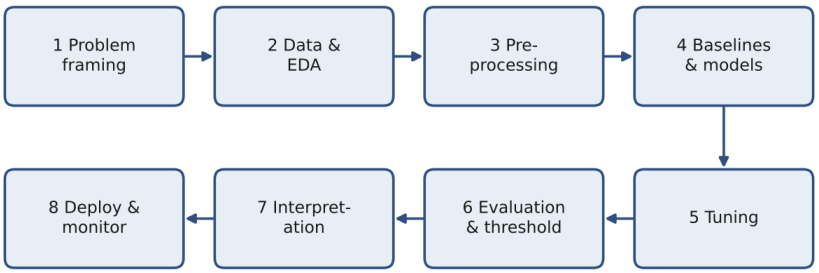


Figure 3.37 Roadmap of the end-to-end churn project. Each stage maps to a subsection; dashed feedback (not drawn) occurs constantly, e.g. error analysis prompting new features.

can rank customers by risk. Framing decisions made at this stage determine everything else:

- **Target definition:** churn = 1 if the customer cancels within 30 days of the snapshot date. Features must use only information available *before* the snapshot to avoid temporal leakage.
- **Primary metric:** because the team acts on a ranked list, threshold-free ranking quality (ROC-AUC) and precision–recall AUC (more informative under class imbalance) are primary; recall at the chosen operating threshold is reported for the business.
- **Cost structure:** a missed churner (false negative) loses roughly \$600 of future margin; an unnecessary offer (false positive) costs about \$60. This 10 : 1 asymmetry will guide the decision threshold.
- **Success criterion:** beat the current rule ("contact all month-to-month customers with tenure < 12 months") on expected net savings, with an interpretable explanation of drivers.

3.34.2 Exploratory Data Analysis

The dataset has nine predictors: tenure (months), contract type, internet service, payment method, tech-support subscription, senior-citizen flag, monthly charges, total charges and the number of support calls in the last 90 days. The churn rate is 25.9 %, a moderate imbalance. Listing 3.47 performs the first checks, and Figure 3.38 shows two of the most informative views: churn rate by contract type and the tenure distribution of churners versus non-churners.

Listing 3.47 First exploratory checks on the churn data.

```
import pandas as pd

df = pd.read_csv("churn.csv")
print(df.shape)                                    # (7000, 10)
print(df.dtypes.value_counts())
print(df.isna().sum().loc[lambda s: s > 0])      # missing values (none in this extract)
print(df["churn"].mean().round(3))                 # 0.259 -> moderate imbalance

# Churn rate by each categorical feature
for col in ["contract", "internet", "payment", "tech_support"]:
    print(df.groupby(col)["churn"].agg(["mean", "size"]).round(3), "\n")

# Numeric features compared between classes
print(df.groupby("churn")[["tenure", "monthly_charges", "support_calls"]].median())

# Correlation check for redundancy: total_charges ~ tenure x monthly_charges
print(df[["tenure", "monthly_charges", "total_charges"]].corr().round(2))
```

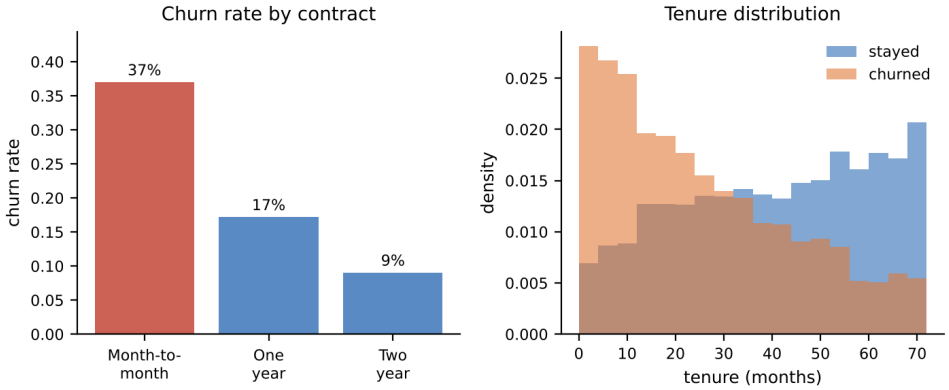


Figure 3.38 Exploratory views of the churn data. Left: month-to-month customers churn far more often than those on longer contracts. Right: churners are concentrated at short tenures.

Three findings shape the modelling. First, contract type is a dominant factor: more than one in three month-to-month customers churn (37 %), compared with 17 % on one-year and fewer than one in ten on two-year contracts. Second, churn risk falls steeply with tenure, so the relationship is non-linear near zero. Third, `total_charges` is almost exactly `tenure × monthly_charges` (correlation with tenure of about 0.82); it carries little new information and inflates collinearity in linear models, but it is retained here because tree models are unaffected and regularisation handles it for logistic regression.

> **Note | Leakage Hunt**
>
> Before modelling, list every feature and ask: *would this value be known at prediction time?* In real

churn data, fields such as ‘cancellation reason’, ‘final bill amount’ or ‘number of retention calls’ are recorded *because* the customer churned and must be excluded. A suspiciously high validation AUC (above 0.95 for churn) is usually a sign of such leakage rather than of a brilliant model.

3.34.3 Preprocessing and Baselines

The data are split once into an 80 % development set and a 20 % test set (1,400 customers, 363 churners), stratified by the target; the test set is not touched until the final evaluation. Preprocessing follows the ColumnTransformer pattern of the scikit-learn section: numeric features are standardised and categorical features one-hot encoded inside a pipeline, so each cross-validation fold learns its own statistics. Listing 3.48 defines the shared preprocessing and the candidate models, including a majority-class baseline.

Listing 3.48 Train–test split, preprocessing and candidate models for the churn project.

```
import pandas as pd
from sklearn.model_selection import train_test_split, StratifiedKFold
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingClassifier

df = pd.read_csv("churn.csv")
y = df.pop("churn"); X = df
NUM = ["tenure", "monthly_charges", "total_charges", "support_calls", "senior"]
CAT = ["contract", "internet", "payment", "tech_support"]
X_dev, X_test, y_dev, y_test = train_test_split(X, y, test_size=0.2, stratify=y,
    random_state=42)

pre = ColumnTransformer([("num", StandardScaler(), NUM),
                         ("cat", OneHotEncoder(handle_unknown="ignore"), CAT)])
candidates = {
    "Majority class": DummyClassifier(strategy="most_frequent"),
    "Logistic regression": LogisticRegression(max_iter=2000, class_weight="balanced"),
    "Random forest": RandomForestClassifier(n_estimators=500, min_samples_leaf=5,
                                            class_weight="balanced", n_jobs=-1,
                                            random_state=0),
    "Gradient boosting": HistGradientBoostingClassifier(class_weight="balanced",
        random_state=0),
}
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)
```

3.34.4 Model Training and Tuning

Each candidate is evaluated with 5-fold cross-validation on the development set (Listing 3.49). Gradient boosting is then tuned with a randomised search over learning rate, tree size, minimum leaf size and L2 regularisation (Listing 3.50). `class_weight='balanced'` reweights the minority class so that the default 0.5 threshold produces useful recall; it does not change the ranking-based AUC much.

Listing 3.49 Cross-validating all candidate pipelines.

```
from sklearn.model_selection import cross_validate

rows = []
for name, est in candidates.items():
    pipe = Pipeline([("pre", pre), ("clf", est)])
    res = cross_validate(pipe, X_dev, y_dev, cv=cv, n_jobs=-1,
                         scoring={"auc": "roc_auc", "ap": "average_precision"})
    rows.append({"model": name,
                 "cv_auc": f"{res['test_auc'].mean():.3f} ± {res['test_auc'].std():.3f}",
                 "cv_ap": round(res["test_ap"].mean(), 3)})
print(pd.DataFrame(rows))
```

Listing 3.50 Randomised hyperparameter search for gradient boosting.

```
from sklearn.model_selection import RandomizedSearchCV

gb_pipe = Pipeline([("pre", pre),
                    ("clf", HistGradientBoostingClassifier(class_weight="balanced",
                     random_state=0))])
space = {"clf__learning_rate": [0.02, 0.05, 0.1],
         "clf__max_leaf_nodes": [7, 15, 31],
         "clf__min_samples_leaf": [20, 50, 100],
         "clf__l2_regularization": [0.0, 0.1, 1.0]}
search = RandomizedSearchCV(gb_pipe, space, n_iter=25, cv=cv, scoring="roc_auc",
                            random_state=0, n_jobs=-1).fit(X_dev, y_dev)
print(search.best_params_)      # small trees (7 leaves), strong regularisation
print(f"CV AUC = {search.best_score_:.3f}")
```

3.34.5 Evaluation

Table 3.43 reports cross-validated and test-set performance for all models. The result is instructive: **regularised logistic regression is the best model**, slightly ahead of the tuned gradient-boosting model, while untuned random forest and gradient boosting trail. Tuning moved gradient boosting towards very small trees with strong regularisation — that is, towards a nearly additive model — which is consistent with the churn signal in this dataset being mostly additive on the log-odds scale. The lesson

generalises: always include a strong simple baseline, because complex models do not automatically win on tabular data.

Table 3.43 Churn prediction results (5-fold CV on the development set; single evaluation on the held-out test set of 1,400 customers; threshold 0.5 with balanced class weights).

Model	CV ROC-AUC	Test ROC-AUC	Test PR-AUC	Recall	Precision	F1
Majority class	0.500 ± 0.000	0.500	0.259	0.000	—	0.000
Logistic regression	0.837 ± 0.004	0.834	0.654	0.788	0.494	0.607
Random forest	0.821 ± 0.008	0.817	0.610	0.744	0.497	0.596
Gradient boosting (default)	0.817 ± 0.011	0.810	0.599	0.725	0.493	0.587
Gradient boosting (tuned)	0.830	0.828	0.636	0.766	0.496	0.602

Is the difference between logistic regression and tuned boosting meaningful? A paired bootstrap of the test-set AUC difference (Listing 3.40, 2,000 resamples) gives $\Delta = +0.006$ with a 95 % CI of $+0.001$ to $+0.012$. The interval just excludes zero, so the advantage is statistically detectable, but its size is negligible in practical terms. With performance effectively tied, the simpler, faster and more interpretable logistic regression is chosen for deployment — a decision driven by effect size and cost, not by the p-value alone.

3.34.6 Choosing the Decision Threshold

A probability becomes an action only through a threshold τ. With the cost structure from the framing step, and assuming the retention offer saves a fraction s of the churners who receive it, the cost of a threshold is given in Equation (3.62). The optimal threshold can be found by scanning τ on out-of-fold predictions (Listing 3.51). For a well-calibrated model the decision rule has a closed form: contact a customer when the expected saving $p \cdot s \cdot C_{FN}$ exceeds the offer cost C_{FP}. With $s = 0.3$ this gives $\tau^* = 60 / (0.3 \times 600) \approx 0.33$, which motivates the default threshold of 0.35 used in the API of the deployment section. Because balanced class weights distort probabilities, the empirical scan (or prior recalibration) is preferred to the formula.

$$\begin{aligned} \mathrm{Cost}(\tau) &= C_{\mathrm{FN}}[\mathrm{FN}(\tau) + (1-s)\,\mathrm{TP}(\tau)] + C_{\mathrm{FP}}[\mathrm{TP}(\tau) + \mathrm{FP}(\tau)] \\ \tau^* &= \frac{C_{\mathrm{FP}}}{s\,C_{\mathrm{FN}}} \quad \text{(calibrated probabilities)} \end{aligned} \tag{3.62}$$

where $\mathrm{TP}(\tau)$, $\mathrm{FP}(\tau)$ and $\mathrm{FN}(\tau)$ are the counts of true positives, false positives and false negatives at threshold τ, C_{FN} is the margin lost per churner, C_{FP} the cost of one offer, and s the fraction of contacted churners who are retained.

Listing 3.51 Selecting a cost-optimal threshold from out-of-fold predictions.

```
import numpy as np
from sklearn.model_selection import cross_val_predict

best_pipe = Pipeline([("pre", pre), ("clf", LogisticRegression(max_iter=2000,
    class_weight="balanced"))])
oof = cross_val_predict(best_pipe, X_dev, y_dev, cv=cv, method="predict_proba")[:, 1]

C_FN, C_FP, success = 600, 60, 0.30          # offer retains 30% of contacted churners
def net_cost(tau):
    pred = oof >= tau
    tp = np.sum(pred & (y_dev == 1)); fp = np.sum(pred & (y_dev == 0))
    fn = np.sum(~pred & (y_dev == 1))
    # contacted churners: offer cost, and a fraction are saved; others are lost
    return (fn + tp * (1 - success)) * C_FN + (tp + fp) * C_FP

taus = np.linspace(0.05, 0.95, 91)
costs = [net_cost(t) for t in taus]
tau_star = taus[int(np.argmin(costs))]
print(f"optimal threshold = {tau_star:.2f}; cost {min(costs):,.0f} vs contact-nobody
    {net_cost(1.01):,.0f}")
```

3.34.7 Interpretation

For stakeholders, the model must explain *why* customers are at risk. Logistic regression coefficients give global effects on the log-odds; permutation importance and SHAP values (introduced in Chapter 2 and in the Responsible AI section of Part A) provide model-agnostic global and local explanations. Listing 3.52 computes both. In this dataset the strongest drivers are short tenure, month-to-month contracts, fibre-optic internet, many support calls and electronic-check payment, while tech-support subscriptions and two-year contracts are protective — findings the retention team can act on, for example by offering contract upgrades to high-risk month-to-month customers.

Listing 3.52 Global and local explanations: coefficients, permutation importance and SHAP.

```
import numpy as np, pandas as pd, shap
from sklearn.inspection import permutation_importance

final = best_pipe.fit(X_dev, y_dev)
names = final.named_steps["pre"].get_feature_names_out()
coefs = pd.Series(final.named_steps["clf"].coef_[0], index=names).sort_values()
print(coefs.round(2))                                  # sign and size on log-odds
    scale

perm = permutation_importance(final, X_test, y_test, scoring="roc_auc",
                              n_repeats=20, random_state=0, n_jobs=-1)
print(pd.Series(perm.importances_mean,
    index=X_test.columns).sort_values(ascending=False).round(3))

```

```
# SHAP for individual customers (explain the probability via a model-agnostic explainer)
Xt = final.named_steps["pre"].transform(X_test)
explainer = shap.LinearExplainer(final.named_steps["clf"], final.named_steps["pre"].transform(X_dev))
sv = explainer(Xt)
shap.plots.waterfall(sv[0])                              # why is customer 0 high risk?
```

3.34.8 Deployment and Monitoring

The final pipeline is retrained on the full development set, evaluated once on the test set, saved with its version and training-data hash, and served through the FastAPI service and Docker image of the previous section. A nightly batch job scores all active customers and writes the ranked list to the customer relationship management system. Monitoring tracks the input distributions (for example the share of month-to-month contracts), the score distribution and — once outcomes arrive 30 days later — realised AUC and calibration. A drop beyond an agreed tolerance triggers retraining. Finally, the business impact is measured with a randomised control group of high-risk customers who receive no offer, since only an experiment can show that the model-driven campaign actually *causes* reduced churn (see the Causal ML section in Part A).

C Case Study 3.8 | Lessons from the Churn Project

- Framing (target window, metric, costs) mattered more than algorithm choice.
- A regularised linear model matched tuned gradient boosting; simplicity won the tie.
- Threshold selection by expected cost turned a probability into a business decision.
- Interpretation turned the model into actionable retention strategies.
- A randomised hold-out group is needed to measure true uplift after deployment.

3.35 End-to-End Deep Learning Project

This section mirrors the previous one for a perception task: classifying photographs of flowers into five species (daisy, dandelion, rose, sunflower and tulip) using the publicly available TensorFlow *tf_flowers* dataset of 3,670 images. With only a few hundred images per class, training a deep convolutional network from scratch would overfit badly. The project therefore uses **transfer learning**: a network pretrained on ImageNet supplies general-purpose visual features, and only a small amount of task-specific learning is needed. The implementation uses PyTorch and torchvision, building on the

PyTorch section; the convolutional architectures themselves are described in Chapter 2.

3.35.1 Data Preparation and Augmentation

The images are split into 70 % training, 15 % validation and 15 % test sets, stratified by class (about 2,570 / 550 / 550 images). The images vary in size and aspect ratio, so they are resized and cropped to 224 × 224 pixels and normalised with the ImageNet channel means and standard deviations that the pretrained network expects. **Data augmentation** — random transformations applied to training images on the fly — effectively enlarges the training set and teaches invariances: a tulip is still a tulip when flipped horizontally, slightly rotated, cropped or photographed in different light. Augmentations must be label-preserving: vertical flips are harmless for flowers but would be wrong for digits such as 6 and 9. Validation and test images receive only deterministic resizing and normalisation (Listing 3.53).

Listing 3.53 Data splits, augmentation pipelines and loaders for the flower classifier.

```
import torch
from torch.utils.data import DataLoader, Subset
from torchvision import datasets, transforms
from sklearn.model_selection import train_test_split

MEAN, STD = [0.485, 0.456, 0.406], [0.229, 0.224, 0.225]       # ImageNet statistics
train_tf = transforms.Compose([
    transforms.RandomResizedCrop(224, scale=(0.6, 1.0)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(15),
    transforms.ColorJitter(brightness=0.3, contrast=0.3, saturation=0.3),
    transforms.ToTensor(), transforms.Normalize(MEAN, STD)])
eval_tf = transforms.Compose([
    transforms.Resize(256), transforms.CenterCrop(224),
    transforms.ToTensor(), transforms.Normalize(MEAN, STD)])

# flower_photos/<class_name>/*.jpg  (one folder per class)
base = datasets.ImageFolder("flower_photos")
idx, labels = list(range(len(base))), base.targets
tr_idx, tmp_idx = train_test_split(idx, test_size=0.30, stratify=labels,
    random_state=0)
va_idx, te_idx = train_test_split(tmp_idx, test_size=0.50,
                                  stratify=[labels[i] for i in tmp_idx],
                                  random_state=0)

train_ds = Subset(datasets.ImageFolder("flower_photos", transform=train_tf), tr_idx)
val_ds = Subset(datasets.ImageFolder("flower_photos", transform=eval_tf), va_idx)
test_ds = Subset(datasets.ImageFolder("flower_photos", transform=eval_tf), te_idx)
loaders = {k: DataLoader(d, batch_size=32, shuffle=(k == "train"), num_workers=4,
    pin_memory=True)
           for k, d in [("train", train_ds), ("val", val_ds), ("test", test_ds)]}
classes = base.classes        # ['daisy', 'dandelion', 'roses', 'sunflowers', 'tulips']
```

3.35.2 Model: Transfer Learning with a Pretrained Backbone

The model is a ResNet-50 (He et al., 2016) pretrained on ImageNet-1k. Its convolutional layers act as a generic feature extractor, and its final 1,000-way classification layer is replaced by a new 5-way layer. Training proceeds in two phases, as Figure 3.39 shows. In **phase 1 (feature extraction)** the backbone is frozen and only the new head is trained, with a relatively high learning rate, so that the randomly initialised head does not send large, destructive gradients into the pretrained weights. In **phase 2 (fine-tuning)** the deeper backbone blocks are unfrozen and the whole network is trained with a learning rate ten to a hundred times smaller, allowing the high-level features to adapt to flowers while preserving the generic low-level filters.

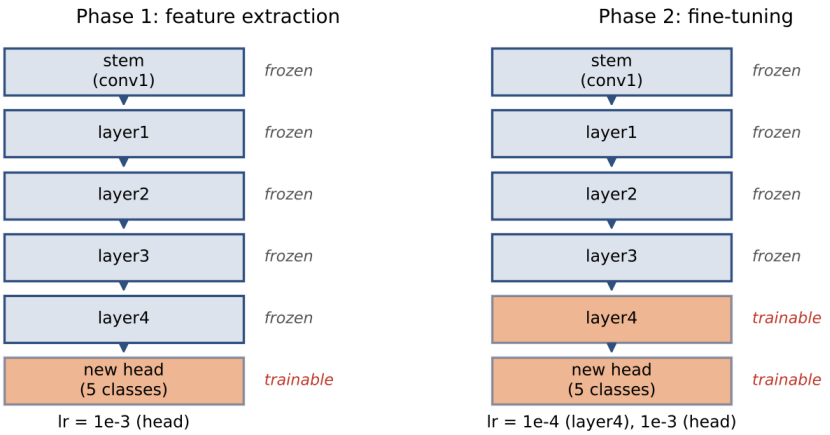


Figure 3.39 Two-phase transfer learning. Phase 1 trains only the new classification head on frozen ImageNet features; phase 2 unfreezes the last residual stage and fine-tunes with a small learning rate.

Listing 3.54 Building the transfer-learning model and the two optimiser configurations.

```
import torch
from torch import nn
from torchvision import models

device = "cuda" if torch.cuda.is_available() else "cpu"
weights = models.ResNet50_Weights.IMAGENET1K_V2
model = models.resnet50(weights=weights)

for p in model.parameters():                    # phase 1: freeze the whole backbone
    p.requires_grad = False
model.fc = nn.Sequential(nn.Dropout(0.3), nn.Linear(model.fc.in_features,
↪  len(classes)))
model = model.to(device)

criterion = nn.CrossEntropyLoss(label_smoothing=0.1)
```

```
opt_phase1 = torch.optim.AdamW(model.fc.parameters(), lr=1e-3, weight_decay=1e-4)

def unfreeze_for_phase2(model):
    for p in model.layer4.parameters():           # unfreeze last residual stage only
        p.requires_grad = True
    return torch.optim.AdamW([
        {"params": model.layer4.parameters(), "lr": 1e-4},   # discriminative learning
        ↪ rates
        {"params": model.fc.parameters(), "lr": 1e-3}], weight_decay=1e-4)
```

3.35.3 Training

Training reuses the loop of Algorithm 3.13, wrapped in a function that runs one epoch in training or evaluation mode (Listing 3.55). Phase 1 runs for 5 epochs; phase 2 runs for up to 15 epochs with a cosine learning-rate schedule and early stopping on validation loss. Batch-normalisation layers in frozen blocks are kept in evaluation mode so that their running statistics, estimated on ImageNet, are not disturbed by small flower batches.

Listing 3.55 Two-phase training with early stopping and history tracking.

```
def run_epoch(model, loader, optimizer=None):
    train = optimizer is not None
    model.train(train)
    if train:                                     # keep frozen BatchNorm layers in
    ↪ eval mode
        for m in model.modules():
            if isinstance(m, nn.BatchNorm2d) and not any(p.requires_grad for p in
            ↪ m.parameters()):
                m.eval()
    total, correct, loss_sum = 0, 0, 0.0
    with torch.set_grad_enabled(train):
        for x, y in loader:
            x, y = x.to(device), y.to(device)
            out = model(x); loss = criterion(out, y)
            if train:
                optimizer.zero_grad(); loss.backward(); optimizer.step()
            loss_sum += loss.item() * len(y); total += len(y)
            correct += (out.argmax(1) == y).sum().item()
    return loss_sum / total, correct / total

history, best, patience = [], float("inf"), 0
for phase, epochs in [(1, 5), (2, 15)]:
    opt = opt_phase1 if phase == 1 else unfreeze_for_phase2(model)
    sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=epochs)
    for ep in range(epochs):
        tr = run_epoch(model, loaders["train"], opt); va = run_epoch(model,
        ↪ loaders["val"])
        sched.step(); history.append((phase, *tr, *va))
        if va[0] < best - 1e-4:
```

```
            best, patience = va[0], 0; torch.save(model.state_dict(),
            ↪  "flowers_best.pt")
        else:
            patience += 1
            if patience >= 4: break
```

Figure 3.40 shows typical training curves. During phase 1 validation accuracy climbs quickly to about 88 %, because ImageNet features already separate flower species reasonably well. When fine-tuning begins at epoch 6 there is a brief wobble as the backbone starts to change, followed by a further rise to about 94 %. The small, stable gap between training and validation loss indicates that augmentation and the frozen early layers keep overfitting under control; training loss stays above zero partly because label smoothing prevents fully confident predictions.

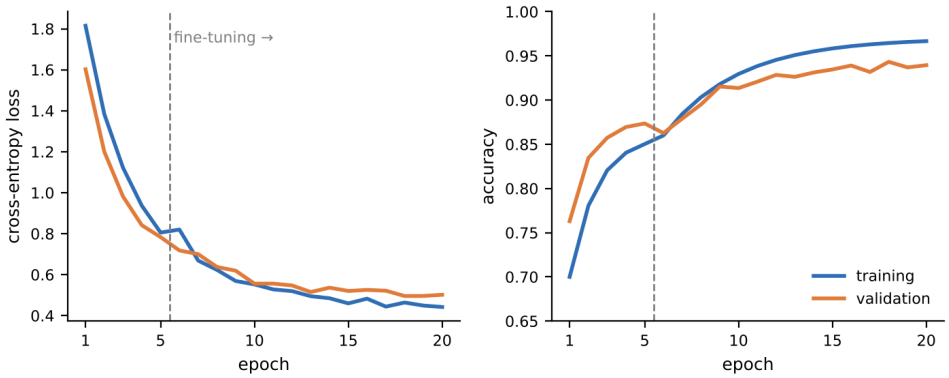


Figure 3.40 Training curves for two-phase transfer learning on the flower dataset (representative run). The dashed line marks the start of fine-tuning; validation accuracy rises from about 88 % to about 94 %.

3.35.4 Evaluation

The best checkpoint is evaluated once on the 550-image test set. Overall accuracy is 93.5 % (95 % Wilson CI roughly 91.1 %–95.3 %), and macro-averaged F1 is 0.934. The confusion matrix in Figure 3.41 and the per-class metrics in Table 3.44 reveal where errors occur: most mistakes are between roses and tulips, which share colours and cup-shaped blooms, and between daisies and dandelions when the latter are photographed as white seed heads. Sunflowers, with their distinctive structure, are almost never confused.

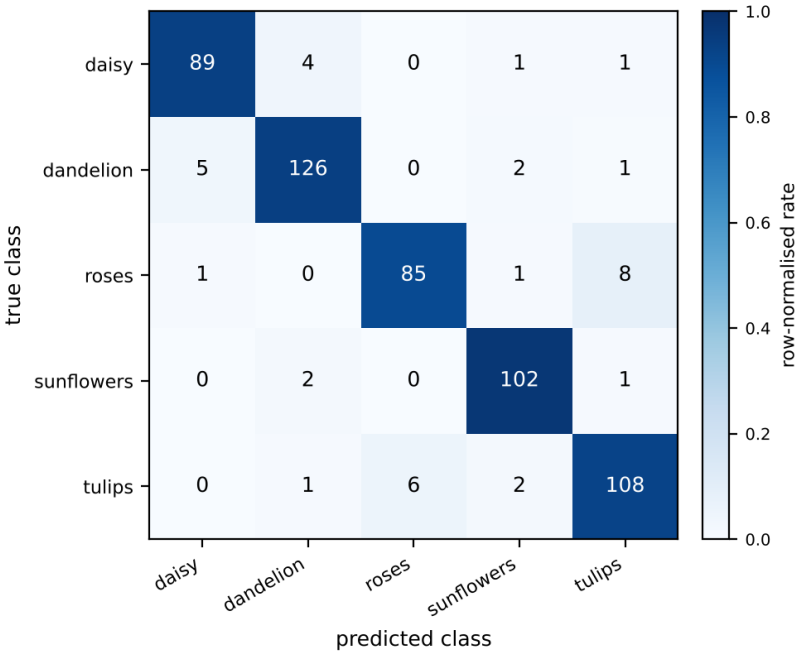


Figure 3.41 Test-set confusion matrix for the fine-tuned ResNet-50 (rows: true class, columns: predicted). Most errors lie in the rose–tulip and daisy–dandelion pairs.

Table 3.44 Per-class test results of the fine-tuned ResNet-50 (n = 550; derived from the confusion matrix).

Class	Support	Precision	Recall	F1
Daisy	95	0.937	0.937	0.937
Dandelion	134	0.947	0.940	0.944
Roses	95	0.934	0.895	0.914
Sunflowers	105	0.944	0.971	0.958
Tulips	117	0.908	0.923	0.915
Macro average	550	0.934	0.933	0.934
Accuracy	550			0.935

Listing 3.56 Test-set evaluation: classification report and confusion matrix.

```
import numpy as np
from sklearn.metrics import classification_report, confusion_matrix,
    ConfusionMatrixDisplay

model.load_state_dict(torch.load("flowers_best.pt", map_location=device)); model.eval()
y_true, y_pred, y_prob = [], [], []
with torch.no_grad():
    for x, y in loaders["test"]:
        p = torch.softmax(model(x.to(device)), dim=1).cpu()
        y_prob.append(p); y_pred += p.argmax(1).tolist(); y_true += y.tolist()

```

```
print(classification_report(y_true, y_pred, target_names=classes, digits=3))
cm = confusion_matrix(y_true, y_pred)
ConfusionMatrixDisplay(cm, display_labels=classes).plot(cmap="Blues",
    xticks_rotation=30)

# Collect the most confident mistakes for error analysis
y_prob = torch.cat(y_prob).numpy()
wrong = np.where(np.array(y_true) != np.array(y_pred))[0]
worst = wrong[np.argsort(-y_prob[wrong].max(1))][:16]      # inspect these images by
    eye
```

3.35.5 Explaining Predictions with Grad-CAM

To check that the network attends to the flower rather than to background cues (a lawn for dandelions, a vase for roses), we use **Gradient-weighted Class Activation Mapping (Grad-CAM)** (Selvaraju et al., 2017). Grad-CAM weights each feature map A^k of the last convolutional layer by the average gradient of the class score with respect to that map, sums the weighted maps and keeps the positive part, as in Equation (3.63). The resulting coarse heat map is upsampled and overlaid on the image. Applied to the most confident errors from Listing 3.56, Grad-CAM often reveals photographs containing several species, or focus on foliage — useful evidence for data cleaning. Grad-CAM and other attribution methods are discussed further in the Responsible AI section of Part A.

$$\alpha_k^c = \frac{1}{Z}\sum_i\sum_j \frac{\partial y^c}{\partial A_{ij}^k}$$
$$L_{\text{GradCAM}}^c = \text{ReLU}\left(\sum_k \alpha_k^c A^k\right) \tag{3.63}$$

where y^c is the score (logit) of class c, A^k the k-th feature map of the chosen convolutional layer, Z the number of spatial positions, and α_k^c the importance weight of map k for class c.

3.35.6 Exporting the Model

The trained network is exported for deployment in a framework-independent format. Listing 3.57 produces a TorchScript file (used by the Gradio demo in Listing 3.45) and an ONNX file that can be served by ONNX Runtime or NVIDIA Triton, or converted for mobile. A numerical check confirms that the exported model reproduces the PyTorch outputs. Further gains are possible with post-training quantisation (8-bit weights cut model size about fourfold) or with knowledge distillation into a smaller network such as MobileNetV3 for edge devices (see the Green AI section in Part A).

Listing 3.57 Exporting to TorchScript and ONNX and verifying the exported model.

```
import numpy as np, onnxruntime as ort

model.eval().cpu()
example = torch.randn(1, 3, 224, 224)

scripted = torch.jit.trace(model, example)              # TorchScript
scripted.save("flower_classifier.pt")

torch.onnx.export(model, example, "flower_classifier.onnx",
                  input_names=["image"], output_names=["logits"],
                  dynamic_axes={"image": {0: "batch"}, "logits": {0: "batch"}},
                  opset_version=17)

sess = ort.InferenceSession("flower_classifier.onnx")
onnx_out = sess.run(None, {"image": example.numpy()})[0]
torch_out = model(example).detach().numpy()
print("max abs diff:", np.abs(onnx_out - torch_out).max())   # ~1e-5: export is faithful
```

Key Point | Deep Learning Project Lessons

- Transfer learning makes small image datasets tractable; freeze first, then fine-tune gently.
- Augment only with label-preserving transformations; evaluate on un-augmented images.
- Inspect the confusion matrix and the most confident errors; they guide data cleaning.
- Use Grad-CAM to detect shortcut learning before deployment.
- Verify exported models numerically against the original.

3.36 Research Trends, Open Problems, and Future Directions

Machine learning research moves quickly, but its central questions are durable: how to learn more from less data, how to generalise beyond the training distribution, how to reason, how to be trustworthy and how to do all of this efficiently. This closing section surveys active research directions and open problems, and then offers practical guidance for students preparing their first research paper or thesis.

3.36.1 Current Trends

Several trends dominate the mid-2020s landscape. **Foundation models** — very large models pretrained on broad data and adapted to many tasks (Bommasani et al., 2021) — have shifted much applied work from training models to adapting them, through

fine-tuning, parameter-efficient methods such as LoRA, prompting and retrieval augmentation (see the RAG section in Part A). **Multimodal models** jointly process text, images, audio and video. **Scaling laws** (Kaplan et al., 2020; Hoffmann et al., 2022) relate performance to model size, data and compute, guiding resource allocation. **Agentic systems** combine language models with tools, memory and planning (Part A, AI Agents). Alongside these, attention has grown to **efficiency** (quantisation, distillation, sparse mixture-of-experts), **data quality and curation**, **evaluation methodology**, and **alignment and safety**.

3.36.2 Open Problems

Table 3.45 lists important open problems, why they matter and representative research directions. Many make excellent thesis topics because progress can be demonstrated on modest compute with careful experimental design.

> **Note | Choosing a Research Problem**
>
> - Prefer problems where you have an *unfair advantage*: unique data, domain knowledge or a measurement others lack.
> - Make the scope testable within your compute budget; a careful study on mid-sized benchmarks can be more valuable than an under-powered large-scale attempt.
> - Negative or replication results, carefully executed, are genuine contributions.
> - Read the last two years of a venue's papers on the topic before committing.

3.36.3 Writing a Machine Learning Research Paper

Most machine learning papers follow an adapted **IMRaD** structure (Introduction, Methods, Results and Discussion), summarised in Table 3.46. Conference papers are short (typically 8–9 pages plus references and appendices), so every section must earn its space. A useful discipline is to write the contribution list first: two to four precise, verifiable claims. Every experiment in the paper should support one of them, and the abstract, introduction and conclusion should state the same claims consistently. Figure 3.42 shows the research cycle into which writing fits; many experienced researchers start drafting early because writing exposes gaps in the argument.

3.36.4 Choosing a Venue

Machine learning is unusual in that top-tier **conferences** are the primary publication venues. The leading general conferences are NeurIPS, ICML and ICLR; AAAI and IJCAI cover broader AI; CVPR, ICCV and ECCV lead in computer vision; ACL, EMNLP

Table 3.45 Open problems in machine learning research.

Problem	Why it matters	Active directions
Out-of-distribution generalisation	Models fail under distribution shift in deployment	Domain generalisation, invariant learning, test-time adaptation
Robustness and security	Adversarial examples, data poisoning, prompt injection	Certified robustness, adversarial training, red-teaming
Reasoning and compositionality	Models struggle with multi-step, systematic reasoning	Chain-of-thought, neuro-symbolic methods, verifiers
Hallucination and factuality	Fluent but false outputs undermine trust	Retrieval grounding, calibration, attribution
Interpretability	Need to understand and audit decisions	Mechanistic interpretability, concept-based explanations
Uncertainty quantification	Reliable confidence is essential for high-stakes use	Conformal prediction, Bayesian deep learning, ensembles
Data efficiency	Labels are costly; rare classes and long tails	Self-supervised, few-shot and active learning
Continual learning	Models must update without forgetting	Replay, regularisation, modular architectures
Causality	Correlation-based models mislead under intervention	Causal representation learning, causal discovery
Fairness and accountability	Disparate impact on groups	Fairness constraints, auditing, documentation
Privacy	Models can memorise and leak training data	Differential privacy, federated learning, unlearning
Efficiency and sustainability	Growing energy and cost of training/inference	Quantisation, sparsity, efficient architectures
Evaluation	Benchmarks saturate or are contaminated	Dynamic benchmarks, human evaluation, held-out audits
Theory of deep learning	Why over-parameterised networks generalise	Implicit regularisation, double descent, NTK

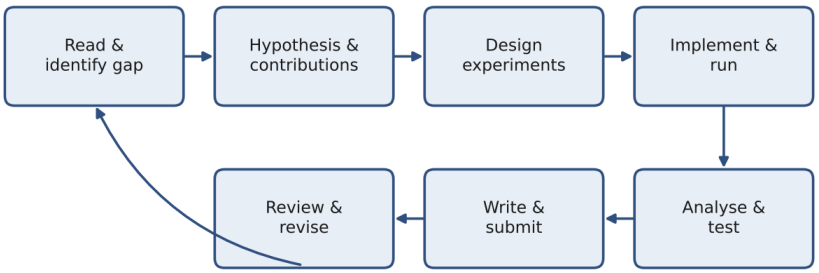


Figure 3.42 The research cycle. Writing is not a final step but part of an iterative loop; reviews and replication feed new questions.

Table 3.46 Structure of a machine learning research paper (adapted IMRaD).

Section	Purpose	Tips
Title and abstract	State problem, approach, key result in ~200 words	Include one quantitative result; avoid hype
Introduction	Motivate problem, identify gap, list contributions	End with a bulleted contribution list
Related work	Position the work relative to prior art	Organise by theme; state differences explicitly
Method	Describe the approach precisely enough to re-implement	Notation table, algorithm box, figure of the model
Experimental setup	Datasets, baselines, metrics, protocol, hyperparameters	Equal tuning budget; seeds; compute used
Results	Main comparison tables and figures	Mean $\pm$ std, significance tests, best results not cherry-picked
Analysis / ablations	Why and when the method works	Ablations, sensitivity, error analysis, failure cases
Discussion and limitations	Honest scope and weaknesses	Required by many venues; strengthens credibility
Conclusion	Summarise findings and future work	No new claims
Ethics / broader impact	Potential harms, data consent, misuse	Follow the venue's checklist
Appendix and code	Proofs, extra results, implementation details	Link to anonymised code for review

and NAACL in natural language processing; KDD in data mining; and AISTATS and UAI in statistical and probabilistic ML. Leading **journals** include the *Journal of Machine Learning Research* (JMLR), *IEEE Transactions on Pattern Analysis and Machine Intelligence* (TPAMI), *Machine Learning* (Springer), *Transactions on Machine Learning Research* (TMLR, with rolling submission) and *Neural Networks*; applied work often targets domain journals in medicine, engineering or the sciences. Workshops attached to major conferences are good venues for early-stage ideas. Assess a venue by its scope, the community you want to reach, review quality and timelines — not by impact factor alone — and be wary of predatory publishers that promise rapid acceptance for a fee.

3.36.5 Ethics Statements and Responsible Research

Major venues now require authors to reflect on the societal impact of their work. NeurIPS introduced broader-impact statements in 2020 and later a detailed paper checklist covering reproducibility, ethics and limitations. A good ethics statement is specific rather than generic: it identifies who could be affected, plausible misuse, dataset consent and licensing, privacy of personal data, potential bias across groups,

and environmental cost. Human-subject studies and annotation work require appropriate institutional ethics approval and fair compensation of annotators. Releasing datasets with datasheets (Gebru et al., 2021) and models with model cards (Mitchell et al., 2019) makes responsible reuse easier.

Practice Tip | Before You Submit

- Do the abstract, introduction and conclusion make the same, supported claims?
- Could a competent reader re-implement the method from the paper and appendix?
- Are baselines tuned fairly, results averaged over seeds and differences tested statistically?
- Are limitations, ethics and compute stated honestly?
- Is the submission anonymised (no names, acknowledgements or identifying links) for double-blind review?

Chapter Summary

This chapter moved from the foundations that underpin modern machine learning to the integrated systems and research practice that turn models into reliable, responsible applications.

Part A began with the mathematical and statistical toolkit: descriptive and inferential statistics, sampling distributions and hypothesis tests; linear algebra (vectors, matrices, eigendecomposition, SVD) and multivariate calculus (gradients, Jacobians, the chain rule) that underlie optimisation; probability theory and distributions; and information theory, whose entropy, cross-entropy and KL divergence define many loss functions. It then turned to data at scale: data engineering and ETL pipelines, big-data frameworks such as Spark, and feature stores that keep training and serving features consistent. Model selection and hyperparameter optimisation, from grid and random search to Bayesian optimisation, connected these foundations to practice. A sequence of advanced learning paradigms followed: causal machine learning, which distinguishes correlation from intervention; federated learning, which trains across decentralised data; online and continual learning, which update models over time without catastrophic forgetting; few-shot and zero-shot learning and meta-learning, which learn from minimal supervision; retrieval-augmented generation and AI agents, which extend large language models with knowledge and tools. Part A closed with responsible AI (fairness, accountability, transparency and explainability), privacy-preserving machine learning (differential privacy, secure computation), green AI and the environmental cost of computation, and integrated case studies.

Part B was practical. It set up Python, Jupyter and Colab environments; developed fluency with NumPy broadcasting and pandas data manipulation; chose and built

effective visualisations; and assembled leak-free scikit-learn pipelines with ColumnTransformer and GridSearchCV, as well as visual KNIME workflows. It built neural networks in Keras (Sequential and Functional APIs, callbacks) and in PyTorch (tensors, autograd, nn.Module, DataLoaders, training loops). It then addressed the research process: selecting and annotating datasets, with inter-annotator agreement measured by Cohen's kappa and labelling made efficient by active learning; designing experiments with benchmarks, baselines and ablations; ensuring reproducibility through seeding, environment pinning and experiment tracking; comparing models with paired and corrected t-tests, McNemar's test, the Wilcoxon signed-rank test and the Friedman–Nemenyi procedure; and interpreting results with confidence intervals, effect sizes, error analysis and learning curves. Deployment with FastAPI, Streamlit, Gradio and Docker was followed by two end-to-end case studies — customer churn on tabular data and flower classification with transfer learning — and a survey of open problems and guidance on writing and publishing research.

The unifying message of the chapter is that trustworthy machine learning is a discipline of the whole pipeline: sound mathematics, careful data, rigorous evaluation, reproducible engineering and responsible deployment matter as much as the choice of algorithm.

Key Terms

Table 3.47 Key terms introduced in Chapter 3.

Term	Definition
Sampling distribution	Distribution of a statistic over repeated samples from a population
Singular value decomposition (SVD)	Factorisation $X = U\Sigma V^T$ underlying PCA and low-rank approximation
Gradient / Jacobian	Vector / matrix of first partial derivatives used in optimisation
Kullback–Leibler divergence	Asymmetric measure of how one distribution differs from another
Cross-entropy	Expected negative log-likelihood of true labels under a model's predictions
ETL pipeline	Extract–transform–load process that prepares data for analysis
Feature store	System that stores, versions and serves features consistently for training and inference
Bayesian optimisation	Sample-efficient hyperparameter search guided by a surrogate model
Causal effect	Change in an outcome caused by intervening on a variable

Term	Definition
Federated learning	Training a shared model across clients without centralising raw data
Online learning	Updating a model incrementally as each new example arrives
Catastrophic forgetting	Loss of earlier knowledge when a model learns new tasks
Few-shot / zero-shot learning	Generalising to new classes from few or no labelled examples
Meta-learning	Learning to learn across a distribution of tasks
Retrieval-augmented generation (RAG)	Grounding a generative model's output on retrieved documents
AI agent	System that perceives, plans and acts with tools toward goals
Differential privacy	Formal guarantee limiting what an output reveals about any individual
Green AI	Practice of measuring and reducing the energy and carbon cost of ML
Virtual environment	Isolated Python interpreter and package set for one project
Broadcasting	NumPy rules for combining arrays of different shapes without copying
Vectorisation	Expressing computation as whole-array operations instead of loops
Pipeline	Chained preprocessing and estimator fitted and tuned as one object
ColumnTransformer	Applies different transformers to different column groups
Callback	Hook invoked during training for early stopping, checkpointing or logging
Autograd	Automatic reverse-mode differentiation of recorded tensor operations
Cohen's kappa	Chance-corrected agreement between two annotators
Active learning	Model-driven selection of the most informative examples to label
Ablation study	Removing components one at a time to measure their contribution
Experiment tracking	Systematic logging of configurations, metrics and artefacts of runs
Corrected resampled t-test	Paired t-test with variance corrected for overlapping CV training sets
McNemar's test	Test of equal error rates using discordant predictions on one test set
Friedman test	Rank-based test comparing several algorithms over many datasets
Critical difference (CD)	Minimum average-rank gap declared significant by the Nemenyi test
Effect size	Magnitude of a difference, independent of sample size
Learning curve	Performance as a function of training-set size, diagnosing bias vs variance

Term	Definition
Transfer learning	Reusing a model pretrained on one task as the starting point for another
Grad-CAM	Gradient-weighted class activation map highlighting image regions behind a prediction
Containerisation	Packaging an application with its environment into a portable image

Review Questions

1. Explain the difference between a sampling distribution and the distribution of the data, and why the distinction matters for confidence intervals.
2. How is the singular value decomposition related to principal component analysis? What do the singular values represent?
3. Show that minimising cross-entropy is equivalent to minimising KL divergence from the data distribution to the model distribution.
4. What problem does a feature store solve, and what is training–serving skew?
5. Compare grid search, random search and Bayesian optimisation for hyperparameter tuning. When is each appropriate?
6. Why can a purely predictive model give misleading answers to a causal question such as "will this discount reduce churn?"
7. Describe the FedAvg algorithm. What challenges arise from non-IID client data?
8. What is catastrophic forgetting, and how do replay and regularisation-based methods mitigate it?
9. Distinguish few-shot learning, zero-shot learning and meta-learning, giving one method for each.
10. Outline the components of a retrieval-augmented generation system and two ways it can fail.
11. Define ε-differential privacy and explain the role of ε.
12. Why should Jupyter notebooks be restarted and run from top to bottom before sharing?
13. State NumPy's broadcasting rules and determine the result shape of combining arrays of shape $(5, 1, 4)$ and $(3, 1)$.
14. Explain how a scikit-learn Pipeline prevents data leakage during cross-validation.

15. Compare the Keras Sequential and Functional APIs. Give an architecture that requires the Functional API.
16. List the five steps of a PyTorch training iteration and explain why `optimizer.zero_grad()` is needed.
17. Why is raw percentage agreement a poor measure of annotation quality? How does Cohen's kappa address this?
18. Why does the standard paired t-test over cross-validation folds have an inflated Type I error, and how does the Nadeau–Bengio correction work?
19. When would you use McNemar's test rather than a t-test, and when would you use the Friedman test?
20. How do learning curves distinguish high bias from high variance, and what remedy does each suggest?
21. Describe the two phases of transfer learning and explain why the learning rate is reduced for fine-tuning.

Exercises and Mini-Projects

1. **Statistics from scratch.** Using NumPy only, implement the bootstrap for the median of a skewed sample and compare the percentile interval with the normal-approximation interval for the mean.
2. **SVD compression.** Load a grey-scale image, compute its SVD and plot reconstruction error against rank; find the rank that retains 95 % of the energy.
3. **Vectorisation benchmark.** Implement k-means assignment with loops and with broadcasting (Listing 3.19); time both for $n = 10^3$, 10^4 and 10^5 points and plot the speed-up.
4. **Visual EDA report.** For a UCI dataset of your choice, produce a one-page EDA figure containing a histogram, box plot, scatter plot and correlation heatmap, following Table 3.31.
5. **Leakage experiment.** On a dataset with many features and few rows, compare cross-validated accuracy when feature selection is done (a) before splitting and (b) inside a Pipeline. Explain the gap.
6. **Framework comparison.** Train the same small CNN on CIFAR-10 in Keras and in PyTorch with matched hyperparameters and seeds; compare accuracy over five seeds and training time.

7. **Annotation study.** With two classmates, label 100 tweets for sentiment using a one-paragraph guideline; compute Cohen's and Fleiss' kappa, revise the guideline and repeat.
8. **Active learning simulation.** Implement Algorithm 3.14 on MNIST with logistic regression; compare uncertainty sampling with random sampling by plotting accuracy against labels used.
9. **Statistical comparison.** Evaluate four classifiers on ten OpenML datasets, apply the Friedman test, compute the Nemenyi CD and draw a critical-difference diagram as in Figure 3.34.
10. **Reproducible pipeline.** Rebuild the churn project with DVC-tracked data, an MLflow experiment and a Docker image; ask a classmate to reproduce your test AUC from the repository alone.
11. **Deploy a model.** Serve the churn pipeline with FastAPI, add a Streamlit front end and deploy both with Docker Compose; add a `/metrics` endpoint that reports request counts and latency.
12. **Transfer learning and Grad-CAM.** Repeat the flower project with a MobileNetV3 backbone; compare accuracy, model size and latency with ResNet-50, and produce Grad-CAM maps for five errors.
13. **Mini research paper.** Choose an open problem from Table 3.45, formulate one testable hypothesis, run a small controlled experiment with baselines and ablations, and write a four-page paper following Table 3.46, including an ethics statement.

Further Reading

- Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.). Springer.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- Murphy, K. P. (2022). *Probabilistic Machine Learning: An Introduction*. MIT Press.
- Deisenroth, M. P., Faisal, A. A., & Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press.
- Cover, T. M., & Thomas, J. A. (2006). *Elements of Information Theory* (2nd ed.). Wiley.
- Pearl, J., & Mackenzie, D. (2018). *The Book of Why: The New Science of Cause and Effect*. Basic Books.

- McKinney, W. (2022). *Python for Data Analysis* (3rd ed.). O'Reilly.
- Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly.
- Chollet, F. (2021). *Deep Learning with Python* (2nd ed.). Manning.
- Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Paszke, A., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems 32*.
- Dietterich, T. G. (1998). Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7), 1895–1923.
- Nadeau, C., & Bengio, Y. (2003). Inference for the generalization error. *Machine Learning*, 52(3), 239–281.
- Demšar, J. (2006). Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7, 1–30.
- Gebru, T., et al. (2021). Datasheets for datasets. *Communications of the ACM*, 64(12), 86–92.
- Mitchell, M., et al. (2019). Model cards for model reporting. *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT)**.
- Huyen, C. (2022). *Designing Machine Learning Systems*. O'Reilly.

CHAPTER 3

Advanced, Integrated, Practical and Research-Oriented Topics

The first two chapters of this book built the core of machine learning and deep learning: the learning problem, classical supervised and unsupervised algorithms, model evaluation, neural networks, convolutional and recurrent architectures, transformers and generative models. This final chapter widens the lens. It gathers the mathematical foundations that make those methods work, the engineering infrastructure that makes them usable at scale, the newer learning paradigms that extend them beyond the single static dataset, and the practical and research skills needed to turn them into reliable systems and publishable results.

AT A GLANCE

36 Sections	**42** Figures	**47** Tables
15 Algorithms	**63** Equations	**57** Code Listings

IN THIS CHAPTER YOU WILL LEARN TO

- ✓ apply descriptive and inferential statistics, maximum likelihood and MAP estimation, hypothesis tests and confidence intervals to machine learning problems.
- ✓ use vectors, matrices, eigen-decomposition, the singular value decomposition and multivariable calculus (gradients, Jacobians, Hessians, chain rule) to analyse learning algorithms.
- ✓ reason with random variables, common probability distributions, Bayes' theorem and the central limit theorem, and compute entropy, cross-entropy, KL divergence and mutual information.
- ✓ design data pipelines using ETL/ELT, batch and streaming processing, data lakes, warehouses and lakehouses, and explain how feature stores and data versioning prevent training–serving skew.
- ✓ describe distributed processing with MapReduce and Apache Spark and train models with Spark MLlib.

ABOUT THE AUTHOR

Luxshi Karunakaran
B.Sc. (Hons) in Computer Science and Technology
M.Sc. in Data Science and Artificial Intelligence (Reading)
Department of Social Statistics, University of Sri Jayewardenepura, Sri Lanka